



KONGUNADU COLLEGE OF ENGINEERING AND TECHNOLOGY

(Autonomous)

Approved by AICET, New Delhi & Affiliated to Anna University, Chennai
Accredited by NBA (CSE, ECE, EEE, Mech), NAAC,
Recognized by UGC with 2(f) & 12(B) An ISO 9001:2005 Certified Institution

**Namakkal - Trichy Main Road,
Tholurpatti (post), Thottiam, Trichy – 621215, Tamil Nadu**

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

COURSE MATERIAL

Course Name with Code : 24CS401 / COMPUTER NETWORKS

Course Faculty : Mrs.R.Subashini

Batch : 2024-2028

Year / Semester : II/IV

S.NO	TOPICS COVERED	PAGE NO
UNIT - I		
1	Data Communications	1
2	Networks- Network Types	3
3	Network Models	5
4	TCP /IP Protocol Suite	5
5	The OSI Model	6
6	Digital -to-Digital Conversion	12
7	Line Coding-Line Coding Schemes	12
8	Transmission Modes	17
9	Transmission Media: Guided -Unguided Media	19
UNIT - II		
10	Introduction, Link-Layer Addressing	29
11	Error Detection and Correction	32
12	Introduction-Block Coding	33
13	CRC- Checksum	34
14	Framing- HDLC	34
15	Point-to-Point Protocol	37
16	Media Access Control: Random Access	39
17	Channelization-Wired LAN	42
18	Standard Ethernet	44

19	Connecting Devices - Virtual LANs	48
UNIT - III		
20	Network Layer Services	52
21	Network Layer Performance	53
22	IPV4 addresses-Internet protocol (IP)- ICMPv4	57
23	Unicast Routing Algorithms	67
24	Distance Vector and Link State Routing	67
25	Static Routing	70
26	Dynamic Routing Protocols :RIP and OSPF	71
27	IPV6 Addressing	73
28	IPV6 Protocol	74
29	Software Defined Networks	75
UNIT - IV		
30	Introduction- Transport Layer Protocols	78
31	Simple-Stop and Wait	81
32	Go -back-N	84
33	Selective Repeat- Piggybacking	86
34	UDP -TCP	87
35	SCTP	96
36	Quality of Service	99
37	Data Flow Characteristics	100
38	Flow Control to improve QoS	101
UNIT - V		
39	World Wide Web and Hyper Text Transfer Protocol	103
40	File Transfer Protocol	111
41	NFS	113
42	Electronic Mail	114
43	Telnet- Secure Shell	115
44	Domain name System	120
45	5G Network Utilization	123
46	CoAP	125
47	MQTT	129
48	Content Beyond Syllabus- WiMAX	132

Course Faculty Signature

HOD

UNIT I

NETWORK MODELS AND PHYSICAL LAYER

Data Communications – Networks – Networks Types. Network Models: TCP/IP Protocol suite - The OSI Model. Digital-to-digital conversion: Line coding – Line Coding Schemes – Transmission Modes – Transmission media: Guided – Unguided media.

1.1 Data Communication

In today's digital world, communication between devices has become essential. Whether it is browsing the internet, sending an email, video conferencing, or controlling IoT devices, data communication plays a central role. Data communication refers to the process of transmitting data between two or more devices over some medium such that the receiver interprets it correctly.

A computer network is an interconnected collection of computers and devices that share resources and communicate using standard protocols. Networks allow sharing of information, hardware, software, and applications.

1.1.1 Components of Data Communication

Components: Any communication system consists of five fundamental components:

(i) Message: The message is the information or data to be communicated. Some forms of data representations are text, number, images, audio and video.

(ii) Sender: The sender is a device that sends the message.

(iii) Receiver: The receiver is a device that receives the message, sent by the sender.

(iv) Medium: The medium is a physical path through which the message can be passed between the sender and the receiver.

(v) Protocol: The protocol is a set of rules which governs the data communication. Without the protocol, two systems can be connected but not communicating. The key elements of a protocol are syntax, semantics and timing.

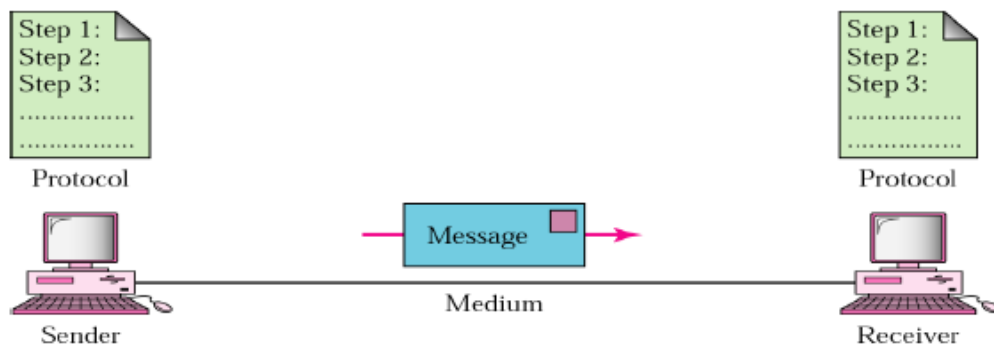


Figure 1.1 Data Communications System

1.1.2 Characteristics of Data Communication

For data communication to be effective, the system must exhibit the following characteristics:

Delivery – The data must be delivered to the correct destination.

Accuracy – Data should be received without alteration.

Timeliness – Data must be delivered within an acceptable time frame.

Jitter – Variation in packet arrival times should be minimized.

1.1.3 Direction of Data Flow

Line configuration refers to the attachment of communication devices to a link. There are two types of line configurations:

(i) Point-to-point: Provides a dedicated link between two devices. The entire channel capacity is reserved for the transmission between two devices only.

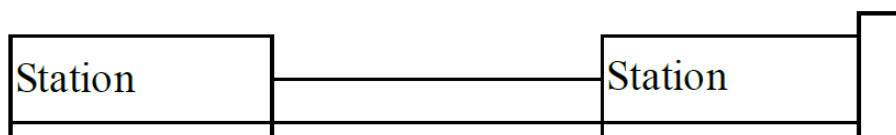


Figure 1.2 Point-to-point connection

(ii) Multipoint: More than two specific devices share a single link. The channel capacity is shared either spatially or temporally.

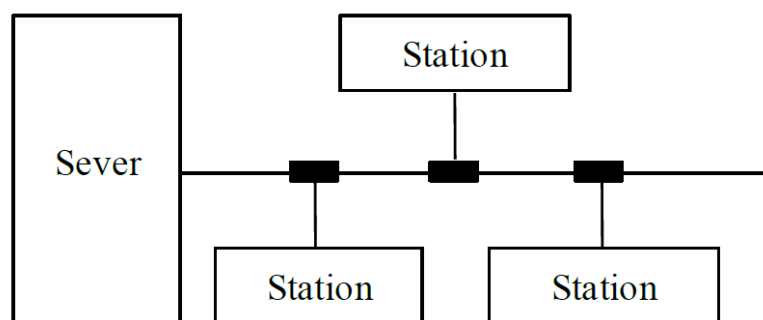


Figure 1.3 Multipoint connection

Communication between two devices can be of three types. They are;

1) Simple: The communication is unidirectional. Only one of the two stations on a link can transmit and other can only receive.

2) Half-duplex: Each station can both transmit and receive, but not at the same time. The entire capacity of the channel is taken by the station which transmits the data.

3) Full-duplex: Both stations can transmit and receive the data at the same time. The capacity of the channel is divided between the signals traveling in opposite directions.

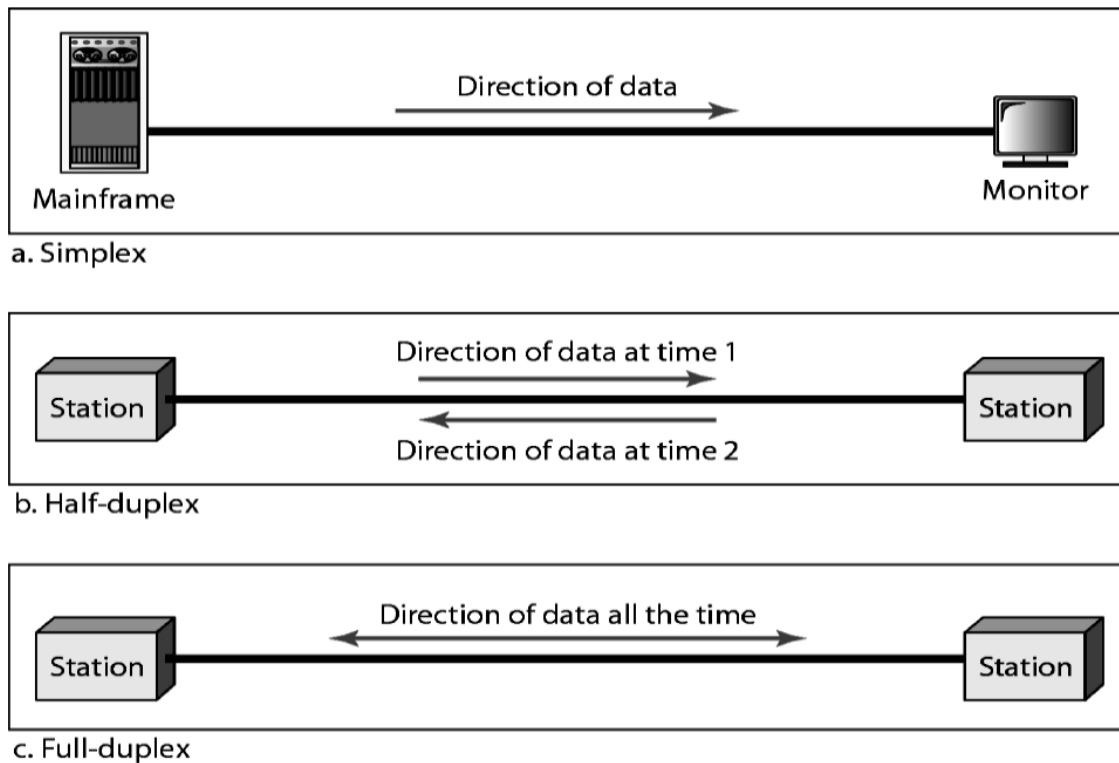


Figure 1.4 Data flow (simplex, half-duplex, and full-duplex)

1.1.4 Networks

A network must be able to meet a certain number of criteria. The most important of these are performance, reliability, and security.

Performance

Performance can be measured in many ways, including transit time and response time. Transit time is the amount of time required for a message to travel from one device to another. Response time is the elapsed time between an inquiry and a response. The performance of a network depends on a number of factors, including the number of users, the type of transmission medium, the capabilities of the connected hardware, and the efficiency of the software. Performance is often evaluated by two networking metrics: throughput and delay. We often need more throughput and less delay. However, these two criteria are often contradictory. If we try to send more data to the network, we may increase throughput but we increase the delay because of traffic congestion in the network.

Reliability: In addition to accuracy of delivery, network reliability is measured by the frequency of failure, the time it takes a link to recover from a failure, and the network's robustness in a catastrophe.

Security: Network security issues include protecting data from unauthorized access, protecting data from damage and development, and implementing policies and procedures for recovery from breaches and data losses.

1.1.5 Network Types

There are three primary categories of networks. They are,

(i) LAN (Local Area Network): It is normally private and will connect the computer in a small area such as the entire computer in single company or building. The network at your business or school is an example of a LAN.

(ii) MAN (Metropolitan Area Network): It is a "LAN" that has been extended so that it covers a larger area such as an entire city. Your ISP is an example of a MAN.

(iii) WAN (Wide Area Network): It is a "LAN" that has been extended to cover a wider area such as multiple sites around the world, an entire country, or even the whole world. It may be private in that it connects all the sites within a single company or it may be public such as the network of computers that make up the network for running all the Google related sites around the world.

(iv) WLAN (Wireless Local Area Network) : A Wireless Local Area Network (WLAN) is a local area network that uses wireless communication—usually radio waves—to connect devices within a limited area like a building, office, or campus, enabling users to maintain mobility while staying connected to the network. WLANs commonly use the IEEE 802.11 standard (Wi-Fi)

(v) SAN :

- **Storage Area Network:** A dedicated high-speed network designed to provide access to consolidated block-level storage, typically used by enterprises to connect servers with storage devices, enhancing storage availability and performance
- **System Area Network:** A high-performance network connecting clusters of computers for tasks like supercomputing; focuses on internal communication, not user-facing connections.
- **Server Area Network:** Refers generally to networks built for interconnecting servers, although this usage is rare.
- **Small Area Network:** Sometimes used to describe smaller local networks, but is less common.

(vi) CAN :

- **Campus Area Network:** Connects multiple LANs within a limited geographical area, such as a university campus, allowing departments and buildings to share resources and communicate efficiently.

- **Controller Area Network:** A robust bus standard designed for microcontrollers and devices to communicate without a host computer, primarily used in automotive and industrial applications.
- **Cluster Area Network:** Used in computing clusters, where computers are networked to work as a single system for tasks requiring high performance.

(vii) PAN (Personal Area Network)

A **Personal Area Network (PAN)** is a small network designed to connect devices surrounding an individual, typically within a few meters. PANs use technologies like Bluetooth, ZigBee, or Infrared and are used for connecting devices such as smartphones, smartwatches, wireless headphones, and fitness trackers.

NETWORK MODELS

1.2 TCP/IP PROTOCOL SUITE

TCP/IP is a protocol-suite used in the Internet today. Protocol-suite refers a set of protocols organized in different layers. It is a hierarchical protocol made up of interactive modules, each of which provides a specific functionality. The term hierarchical means that each upper-level protocol is supported by the services provided by one or more lower-level protocols.

1.2.1 Layered Architecture

- (i) Source host(computer A)
- (ii) Link-layer switch in link 1
- (iii) Router
- (iv) Link-layer switch in link 2
- (v) Destination host (computer B)

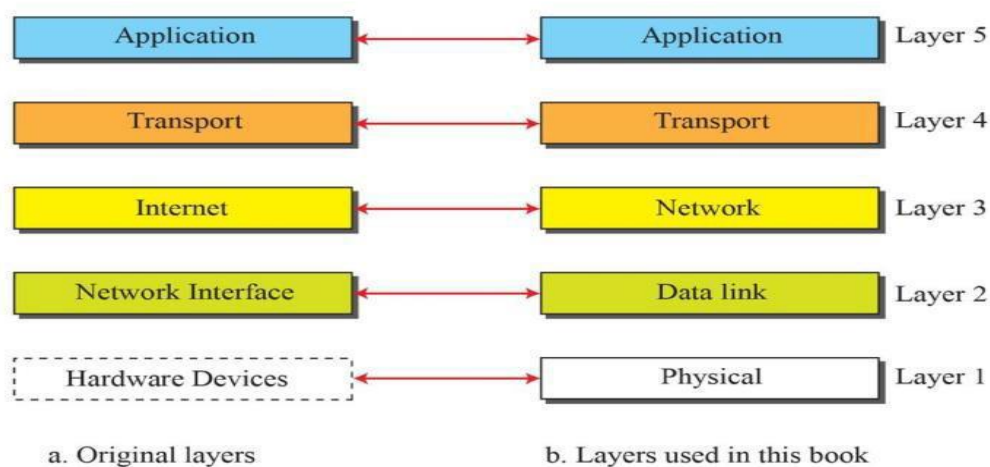


Figure 1.11 Layers in the TCP/IP protocol suite

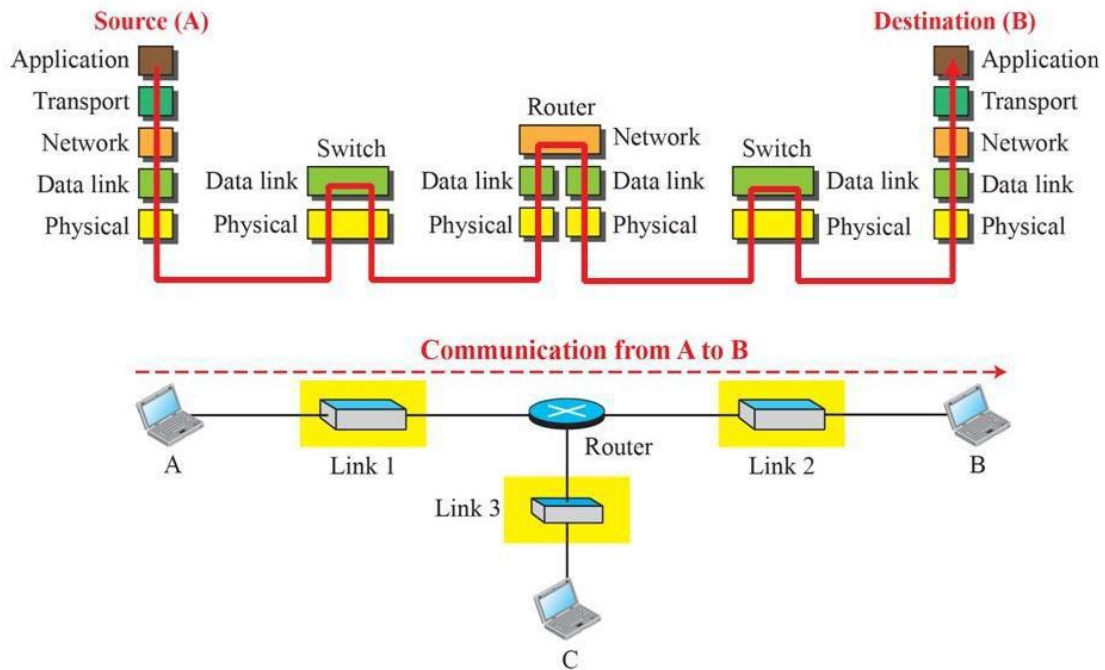


Figure 1.12 Communication through an Internet

1.2.2 Layers in the TCP/IP Protocol Suite

However, the duty of the data-link and physical layers is hop-to-hop. A hop is a host or router. The domain of duty of the top three layers is the internet. The domain of duty of the two lower layers is the link. In top 3 layers, the data unit should not be changed by any router or link- layer switch.

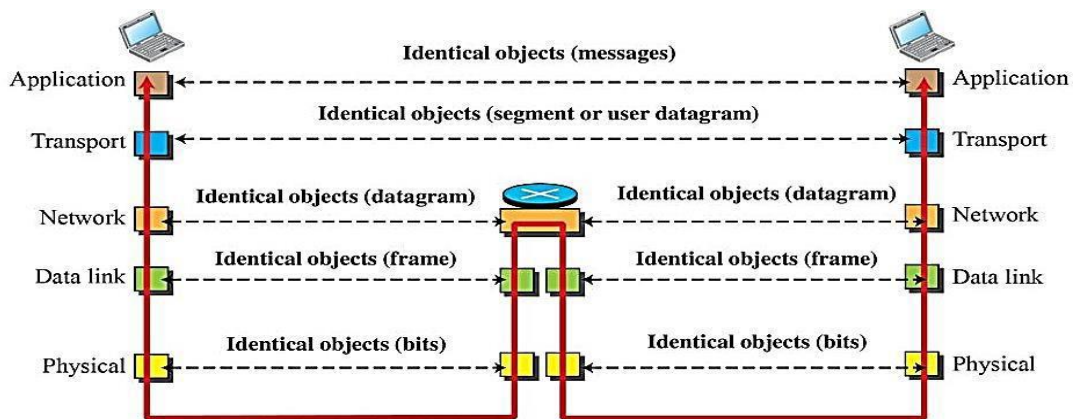


Figure 1.14 Identical objects in the TCP/IP protocol suite

1.3The OSI Model

The OSI Model (Open Systems Interconnection Model) is a conceptual framework used to describe the functions of a networking system. The OSI model characterizes computing

functions into a universal set of rules and requirements in order to support interoperability between different products and software.

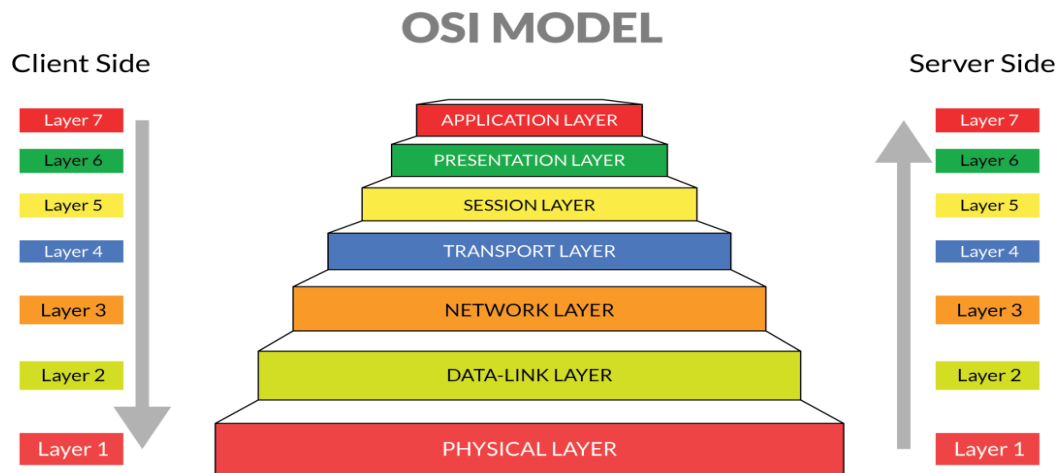


Figure 1.15 OSI Model – 7 Layers

In the OSI reference model, the communications between a computing system are split into seven different abstraction layers:

1. Physical,
2. Data Link,
3. Network,
4. Transport,
5. Session,
6. Presentation,
7. Application.

OSI (Open Source Interconnection) 7 Layer Model					
Layer	Application/Example	Central Device/Protocols	DOD4 Model		
Application (7) Serves as the window for users and application processes to access the network services.	End User layer Program that opens what was sent or creates what is to be sent Resource sharing • Remote file access • Remote printer access • Directory services • Network management	User Applications SMTP	GATEWAY	Can be used on all layers	Process
Presentation (6) Formats the data to be presented to the Application layer. It can be viewed as the "Translator" for the network.	Syntax layer encrypt & decrypt (if needed) Character code translation • Data conversion • Data compression • Data encryption • Character Set Translation	JPEG/ASCII EBDIC/TIFF/GIF PICT			
Session (5) Allows session establishment between processes running on different stations.	Synch & send to ports (logical ports) Session establishment, maintenance and termination • Session support - perform security, name recognition, logging, etc.	Logical Ports RPC/SQL/NFS NetBIOS names			
Transport (4) Ensures that messages are delivered error-free, in sequence, and with no losses or duplications.	TCP Host to Host, Flow Control Message segmentation • Message acknowledgement • Message traffic control • Session multiplexing	PACKET FILTERING TCP/SPX/UDP	Host to Host		Internet
Network (3) Controls the operations of the subnet, deciding which physical path the data takes.	Packets ("letter", contains IP address) Routing • Subnet traffic control • Frame fragmentation • Logical-physical address mapping • Subnet usage accounting				
Data Link (2) Provides error-free transfer of data frames from one node to another over the Physical layer.	Frames ("envelopes", contains MAC address) [NIC card — Switch — NIC card] (end to end) Establishes & terminates the logical link between nodes • Frame traffic control • Frame sequencing • Frame acknowledgment • Frame delimiting • Frame error checking • Media access control	Switch Bridge WAP PPP/SLIP	Land Based Layers		Network
Physical (1) Concerned with the transmission and reception of the unstructured raw bit stream over the physical medium.	Physical structure Cables, hubs, etc. Data Encoding • Physical medium attachment • Transmission technique • Baseband or Broadband • Physical medium transmission Bits & Volts	Hub			

Figure 1.16 OSI 7 Layer Model with functions and protocols

1.2.2.1 Description of Each Layer

Physical Layer

The physical layer receives bits from the data-link layer and sends through the transmission media. The physical layer is responsible for movements of individual bits from one node to another node. Transmission media is another hidden layer under the physical layer. Two devices are connected by a transmission medium (cable or air). The transmission medium does not carry bits; it carries electrical or optical signals.

Data Link Layer

Data-link-layer (DLL) is responsible for moving frames from one node to another node over a link. The link can be wired LAN/WAN or wireless LAN/WAN. The data-link layer

- Gets the datagram from network layer
- Encapsulates the datagram in a packet called a frame.
- Sends the frame to physical layer.

TCP/IP model does not define any specific protocol. DLL supports all the standard and proprietary protocols. Each protocol may provide a different service. Some protocols provide complete error detection and correction; some protocols provide only error correction.

Network Layer

The network layer is responsible for source-to-destination transmission of data. The network layer is also responsible for routing the packet. The routers choose the best route for each packet. Why we need the separate network layer?

- The separation of different tasks between different layers
- The routers do not need the application and transport layers.
- TCP/IP model defines 4 protocols. They are;

(i) IP (Internetworking Protocol)

- IP is the main protocol of the network layer.
- IP defines the format and the structure of addresses.
- IP is also responsible for routing a packet from its source to its destination.
- It provides a best-effort delivery service.
- Best effort means IP does its best to get the packet to its destination, but with no guarantees.
- IP does not provide flow control, error control and congestion control services.
- If an application requires above services, the application should rely only on the transport-layer protocol.

(ii) ARP

- ARP is used to find the physical-address of the node when its Internet-address is known.
- Physical address is the 48-bit address that is imprinted on the NIC or LAN card.
- Internet address (IP address) is used to uniquely & universally identify a device in the internet.

(iii) ICMP

- ICMP is used to inform the sender about datagram-problems that occur during transit.

(iv) IGMP

- IGMP is used to send the same message to a group of recipients.

Transport Layer

Transport Layer protocols are responsible for delivery of a message from a process to another process. The transport layer gets the message from the application layer and encapsulates the message in a packet called a segment then sends the segment to network layer. TCP/IP model defines 3 protocols for transport layer;

(i) TCP (Transmission Control Protocol)

(ii) UDP (User Datagram Protocol)

(iii) SCTP (Stream Control Transmission Protocol)

(i) TCP

- TCP is a reliable connection-oriented protocol.
- A connection is established b/w the sender and receiver before the data can be transmitted.
- TCP provides flow control, error control and congestion control services.

(ii) UDP

- UDP is the simplest of the 3 transport protocols.
- It is an unreliable, connectionless protocol.
- It does not provide flow, error, or congestion control.

(iii) SCTP

- SCTP provides support for newer applications such as voice over the Internet, it combines the best features of UDP and TCP.

Application Layer

The two application layers exchange messages between each other. Communication at the application layer is between two processes (two programs running at this layer). To communicate, a process sends a request to the other process and receives a response. Process-to-process communication is the duty of the application layer. TCP/IP model defines following protocols;

- (i) FTP (FileTransfer Protocol)
- (ii) SMTP (Simple Mail Transfer Protocol))
- (iii) DNS (Domain Name System)
- (iv) HTTP (Hyper Text Transfer Protocol)
- (v) SNMP (Simple NetworkManagement Protocol)
- (vi) TELNET (Terminal Network)

- SMTP is used to transport email between a source and destination.
- TELNET is used for accessing a site remotely.
- FTP is used for transferring files from one host to another.
- DNS is used to find the IP address of a computer.
- SNMP is used to manage the Internet at global and local levels.
- HTTP is used for accessing the World Wide Web (WWW).

1.2.3 Encapsulation and Decapsulation

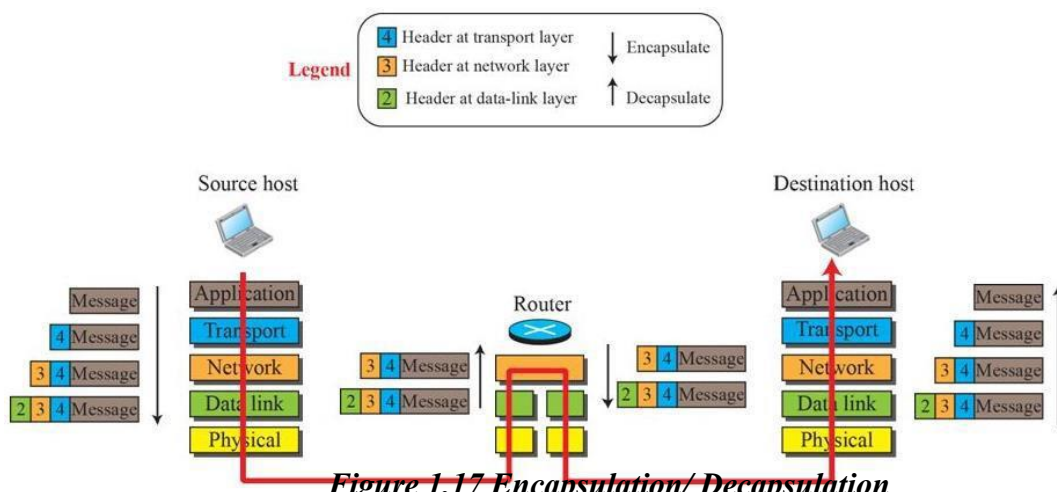


Figure 1.17 Encapsulation/ Decapsulation

Encapsulation at the Source Host (Figure 1.17)

At the application layer, the data to be exchanged is referred to as a message. A message normally does not contain any header or trailer. The message is passed to the transport layer.

The transport layer takes the message as the payload. Transport layer adds its own header to the payload. The header contains identifiers of the source and destination application programs and information needed for flow, error control, or congestion control. The transport-layer packet is called the segment (in TCP) and the user datagram (in UDP). The segment is passed to the network layer.

Decapsulation and Encapsulation at the Router

At the router, we have both encapsulation & decapsulation and because the router is connected to two or more links.

Data-link layer

Receives frame from physical layer, decapsulates the datagram from the frame and passes the datagram to the network layer.

The network layer

Inspects the source and destination addresses in the datagram header and Consults forwarding table to find next hop to which the datagram is to be delivered. The datagram is then passed to the data-link layer of the next link. The data-link layer of the next link Encapsulates the datagram in a frame and Passes the frame to the physical layer for transmission.

Decapsulation at the Destination Host

At the destination host, each layer decapsulates the packet received from lower layer and removes the payload then delivers the payload to the next-higher layer

1.2.4 Addressing

We have logical communication between pairs of layers. Any communication that involves 2 parties needs 2 addresses: source address and destination address.

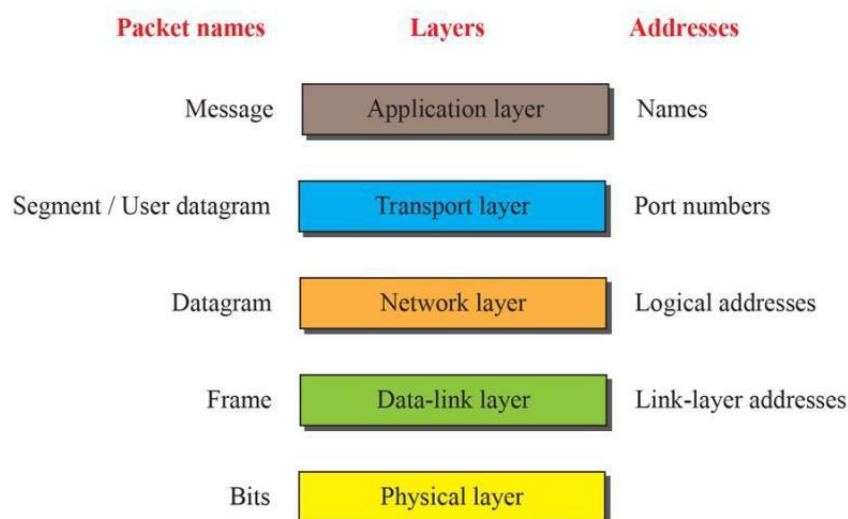


Figure 1.18 Addressing in the TCP/IP protocol suite

1.2.5 Multiplexing and Demultiplexing

Multiplexing means a protocol at a layer can encapsulate a packet from several next- higher layer protocols (one at a time). Demultiplexing means a protocol can decapsulates and deliver a packet to several next-higher layer protocols (one at a time)

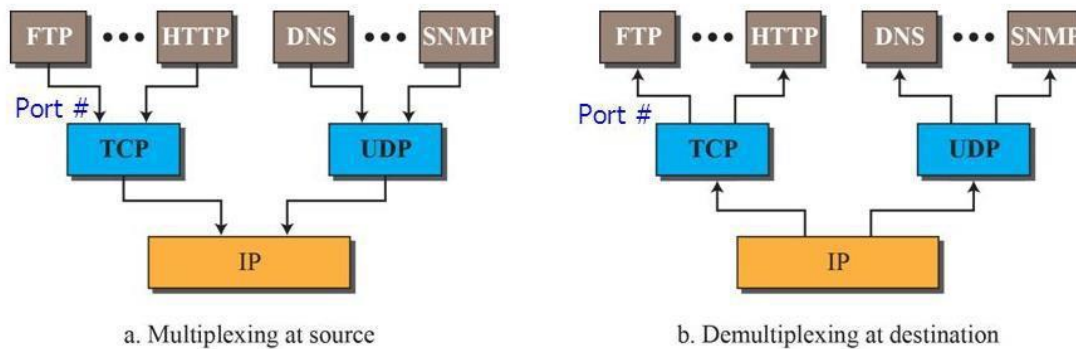


Figure 1.19 Multiplexing and Demultiplexing

1.4 Digital Transmission

Data can be represented either in analog or digital form. The computers used the digital form to store the information. Therefore, the data needs to be converted in digital form so that it can be used by a computer.

1.4.1 Digital-To-Digital Conversion

Digital-to-digital encoding is the representation of digital information by a digital signal. When binary 1s and 0s generated by the computer are translated into a sequence of voltage pulses that can be propagated over a wire, this process is known as digital-to-digital encoding.

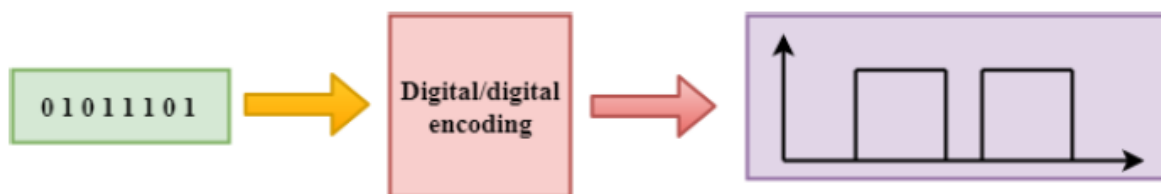


Figure 1.23 Digital to Digital Encoding

1.4.3 Line Coding Schemes

The line coding process is a fundamental step in digital communication systems, where binary signals are prepared for reliable transmission over physical media. In this process, a sequence of binary information (comprising 1s and 0s) is transformed into a specific digital code suitable for transmission. Within baseband digital transmission, the electrical waveforms employed are in the form of pulses, and the transformation of digital data into digital waveforms is precisely what is referred to as line coding. Essentially, line coding schemes define the rules

for how bits are mapped into signal waveforms, thereby determining how efficiently and accurately information can be transported across a medium.

Depending on the method of representation, line coding techniques are broadly classified into three categories:

- (i) Unipolar schemes (such as Non-Return-to-Zero, NRZ)
- (ii) Polar schemes (including NRZ-Level, NRZ-Invert, Return-to-Zero, and Biphase codes like Manchester and Differential Manchester)
- (iii) Bipolar schemes (such as Alternate Mark Inversion, AMI, and Pseudoternary). Each of these schemes has its own characteristics, advantages, and limitations, making them suitable for different communication scenarios.

Unipolar Encoding

Digital transmission system sends the voltage pulses over the medium link such as wire or cable. In most types of encoding, one voltage level represents 0, and another voltage level represents 1. The polarity of each pulse determines whether it is positive or negative. This type of encoding is known as Unipolar encoding as it uses only one polarity. In Unipolar encoding, the polarity is assigned to the 1 binary state. In this, 1s are represented as a positive value and 0s are represented as a zero value. In Unipolar Encoding, '1' is considered as a high voltage and '0' is considered as a zero voltage. Unipolar encoding is simpler and inexpensive to implement.

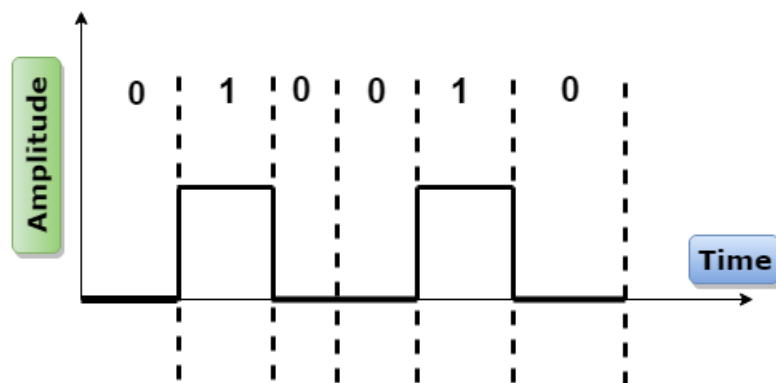


Figure 1.25 Unipolar Encoding

Unipolar encoding has two problems that make this scheme less desirable:

- o DC Component
- o Synchronization

Polar Encoding

Polar encoding is an encoding scheme that uses two voltage levels: one is positive, and another is negative. By using two voltage levels, an average voltage level is reduced, and the DC component problem of unipolar encoding scheme is alleviated.

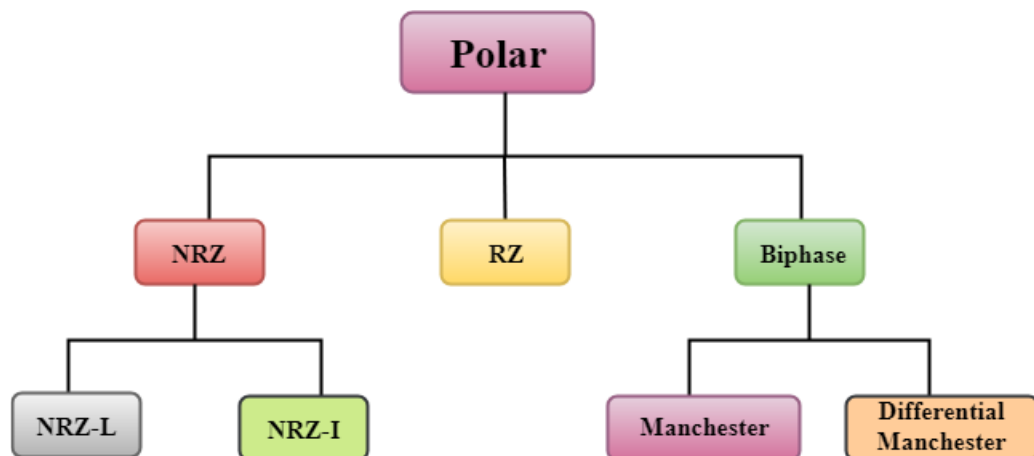


Figure 1.26 Types of Polar Encoding

NRZ

o NRZ stands for non-return zero, In NRZ encoding, the level of the signal can be represented either positive or negative.

The two most common methods used in NRZ are:

NRZ-L: In NRZ-L encoding, the level of the signal depends on the type of the bit that it represents. If a bit is 0 or 1, then their voltages will be positive and negative respectively. Therefore, we can say that the level of the signal is dependent on the state of the bit.

NRZ-I: NRZ-I is an inversion of the voltage level that represents 1 bit. In the NRZ-I encoding scheme, a transition occurs between the positive and negative voltage that represents 1 bit. In this scheme, 0 bit represents no change and 1 bit represents a change in voltage level.

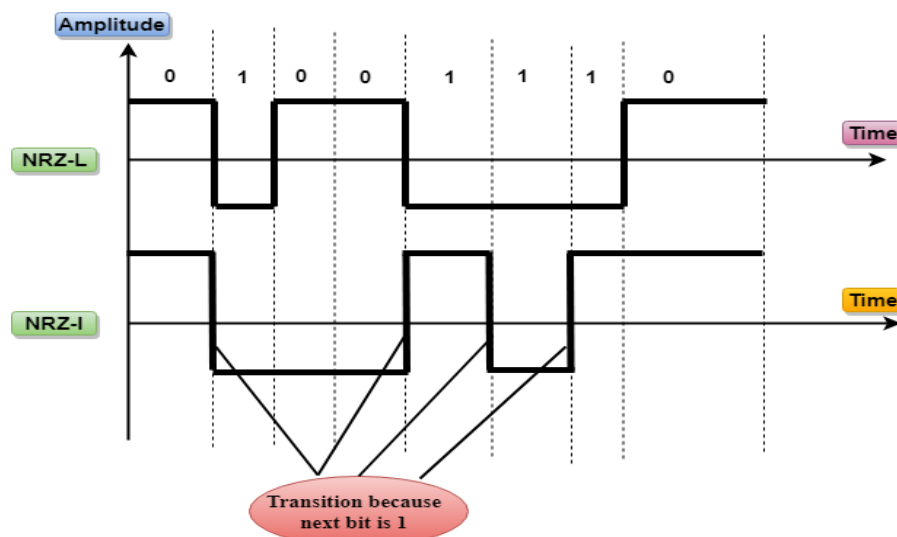


Figure 1.27 NRZ Line Coding Schemes: NRZ-L vs NRZ-I

RZ

o RZ stands for Return to zero. There must be a signal change for each bit to achieve synchronization. However, to change with every bit, we need to have three values: positive, negative and zero. RZ is an encoding scheme that provides three values, positive voltage represents 1, the negative voltage represents 0, and zero voltage represents none. In the RZ scheme, halfway through each interval, the signal returns to zero. In RZ scheme, 1 bit is represented by positive-to-zero and 0 bit is represented by negative-to-zero.

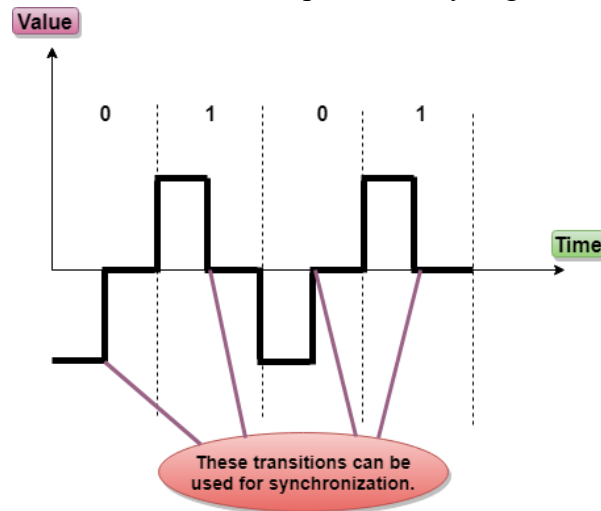


Figure 1.28 RZ Line Coding Schemes

Disadvantage of RZ:

It performs two signal changes to encode one bit that acquires more bandwidth.

Biphase

Biphase is an encoding scheme in which signal changes at the middle of the bit interval but does not return to zero. Biphase encoding is implemented in two different ways:

Manchester

It changes the signal at the middle of the bit interval but does not return to zero for synchronization. In Manchester encoding, a negative-to-positive transition represents binary 1, and positive-to-negative transition represents 0. Manchester has the same level of synchronization as RZ scheme except that it has two levels of amplitude.

Differential Manchester

It changes the signal at the middle of the bit interval for synchronization, but the presence or absence of the transition at the beginning of the interval determines the bit. A transition means binary 0 and no transition means binary 1. o In Manchester Encoding scheme, two signal changes represent 0 and one signal change represent 1.

Bipolar : Bipolar encoding scheme represents three voltage levels: positive, negative, and zero. In Bipolar encoding scheme, zero level represents binary 0, and binary 1 is represented by alternating positive and negative voltages. If the first 1 bit is represented by positive

amplitude, then the second 1 bit is represented by negative voltage, third 1 bit is represented by the positive amplitude and so on. This alternation can also occur even when the 1 bits are not consecutive.

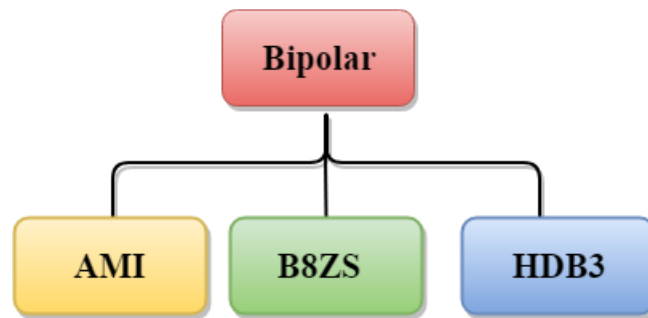


Figure 1.30 Bipolar Line Coding and its Variants (AMI, B8ZS, HDB3)

AMI

AMI stands for alternate mark inversion where mark work comes from telegraphy which means 1. So, it can be redefined as alternate 1 inversion. In Bipolar AMI encoding scheme, 0 bit is represented by zero level and 1 bit is represented by alternating positive and negative voltages.

Advantage:

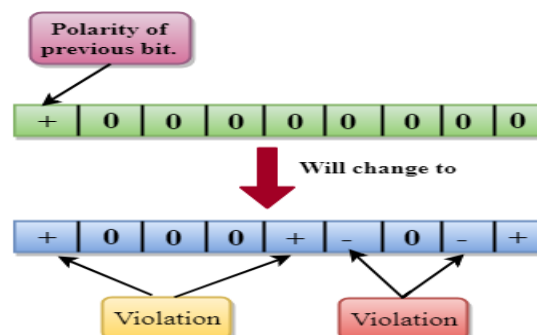
- DC component is zero.
- Sequence of 1s bits are synchronized.

Disadvantage:

- This encoding scheme does not ensure the synchronization of a long string of 0s bits.

B8ZS

- B8ZS stands for Bipolar 8-Zero Substitution. This technique is adopted in North America to provide synchronization of a long sequence of 0s bits. In most of the cases, the functionality of B8ZS is similar to the bipolar AMI, but the only difference is that it provides the synchronization when a long sequence of 0s bits occur.
- If the polarity of previous 1 bit is negative, then the eight 0s will be encoded as zero, zero, zero, negative, positive, zero, positive, negative.



HDB3

- HDB3 stands for **High-Density Bipolar 3**. HDB3 technique was first adopted in Europe and Japan. HDB3 technique is designed to provide the synchronization of a long sequence of 0s bits. In the HDB3 technique, the pattern of violation is based on the polarity of the previous bit. When four 0s occur, HDB3 looks at the number of 1s bits occurred since the last substitution.
- If the number of 1s bits is odd, then the violation is made on the fourth consecutive of 0. If the polarity of the previous bit is positive, then the violation is positive. If the polarity of the previous bit is negative, then the violation is negative.

1.4.4 Transmission Modes

Transmission mode, also called communication mode, refers to the method of transferring data between two devices over a communication channel such as optical fiber, copper wire, wireless medium, or storage media. Data travels in the form of electromagnetic waves, and depending on the method used, it can be delivered as a sequence of pulses through digital modulation techniques.

The data transmission mode defines the direction in which information flows between communicating devices, which is why it is also known as data communication mode or sometimes as directional mode.

Within the OSI model, the Physical Layer is responsible for data transmission. It determines the direction of data flow so that information successfully reaches the receiving device through the chosen medium.

Transmission modes are classified into three types:

- Simplex
- Half Duplex
- Full Duplex

1. Simplex Mode

In simplex transmission, data flows in one direction only—from sender to receiver. The channel is either transmit-only or receive-only, much like a one-way street where traffic moves in a single direction.



Figure 1.33 Simplex Transmission Mode

Examples:

- Keyboard to computer (keyboard sends input, monitor only displays output).
- Radio broadcasting (signal flows from the station to listeners, but not back).

Advantages:

- The full bandwidth of the channel is dedicated to transmission.
- Large amounts of data can be sent without loss.

Disadvantages:

- Communication is unidirectional, so no feedback or response is possible.
- Less suitable where interactive communication is required.

2. Half Duplex Mode

In half duplex transmission, data can move in both directions, but only one side at a time. While one device transmits, the other must wait, and roles can switch afterward.

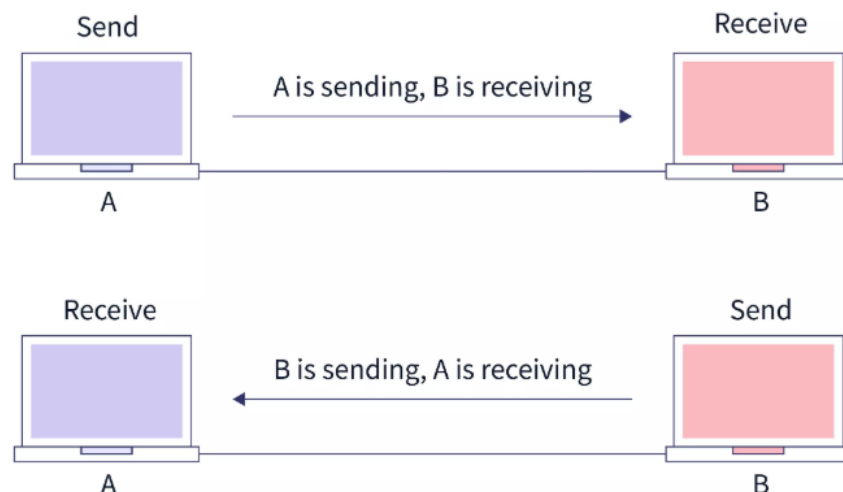


Figure 1.34 Half Duplex Transmission Mode

Examples:

- Walkie-talkie communication (one person speaks, the other listens, then they switch).

Advantages:

- Supports two-way communication.
- Error detection and retransmission are possible.
- Entire channel bandwidth is used in one direction at a time.

Disadvantages:

- Devices must take turns, causing delays.
- Not suitable for real-time two-way interaction.

3. Full Duplex Mode

In full duplex transmission, data flows simultaneously in both directions. The communication channel is divided into two separate paths (essentially two simplex links), allowing sender and receiver to exchange information at the same time.

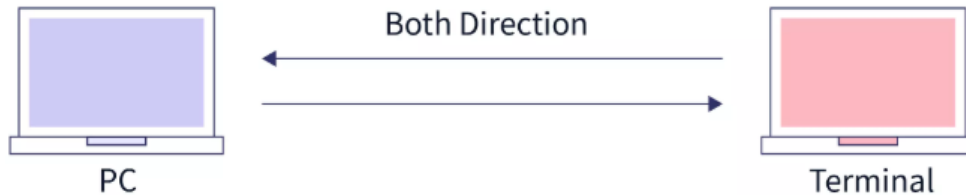


Figure 1.35 Half Duplex Transmission Mode

Examples:

- Telephone systems (both parties can talk and listen simultaneously).
- Messaging apps (sending and receiving occur concurrently).

Advantages:

- Fastest mode since transmission happens in both directions at once.
- Maximum efficiency with real-time communication.

Disadvantages:

- If no dedicated separate path exists, the channel's capacity is split, reducing speed per direction.

1.5 TRANSMISSION MEDIAS

A transmission medium can be broadly defined as anything that can carry information from a source to a destination. In data communications the definition of the information and the transmission medium is more specific. The transmission medium is usually free space, metallic cable, or fiber-optic cable. The information is usually a signal that is the result of a conversion of data from another form.

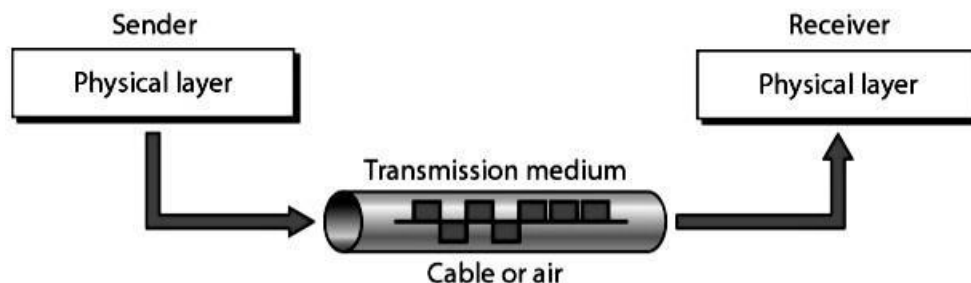


Figure 1.36 Transmission medium

Transmission media can be divided into two broad categories: Guided medium and unguided medium.

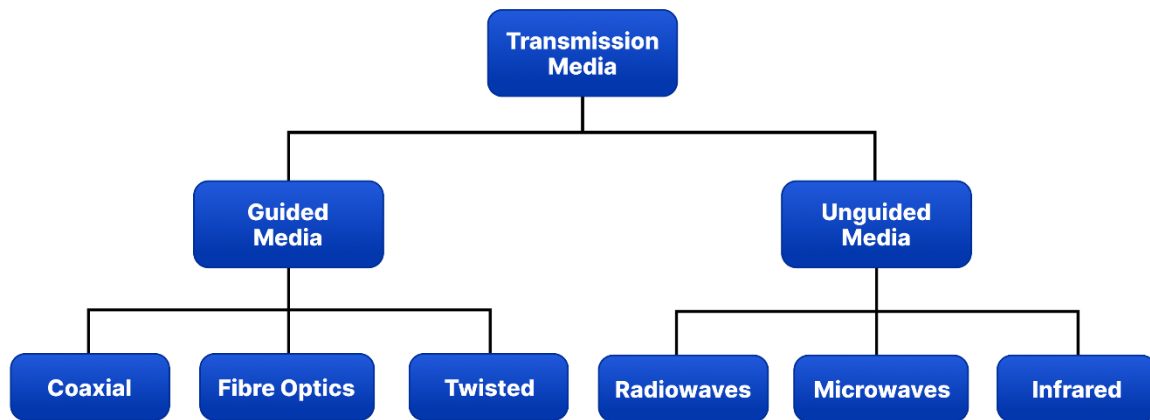


Figure 1.37 Types of Transmission media

1.5.1 Guided Media

Guided media provide a conduit from one device to another. A signal traveling along any of these media is directed and contained by the physical limits of the medium. Twisted-pair and coaxial cable use metallic (copper) conductors that accept and transport signals in the form of electric current. Optical fiber is a cable that accepts and transports signals in the form of light.

1.5.1.1 Twisted-Pair Cable

Twisting makes it probable that both wires are equally affected by external influences. The number of twists per unit of length has some effect on the quality of the cable.

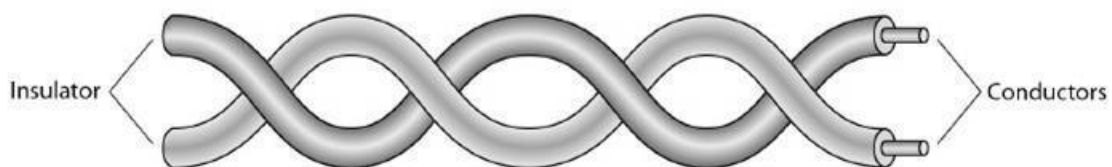


Figure 1.38 Twisted-pair cable

Unshielded Versus Shielded Twisted-Pair Cable

The most common twisted-pair cable used in communications is referred to as unshielded twisted-pair (UTP). IBM has also produced a version of twisted-pair cable for its use called shielded twisted-pair (STP). STP cable has a metal foil or braided mesh covering that encases each pair of insulated conductors. Metal casing improves the quality of cable by preventing the penetration of noise or crosstalk. It is bulkier and more expensive.

Connectors

The most common UTP connector is RJ45 (RJ stands for registered jack). The RJ45 is a keyed connector, meaning the connector can be inserted in only one way.

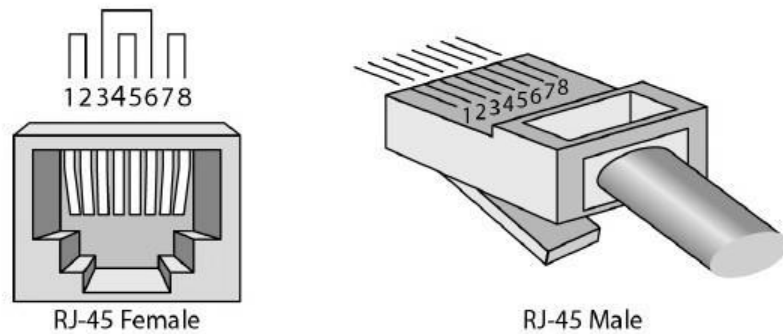


Figure 1.39 UTP connector

Categories of Unshielded Twisted-Pair Cable

The Electronic Industries Association (EIA) has developed standards to classify unshielded twisted-pair cable into seven categories.

Category	Specification	Data Rate (Mbps)	Use
1	Unshielded twisted-pair used in telephone	<0.1	Telephone
2	Unshielded twisted-pair originally used in T-lines	2	T-1 lines
3	Improved CAT-2 used in LANs	10	LANs
4	Improved CAT 3 used in Token Ring networks	20	LANs
5	Cable wire is normally 24 AWG with a jacket and outside sheath	100	LANs
5E	An extension to category 5 that includes extra features to minimize the crosstalk and electromagnetic interference	125	LANs
6	A new category with matched components coming from the same manufacturer. The cable must be tested at a 200Mbps data rate.	200	LANs
7	Sometimes called SSTP (shielded screen twisted-pair). Each pair is individually wrapped in a helical metallic foil followed by a metallic foil shield in addition to the outside sheath. The shield decreases the effect of crosstalk and increases the data rate.	600	LANs

Table 1.1 Categories of unshielded twisted-pair cables

Applications

- a) Twisted-pair cables are used in telephone lines to provide voice and data channels.

b) Local-area networks, such as 10Base-T and 100Base-T, also use twisted-pair cables.

Performance

A twisted-pair cable can pass a wide range of frequencies. With increasing frequency, the attenuation, measured in decibels per kilometer (dB/km), sharply increases with frequencies above 100 kHz. Gauge is a measure of the thickness of the wire.

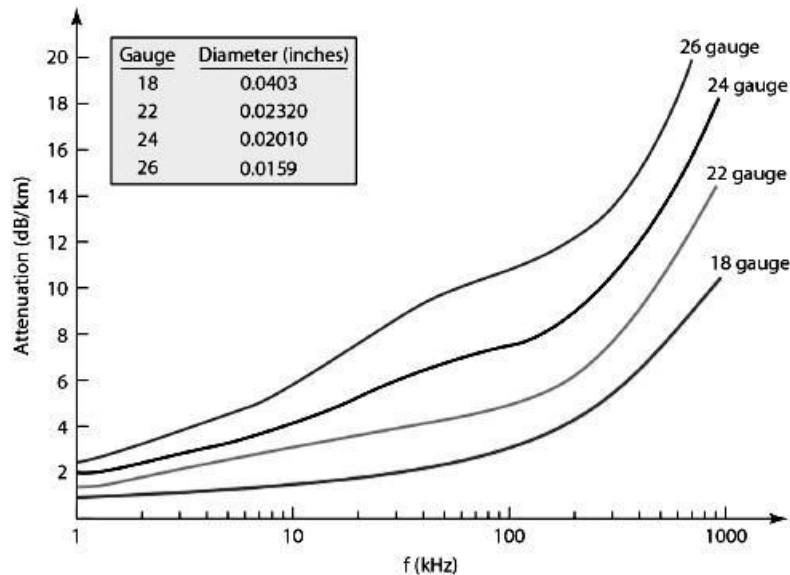


Figure 1.40 UTP Cable – Performance

1.5.1.2 Coaxial Cable

Coaxial cable carries signals of higher frequency ranges. Instead of having two wires, coax has a central core conductor of solid or stranded wire (usually copper) enclosed in an insulating sheath, which is, in turn, encased in an outer conductor of metal foil, braid, or a combination of the two. The outer metallic wrapping serves both as a shield against noise and as the second conductor, which completes the circuit. This outer conductor is also enclosed in an insulating sheath, and the whole cable is protected by a plastic cover.

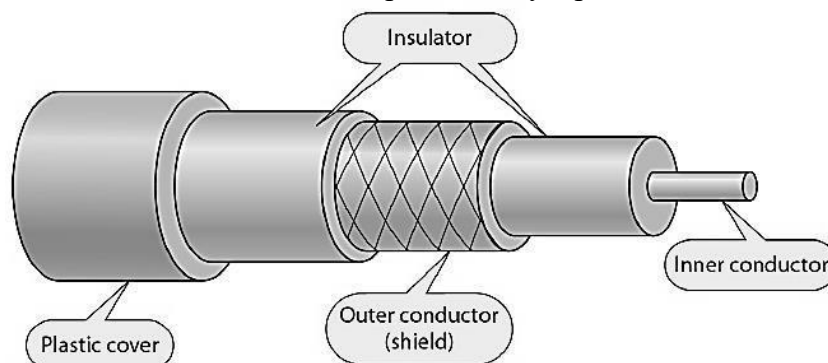


Figure 1.41 Coaxial cable

Coaxial Cable Standards

Coaxial cables are categorized by their radio government (RG) ratings. Each RG number denotes a unique set of physical specifications, including the wire gauge of the inner conductor, the thickness and type of the inner insulator, the construction of the shield, and the size and type of the outer casing. Each cable defined by an RG rating is adapted for a specialized function.

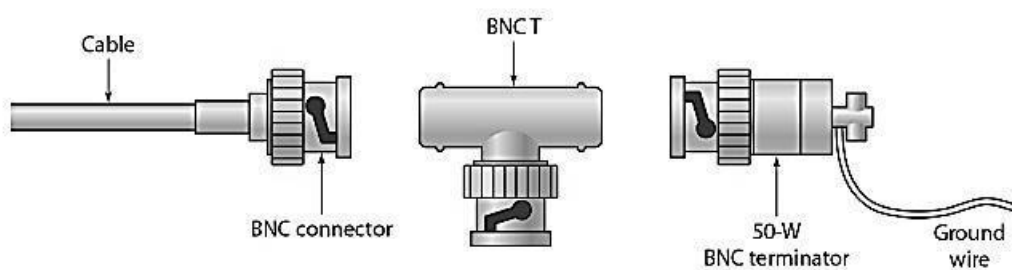
Category	Impedance	Use
RG – 59	75 Ω	Cable TV
RG – 58	50 Ω	Thin Ethernet
RG – 11	50 Ω	Thick Ethernet

Table 1.2 Categories of coaxial cables

Coaxial Cable Connectors

To connect coaxial cable to devices, we need coaxial connectors. The most common type of connector used today is the Bayone-Neill-Concelman (BNC) connector. Three popular types of connectors:

- BNC connector - used to connect the end of the cable to a device, such as a TV set.
- BNC T connector - The BNC T connector is used in Ethernet networks to branch out a connection to a computer or other device
- BNC terminator - The BNC terminator is used at the end of the cable to prevent the reflection of the signal.



(a) BNC Connector (b) BNCT Connector (c) BNC Terminator

Figure 1.42 BNC connectors

Applications of the coaxial cable

- Cable TV
- Telecommunication
- Traditional Ethernet LANs

Performance of the coaxial cable

The attenuation is much higher in coaxial cables than in twisted-pair cable. Although coaxial cable has a much higher bandwidth, the signal weakens rapidly and requires the frequent use of repeaters.

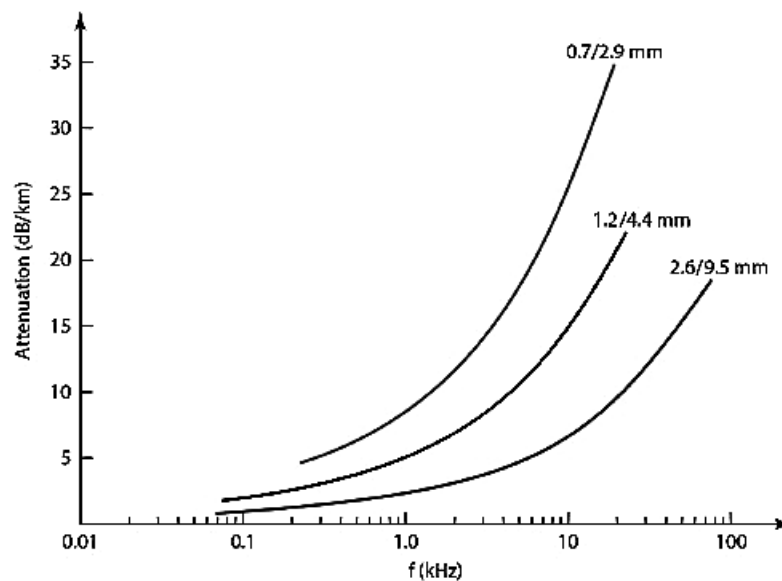
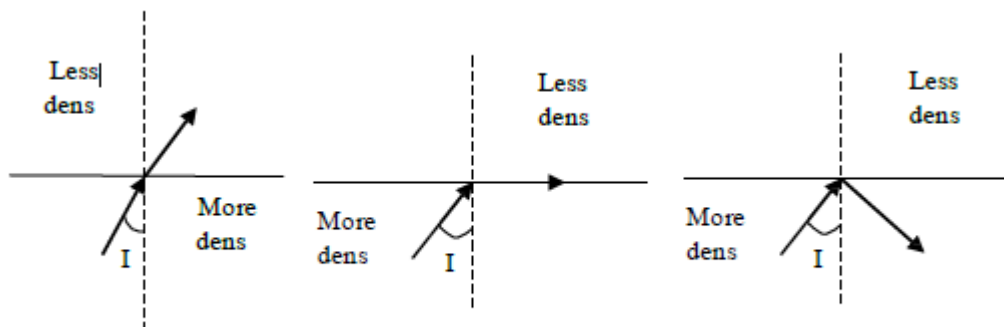


Figure 1.43 Performance of the coaxial cable

1.5.1.3 Fiber-Optic Cable

A fiber-optic cable is made of glass or plastic and transmits signals in the form of light. Light travels in a straight line as long as it is moving through a single uniform substance. If a ray of light traveling through one substance suddenly enters another substance the ray changes direction.



I < Critical angle, refraction I = Critical angle, refraction I > Critical angle, refraction

Figure 1.44 Bending of light ray

Bending of Light

- If the angle of incidence is less than the critical angle, the ray refracts and moves closer to the surface.
- If the angle of incidence is equal to the critical angle, the light bends along the interface.

c) If the angle of incidence is greater than the critical angle, the ray reflects and travels again in the denser substance.

The critical angle is a property of the substance, and its value differs from one substance to another. Optical fibers use reflection to guide light through a channel. Glass or plastic core is surrounded by a cladding of less dense glass or plastic. The difference in density of the two materials must be such that a beam of light moving through the core is reflected off the cladding instead of being refracted into it.

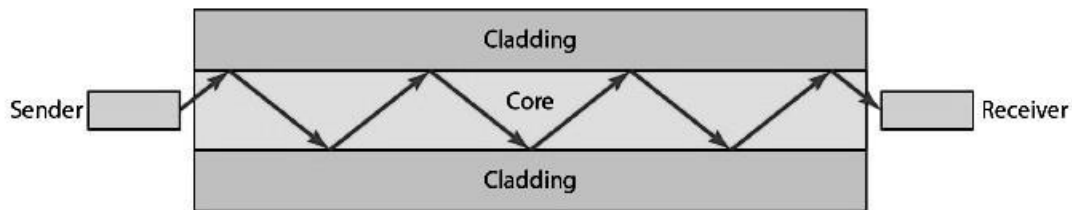


Figure 1.45 Optical fiber

Propagation modes

If the angle of incidence is less than the critical angle, the ray refracts and moves closer to the surface.

(i) Multimode fiber: Multiple beams from a light source move through the core in different paths. Multimode can be implemented in two forms step-index and graded-index.

a) Multimode step-index fiber: The density of the core remains constant from the center to the edges. A beam of light moves through this constant density in a straight line until it reaches the interface of the core and the cladding.

b) Multimode graded-index fiber: The density of the core is varying. Density is highest at the centre of the core and decreases gradually to its lowest at the edge.

(ii) Single mode: Single-mode uses step-index fiber and a highly focused source of light that limits beams to a small range of angles, all close to the horizontal. The single mode fiber itself is manufactured with a much smaller diameter than that of multimode fiber.

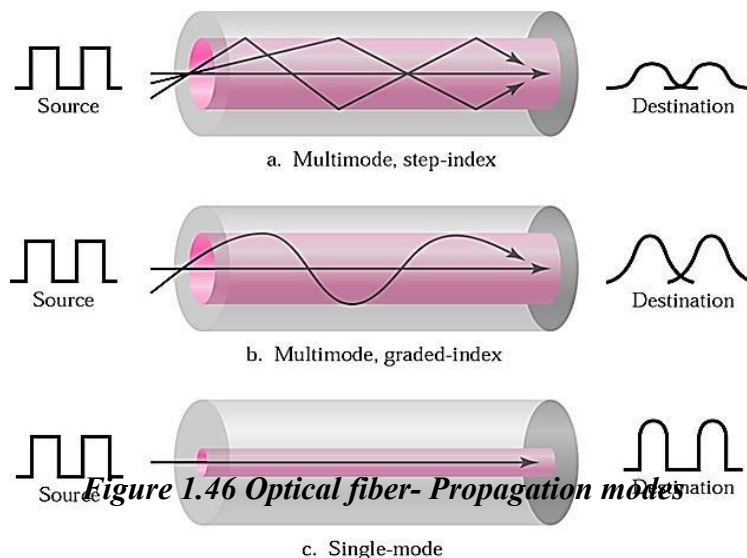


Figure 1.46 Optical fiber- Propagation modes

Fiber Sizes

Optical fibers are defined by the ratio of the diameter of their core to the diameter of their cladding, both expressed in micrometers. The common sizes are

Type	Core (μm)	Cladding (μm)	Mode
50/125	50.0	125	Multimode, graded index
62.5/125	62.5	125	Multimode, graded index
100/125	100.0	125	Multimode, graded index
7/125	7.0	125	Single mode

Table 1.3 Fiber types

Cable Composition

The outer jacket is made of either PVC or Teflon. Inside the jacket are Kevlar strands to strengthen the cable. Kevlar is a strong material used in the fabrication of bulletproof vests. Below the Kevlar is another plastic coating to cushion the fiber. The fiber is at the center of the cable, and it consists of cladding and core.

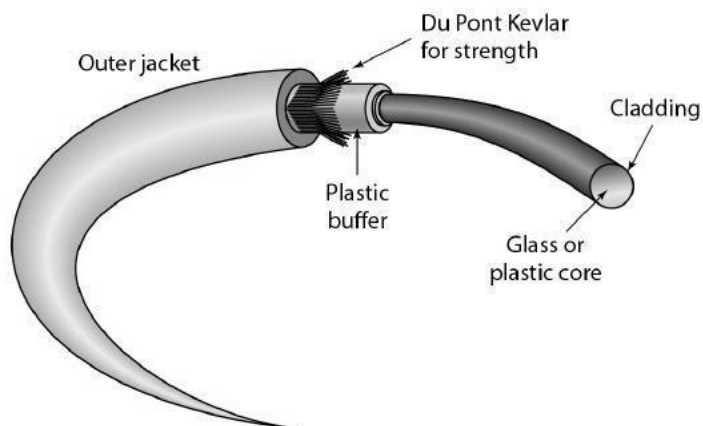


Figure 1.47 Fiber construction

Fiber-Optic Cable Connectors

- a) The subscriber channel (SC) connector is used for cable TV. It uses a push/pull locking system.
- b) The straight-tip (ST) connector is used for connecting cable to networking devices. It uses a bayonet locking system. It is more reliable than SC.
- c) MT-RJ is a connector that is the same size as RJ45, used in fast Ethernet

1.5.2 Unguided Transmission Medias

Unguided media transport electromagnetic waves without using a physical conductor. This type of communication is often referred to as wireless communication. Signals are normally broadcast through free space and thus are available to anyone who has a device capable of receiving them. The part of the electromagnetic spectrum, ranging from 3 kHz to 900 THz, is used for wireless communication. Unguided signals can travel from the source to destination in several ways;

- (i) **Ground propagation:** Radio waves travel through the lowest portion of the atmosphere.
- (ii) **Sky propagation:** Higher frequency radio waves radiate upward into the ionosphere and they are reflected back to earth.
- (iii) **Line-of-sight propagation:** Very high frequency signals are transmitted in straight lines directly from antenna to antenna

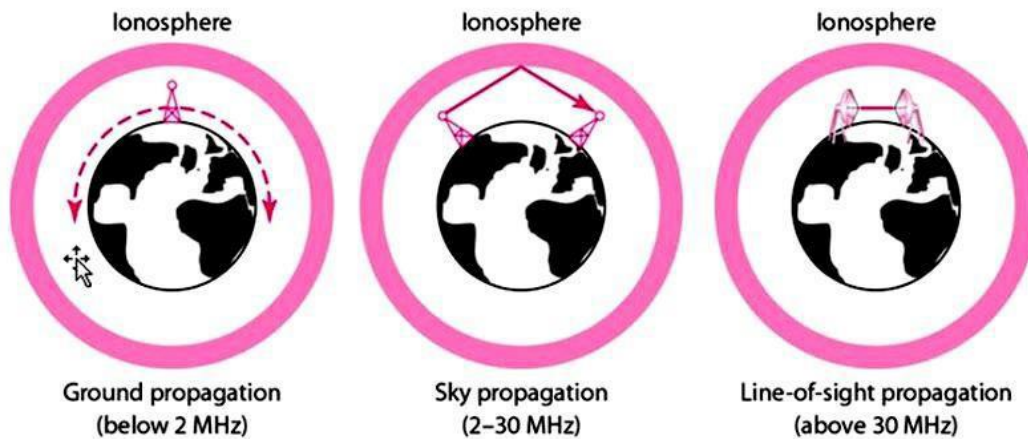


Figure 1.48 Propagation of Waves

We can divide wireless transmission into 3 broad groups. They are,

- i) Radio waves
- ii) Microwaves
- iii) Infrared waves

1.5.2.1 Radio waves

Electromagnetic waves ranging in frequencies between 3 kHz and 1 GHz are normally called radio waves. They are omnidirectional. When an antenna transmits radio waves, they are propagated in all directions, means that the sending and receiving antennas do not have to be aligned. A sending antenna sends waves that can be received by any receiving antenna. Radio waves with low and medium frequencies can penetrate walls. Radio waves are used for multi-communication (TV, radio, paging systems).

Disadvantages of Radio waves

- a) Penetrate the walls

b) Omnidirectional



Figure 1.49 Omnidirectional antenna

(a) Dish antenna

(b) Horn antenna

Figure 1.50 Unidirectional antennas

1.5.2.3 Infrared

Infrared waves, with frequencies from 300 GHz to 400 THz. Infrared waves having high frequencies cannot penetrate walls. Infrared signals can be used for short-range communication in a closed area using line-of-sight propagation.

Guided media	Unguided media
Signal energy propagates within the guided media	Signal energy propagates through air
Suitable for point-to-point communication	Suitable for broadcasting
Signals appear in the form of voltage	Signals appear in the form of electromagnetic waves
Ex: Twisted pair, Co-axial, Fiber optics	Ex: Radio wave, Micro wave, Infrared

UNIT II DATA LINK LAYER

Introduction – Link Layer Addressing – Error Detection and Correction: Introduction – Block Coding – CRC – Checksum– Framing – HDLC - Point-to-point protocol. Media Access Control: Random Access – Channelization - Wired LAN: Standard Ethernet – Connecting Devices – Virtual LANs.

2.1 INTRODUCTION

The Internet is a combination of networks attached together by connecting devices (routers or switches). If a packet is to travel from a host to another host, it needs to pass through these networks as shown in Figure 2.1. Communication at the data-link layer is made up of five separate logical connections between the data-link layers in the path.

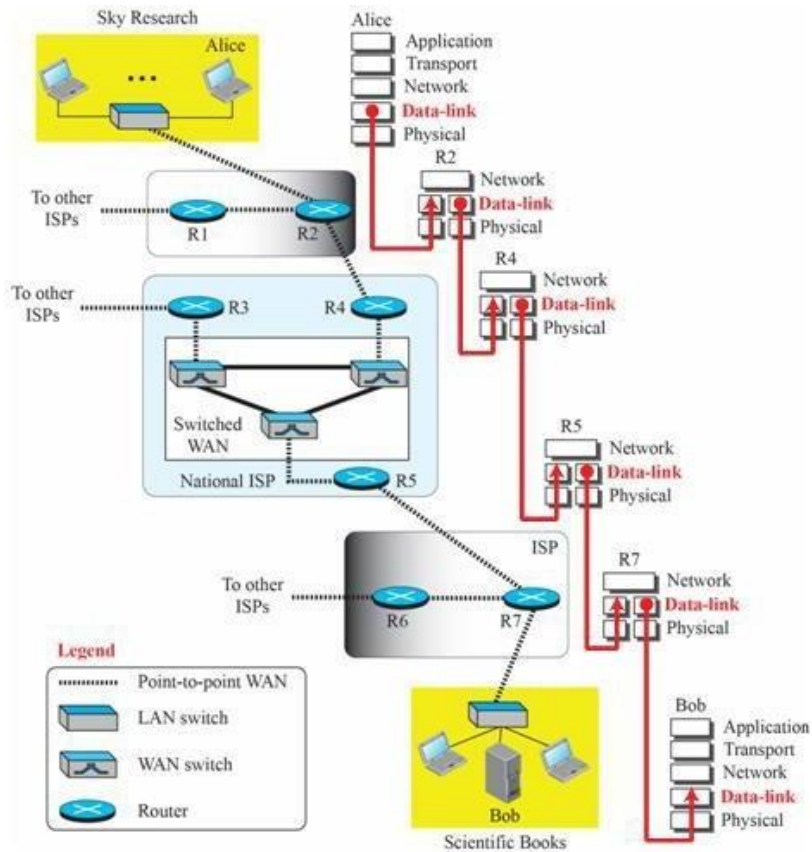


Figure 2.1 Communication at the data-link layer

The data-link layer at Alice's computer communicates with the data-link layer at router R2. The data-link layer at router R2 communicates with the data-link layer at router R4, and so on. Finally, the data-link layer at router R7 communicates with the data-link layer at Bob's computer. *Only one data-link layer is involved at the source or the destination, but two data-link layers are involved at each router.*

2.2 LINK-LAYER ADDRESSING

In the Internet, the source and destination IP addresses define the two ends but cannot define which links the datagram should pass through. The IP addresses in a datagram should not be changed. When the destination IP address in a datagram changes the packet never reaches its destination. If the source IP address in a datagram changes the router can never communicate with the source if a response needs to be sent back or an error needs to be reported back to the source. This issue can be solved by using the link-layer addresses of the two nodes.

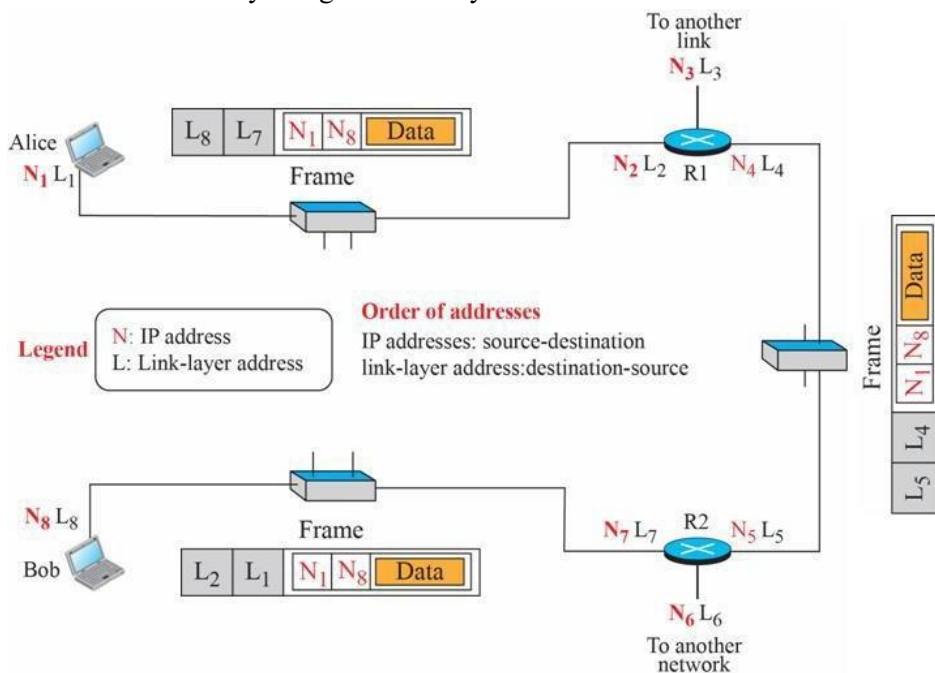


Figure 2.5 IP addresses and link-layer addresses in a small internet

In figure 2.5, each link consist a frame. Each frame carries the same datagram with the same source and destination addresses (N_i and N_8), but the link-layer addresses of the frame change from link to link. In link 1, the link-layer addresses are L_1 and L_2 . In link 2, they are L_4 and L_5 . In link 3, they are L_7 and L_8 . Here, the IP addresses and the link layer addresses are not in the same order. For IP addresses, the source address comes before the destination address. For the link-layer address, the destination address comes before the source.

2.2.1 Types of Addresses

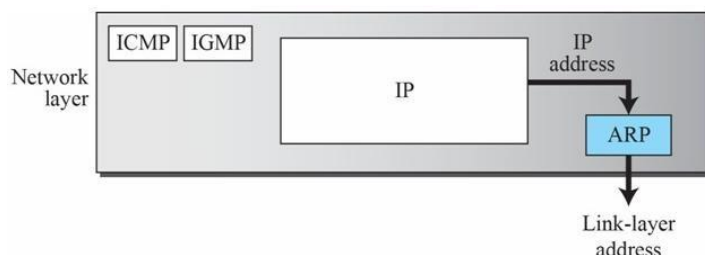
Link-layer protocols define three types of addresses namely unicast, multicast, and broadcast.

- (i) **Unicast Address:** Each host or each interface of a router is assigned a unicast address. Unicasting means one-to-one communication. A frame with a unicast address destination is destined only for one entity in the link.
- (ii) **Multicast Address:** Multicasting means one-to-many communication.
- (iii) **Broadcast Address:** Broadcasting means one-to-all communication. A frame with a destination broadcast address is sent to all entities in the link.

2.2.2 Address Resolution Protocol (ARP)

Anytime a node has an IP datagram to send to another node in a link, it has the IP address of the receiving node. The source host knows the IP address of the default router. Each router except the last one in the path gets the IP address of the next router by using its for-warding table. The last router knows the IP address of the destination host.

Without using the link-layer address of the next node, the IP address of the next node is not helpful in moving a frame through a link. The ARP protocol is one of the auxiliary protocols, which accepts an IP address from the IP protocol and maps the address to the corresponding link-layer address then passes it to the data-link layer.



another host or router in its network, it sends an ARP request packet. The packet includes the link-layer and IP addresses of the sender and the IP address of the receiver. Because, the sender doesn't know the link layer address of the receiver. The query is broadcast over the link using the link-layer broadcast address.

2.3. ERROR DETECTION AND CORRECTION

Data-link layer uses error control techniques to ensure that frames, i.e. bit streams of data, are transmitted from the source to the destination with a certain extent of accuracy.

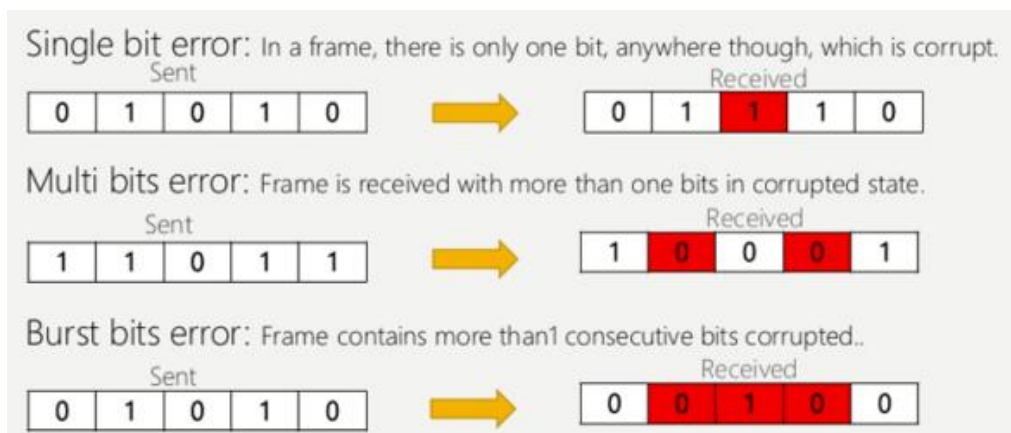
Errors

When bits are transmitted over the computer network, they are subject to get corrupted due to interference and network problems. The corrupted bits leads to spurious data being received by the destination and are called errors.

Types of Errors

Errors can be of three types, namely single bit errors, multiple bit errors, and burst errors.

- Single bit error – In the received frame, only one bit has been corrupted, i.e. either changed from 0 to 1 or 1 to 0
- Multiple bits error – In the received frame, more than one bit is corrupted.
- Burst error – In the received frame, more than one consecutive bit is corrupted.



Error Control

Error control can be done in two ways

- Error detection – Error detection involves checking whether any error has occurred or not. The number of error bits and the type of error does not matter.
- Error correction – Error correction involves ascertaining the exact number of bits that has been corrupted and the location of the corrupted bits.

For both error detection and error correction, the sender needs to send some additional bits along with the data bits. The receiver performs necessary checks based upon the additional redundant bits. If it finds that the data is free from errors, it removes the redundant bits before passing the message to the upper layers.

Error Detection Techniques

There are three main techniques for detecting errors in frames:

- Parity Check
- Checksum
- Cyclic Redundancy Check (CRC).

Parity Check

The parity check is done by adding an extra bit, called parity bit to the data to make a number of 1s either even in case of even parity or odd in case of odd parity. While creating a frame, the sender counts the number of 1s in it and adds the parity bit in the following way

In case of even parity: If a number of 1s is even then parity bit value is 0. If the number of 1s is odd then parity bit value is 1.

In case of odd parity: If a number of 1s is odd then parity bit value is 0. If a number of 1s is even then parity bit value is 1.

On receiving a frame, the receiver counts the number of 1s in it. In case of even parity check, if the count of 1s is even, the frame is accepted, otherwise, it is rejected. A similar rule is adopted for odd parity check. The parity check is suitable for single bit error detection only.

Conversion of Digital Data to Digital Signal involves three techniques:

1. Line Coding
2. Block Coding
3. Scrambling

Out of which Line coding is always needed, block coding and scrambling may or may not be needed.

2.4 BLOCK CODING:

Block coding helps in error detection and re-transmission of the signal. It is normally referred to as mB/nB coding as it replaces each m-bit data group with an n-bit data group (where $n > m$). Thus, it adds extra bits (redundancy bits) which helps in synchronization at receiver's and sender's end and also providing some kind of error

detecting capability.

It normally involves three steps: division, substitution, and combination. In the division step, a sequence of bits is divided into groups of m-bits. In the substitution step, we substitute an m-bit group for an n-bit group. Finally, the n-bit groups are combined together to form a stream which has more bits than the original bits.

CYCLIC REDUNDANCY CHECK (CRC)

Cyclic Redundancy Check (CRC) involves binary division of the data bits being sent by a predetermined divisor agreed upon by the communicating system. The divisor is generated using polynomials. Here, the sender performs binary division of the data segment by the divisor. It then appends the remainder called CRC bits to the end of the data segment. This makes the resulting data unit exactly divisible by the divisor.

The receiver divides the incoming data unit by the divisor. If there is no remainder, the data unit is assumed to be correct and is accepted. Otherwise, it is understood that the data is corrupted and is therefore rejected.

Checksum

In this error detection scheme, the following procedure is applied

- Data is divided into fixed sized frames or segments.
- The sender adds the segments using 1's complement arithmetic to get the sum. It then complements the sum to get the checksum and sends it along with the data frames.
- The receiver adds the incoming segments along with the checksum using 1's complement arithmetic to get the sum and then complements it.
- If the result is zero, the received frames are accepted; otherwise, they are discarded.

FRAMEING

To provide the flexibility, HDLC defines three types of frames namely *information frames (I-frames)*, *supervisory frames (S-frames)*, and *unnumbered frames (V-frames)*. Each type of frame serves as an envelope for the transmission of a different type of message.

- I-frames are used to transport user data and control information relating to user data (piggybacking).
- S-frames are used only to transport control information.
- V-frames are reserved for system management. Information carried by V-frames is intended for managing the link itself.

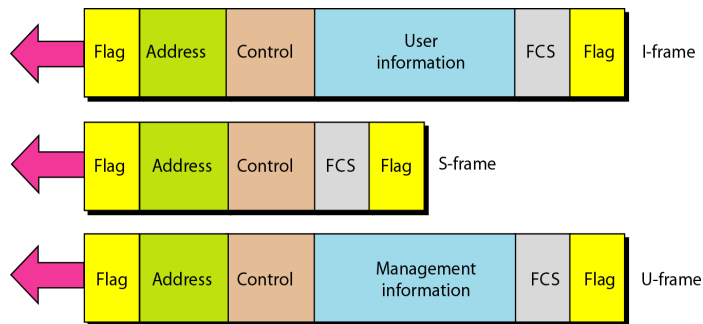


Figure 2.27 HDLC frames

2.6 HIGH-LEVEL DATA LINK CONTROL (HDLC)

High-level Data Link Control (HDLC) is a bit-oriented protocol for communication over point-to-point and multipoint links. It implements the ARQ mechanisms. The HDLC protocol embeds information in a data frame that allows devices to control data flow and correct errors. In 1979, the ISO made HDLC the standard as a Bit-oriented control protocol. The HDLC provides a transparent transmission service at the data link layer of the OSI. The users of the HDLC service provide PDUs which are encapsulated to form data link layer frames. These frames are *separated by HDLC "flags" and are modified by "zero bit insertion" to guarantee transparency.*

Each piece of data is encapsulated in an HDLC frame by adding a trailer and a header. The header contains an HDLC address and an HDLC control field. The trailer is found at the end of the frame, and contains a (CRC) which detects any errors which may occur during transmission. The frames are separated by HDLC flag sequences which are transmitted between each frame and whenever there is no data to be transmitted. HDLC provides two common transfer modes that can be used in different configurations: *normal response mode (NRM) and asynchronous balanced mode (ABM).*

(i) Normal response mode (NRM)

In normal response mode (NRM), the station configuration is unbalanced. We have one primary station and multiple secondary stations. A primary station can send commands and a secondary station can only respond. The NRM is used for both point-to-point and multiple-point links,

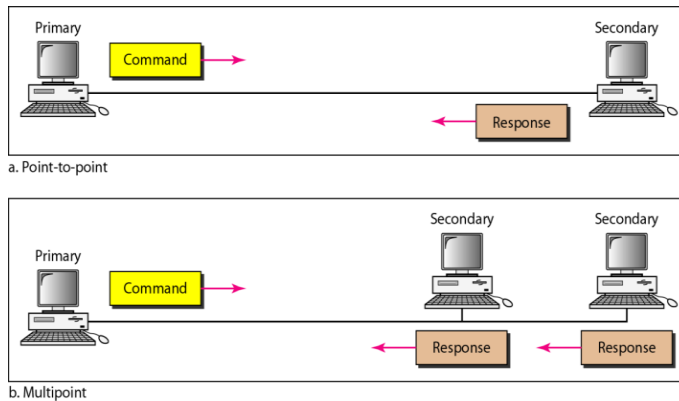


Figure 2.25 Normal response mode

(i) Asynchronous Balanced Mode

In asynchronous balanced mode (ABM), the configuration is balanced. The link is point-to-point, and each station can function as a primary and a secondary (acting as peers), as shown in Figure 2.26. This is the common mode today.

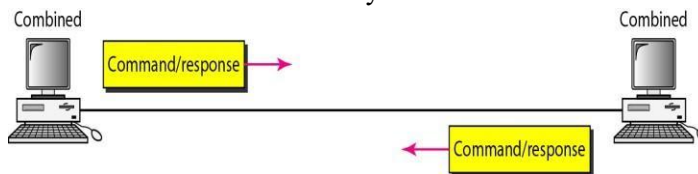


Figure 2.26 Asynchronous balanced mode

Fields and their use in different frame types

- (i) **Flag field:** The flag field of an HDLC frame is an 8-bit sequence with the bit pattern 01111110 that identifies both the beginning and the end of a frame and serves as a synchronization pattern for the receiver. The ending flag of one frame can be used as the beginning flag of the next frame.
- (ii) **Address field:** The second field of an HDLC frame contains the address of the secondary station. If a primary station created the frame, it contains a *to address*. If a secondary creates the frame, it contains a *from address*. An address field can be 1 byte or several bytes long, depending on the needs of the network. One byte can identify up to 128 station

2.6.3 Point-To-Point Protocol (PPP)

- PPP is one of the most common protocols for point-to-point access.
- Today, millions of Internet users who connect their home computers to the server of an ISP use PPP.

Framing

- PPP uses a character-oriented (or byte-oriented) frame as shown in figure 2.29.

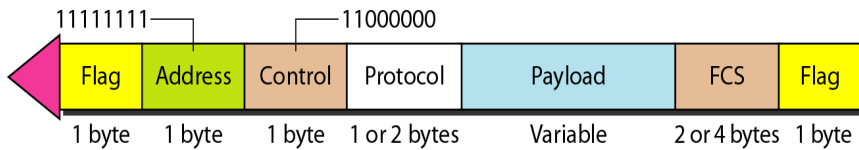


Figure 2.29 PPP frame format

Various fields of PPP frame

(i) Flag

- This field has a synchronization pattern 01111110.
- This field identifies both the beginning and the end of a frame.

(ii) ss

- This field is set to the constant value 11111111 (broadcast address).

(iii) Control

- This field is set to the constant value 00000011 (imitating unnumbered frames in HDLC).
- PPP does not provide any flow control.
- Error control is also limited to error detection.

(iv) Protocol

- This field defines what is being carried in the payload field.
- Payload field carries either i) user data or ii) other control information.
- By default, size of this field = 2 bytes.

(v) FCS

- This field is the PPP error-detection field.
- This field can contain either a 2- or 4-byte standard CRC.

Byte Stuffing

Since PPP is a byte-oriented protocol, the flag in PPP is a byte that needs to be escaped whenever it appears in the data section of the frame. The escape byte is 01111101, which means that every time the flag like pattern appears in the data, this extra byte is stuffed to tell the receiver that the next byte is not a flag. Obviously, the escape byte itself should be stuffed with another escape byte.

Transition Phases

- The transition diagram starts with the dead state as shown in figure 2.30.

(a) Dead State

- In dead state, there is no active carrier and the line is quiet.

(b) Establish State

- When 1 of the 2 nodes starts communication, the connection goes into the establish state.
- In establish state, options are negotiated between the two parties.

(c) Authenticate State

- If the 2 parties agree that they need authentication, then the system needs to do authentication; otherwise, the parties can simply start communication.

(d) Open State

- Data transfer takes place in the open state.

(e) Terminate State

- When 1 of the endpoints wants to terminate connection, the system goes to terminate state.

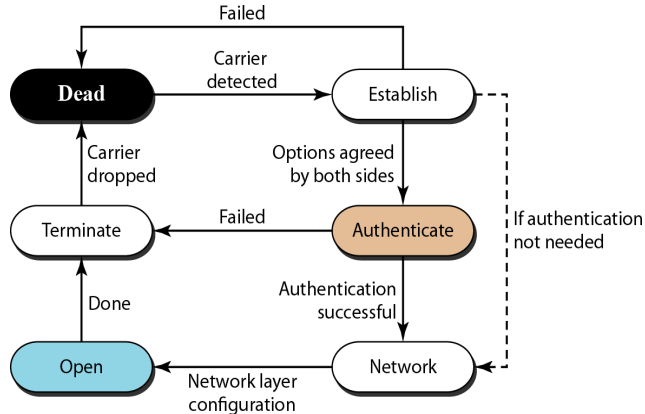


Figure 2.30 Transition phases

2.7 MEDIA ACCESS CONTROL

The two main functions of the data link layer are data link control and media access control. The data link control deals with the design and procedures for communication between two adjacent nodes: node-to-node communication. The second function of the data link layer is media access control, or how to share the link.

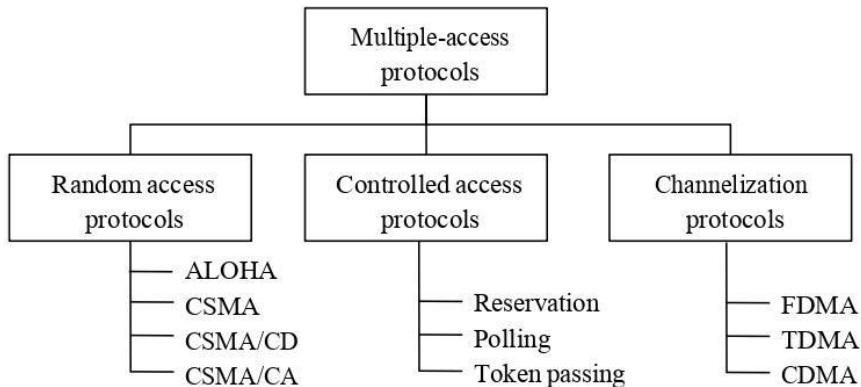


Figure 2.31 Taxonomy of multiple-access protocols

2.7.1 RANDOM ACCESS

In random access no station is superior to another station and none is assigned the control over another. A station that has data to send uses a procedure defined by the protocol to make a decision on whether or not to send. This decision depends on the state of the medium (idle or busy). Two features of random access are;

- (i) There is no scheduled time for a station to transmit. Transmission is random among the stations. That is why these methods are called **random access**.
- (ii) No rules specify which station should send next. Stations compete with one another to access the medium. That is why these methods are also called **contention methods**.

In a random access method, each station has the right to the medium without being controlled by any other station. If more than one station tries to send, there is an access conflict- collision-and the frames will be either destroyed or modified. To avoid access conflict or to resolve it when it happens, each station follows a procedure that answers the following questions:

- 1) When can the station access the medium?
- 2) What can the station do if the medium is busy?
- 3) How can the station determine the success or failure of the transmission?
- 4) What can the station do if there is an access conflict?

The random access method using ALOHA protocol which used a very simple procedure called multiple access (MA). The method was improved with the addition of a procedure that forces the station to sense the medium before transmitting. This was called carrier sense multiple access. This method later evolved into two parallel methods:

- (i) Carrier sense multiple access with collision detection (CSMA/CD) : CSMA/CD tells the station what to do when a collision is detected
- (ii) Carrier sense multiple access with collision avoidance (CSMA/CA): CSMA/CA tries to avoid the collision.

ALOHA

ALOHA, the earliest random access method was designed for a radio (wireless) LAN, but it can be used on any shared medium. When the medium is shared between the stations, the data from the two stations collide and become garbled.

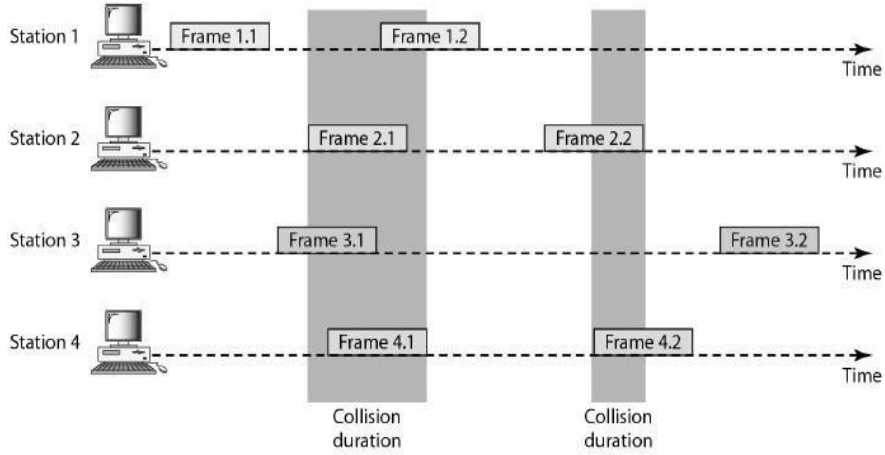


Figure 2.32 Frames in a pure ALOHA network

A collision involves two or more stations. If all these stations try to resend their frames after the time-out, the frames will collide again. Pure ALOHA dictates that when the time-out period passes, each station waits a random amount of time before resending its frame. The randomness will help avoid more collisions. We call this time the **back-off time** T_B . Pure ALOHA has a 2nd method to prevent congesting the channel with retransmitted frames. After a maximum number of retransmission attempts K_{max} a station must give up and try later.

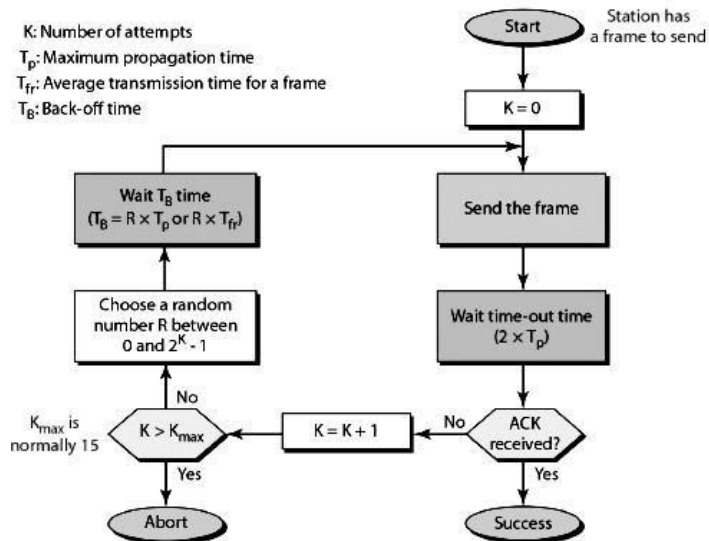


Figure 2.33 Procedure for pure ALOHA protocol

2.7.2 Channelization

Channelization is a multiple-access method in which the available bandwidth of a link is shared in time, frequency, or through code, between different stations. Three Channelization protocols are used. They are,

(iii) FDMA

(iv) TDMA

(v) CDMA

Frequency-division multiple access (FDMA)

In frequency-division multiple access (FDMA), the available bandwidth is divided into frequency bands. Each station is allocated a band to send its data. Each band is reserved for a specific station, and it belongs to the station all the time. Each station also uses a band - pass filter to confine the transmitter frequencies.

To prevent station interferences, the allocated bands are separated from one another by small guard bands. FDMA specifies a predetermined frequency band for the entire period of communication (a continuous flow of data that may not be packetized).

Time-Division Multiple Access (TDMA)

In time-division multiple access (TDMA), the stations share the bandwidth of the channel in time. Each station is allocated a time slot during which it can send data. Each station transmits its data in assigned time slot.

The main problem with TDMA lies in achieving synchronization between the different stations. Each station needs to know the beginning of its slot and the location of its slot. This is difficult because of propagation delays introduced in the system if the stations are spread over a large area.

To compensate for the delays, we can insert guard times. Synchronization is normally accomplished by having some synchronization bits (normally referred to as preamble bits) at the beginning of each slot.

Code-Division Multiple Access (CDMA)

CDMA differs from FDMA because only one channel occupies the entire bandwidth of the link. It differs from TDMA because all stations can send data simultaneously; there is no timesharing. In CDMA, one channel carries ~~421~~ ⁴² transmissions simultaneously.

2.8 Wired LAN

ETHERNET (IEEE 802.3)

A LAN can be used as an isolated network to connect computers in an organization for sharing resources. Most of the LANs today are linked to a wide area network (WAN) or the Internet. The LAN market has seen several technologies such as,

- (i) Ethernet
- (ii) Token Ring
- (iii) Token Bus
- (iv) FDDI
- (v) ATM LAN.

The IEEE Standard Project 802 is designed to regulate the manufacturing and interconnectivity between different LANs.

2.8.1 IEEE STANDARDS

The IEEE 802 standard was adopted by the American National Standards Institute (ANSI). In 1987, the International Organization for Standardization (ISO) also approved it as an international standard. The relationship of the 802 Standard to the traditional OSI model is shown in figure 2.44. The IEEE has subdivided the data link layer into two sub layers:

- (i) Logical link control (LLC)
- (ii) Media access control (MAC).

The data link layer in the IEEE standard is divided into two sublayer. They are,

- (i) Logical Link Control (LLC)
- (ii) Media Access Control (MAC)

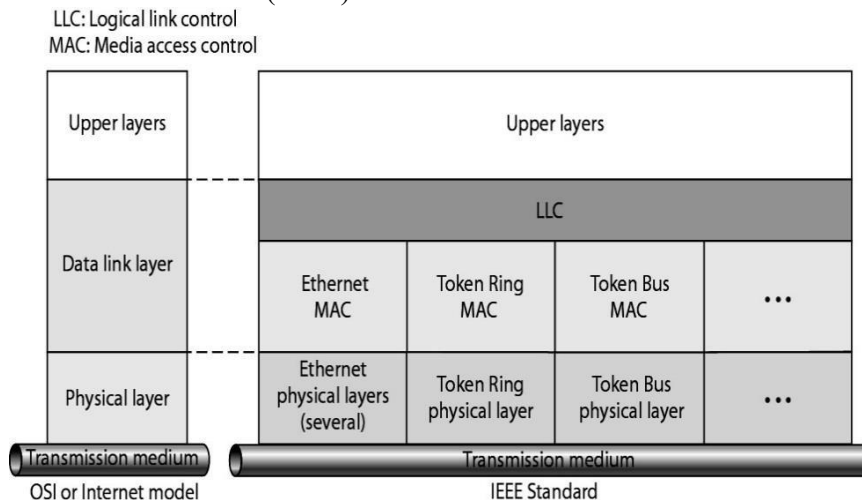


Figure 2.44 IEEE standard for LANs

Logical Link Control (LLC)

In IEEE Project 802, flow control, error control, and part of the framing duties are collected into a sublayer called the logical link control. Framing is handled in both the LLC sublayer and the MAC sublayer. The LLC provides a single data link control protocol for all IEEE LANs, but the MAC sublayer provides different protocols for different LANs. A single LLC protocol can provide interconnectivity between different LANs because it makes the MAC sublayer transparent.

Media Access Control (MAC)

IEEE Project 802 has created a sublayer called media access control that defines the specific access method for each LAN. For example, it defines CSMA/CD as the media access method for Ethernet LANs and the token passing method for Token Ring and

2.8.2 STANDARD ETHERNET

Ethernet data link layer protocol provides connectionless service to the network layer. (No handshaking between sending and receiving machine). It also provides an unreliable service to the network layer. Here the receiver doesn't send ACK or NAK to sender. This means that the stream of datagram's passed to network layer can have gaps (missing data).

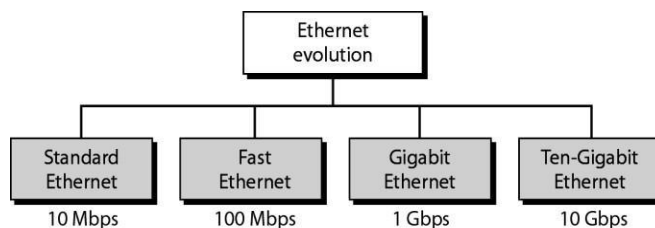


Figure 2.46 Ethernet evolution

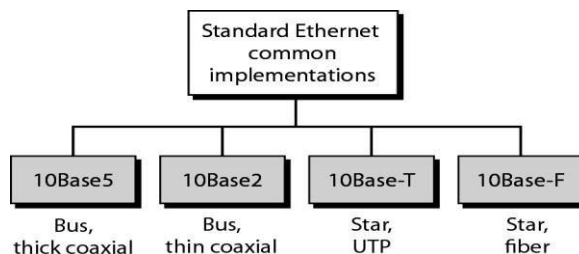
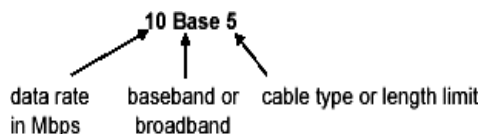


Figure 2.47 Categories of traditional Ethernet

10BASE5

- Data transfer rate is 10 Mbps.
- 500 meter segment length.
- Signal regeneration can be done with help of repeaters.
- Thick Coax is used as a transmission medium.



Advantages:

- (i) Low attenuation,
- (ii) Excellent noise immunity
- (iii) Superior mechanical strength

Disadvantages:

- (i) Bulky
- (ii) Difficult to pull
- (iii) Transceiver boxes are too expensive
- (iv) Wiring represented a significant part of total installed cost.

1 BASE 5 (Star LAN)

- Data transfer rate is 1 Mbps
- 250 meter segment length.
- Signal regeneration can be done with help of repeaters.
- Transceiver integrated onto the adapter.
- Implemented with the help of star topology
- Two pairs of unshielded twisted pair cable are used as a transmission media.

Advantages:

- (i) It is easier to use installed wiring in the walls.

2.8.3 Changes In The Standard

2.8.3.1 Bridged Ethernet

The first step in the Ethernet evolution was the division of a LAN by bridges. Bridges have two effects on an Ethernet LAN. They are,

- (i) Raise the bandwidth
- (ii) Separate collision domains.

Raising the Bandwidth

In an un-bridged Ethernet network, the total capacity (10 Mbps) is shared among all stations with a frame to send. The stations share the bandwidth of the network. For example, if two stations have a lot of frames to send, they probably alternate in usage. When one station is sending, the other one refrains from sending.

A bridge divides the network into two or more networks. Bandwidth-wise, each network is independent. For example, a network with 12 stations is divided into two networks, each with 6 stations. Now each network has a capacity of 10 Mbps. The 10-Mbps capacity in each segment is now shared between 6 stations not 12 stations.

In a network with a heavy load, each station is offered 10/6 Mbps instead of 10/12 Mbps. If we use a four-port bridge, each station is now offered 10/3 Mbps, which is 4 times more than an un-bridged network.

Separating Collision Domains

In the bridged network, the collision domain becomes much smaller and the probability of collision is reduced tremendously.

2.8.3.2 Switched Ethernet

The basic idea behind the switched Ethernet is to overcome the drawbacks of Hub concept. The switch learns destination locations by remembering the ports of the associated source address in a table. The switch may not have to broadcast to all output ports. It may be able to send the frame only to the destination port. A big performance advantage of a switch over a hub is that, more than one frame transfer can go through it concurrently.

The advantage comes when the switched Ethernet backplane is able to repeat more than one frame in parallel (a separate backplane bus line for each node). The frame is relayed onto the required output port via the port's own backplane bus line. Under this scheme collisions are still possible when two concurrently arriving frames are destined for the same station. Each parallel transmission can take place at 10Mbps.

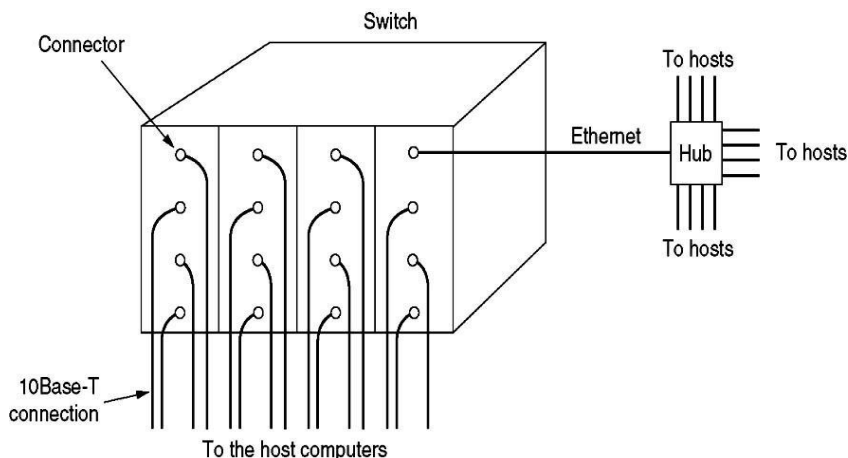


Figure 2.48 Example of switched Ethernet

2.8.3.3 Fast Ethernet

- Data transmission rate is 100 Mbps.
- Using the same frame format, media access, and collision detection rules as 10 Mbps Ethernet.
- It is possible to combine 10 Mbps Ethernet and Fast Ethernet on same network using a switch.
- Twisted pair (CAT 5) or fiber optic cable (no coax) can be used as a transmission media.
- Implemented with star-wire topology.

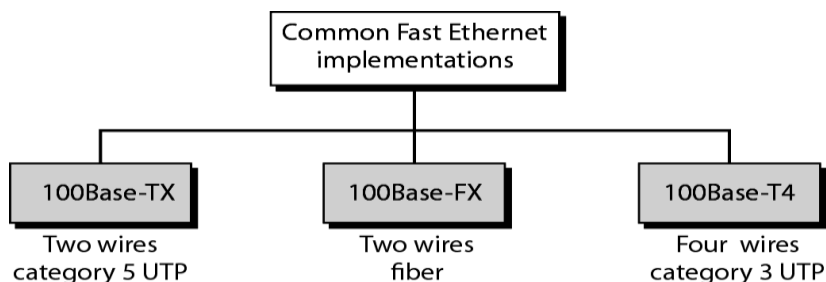


Figure 2.49 Fast Ethernet implementations

2.9 CONNECTING DEVICES:

The connecting devices are divided into five different categories based on the layer in which they operate in a network.

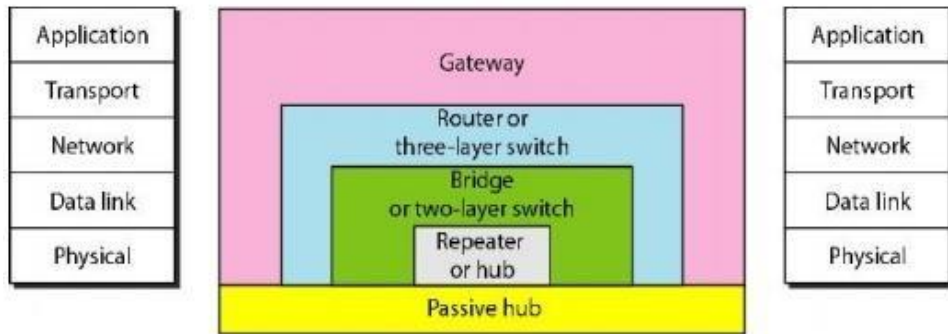


Figure: Five categories of connecting devices

The five categories contain devices which can be defined as:

1. Those which operate below the physical layer such as a passive hub.
2. Those which operate at the physical layer (a repeater or an active hub).
3. Those which operate at the physical and data link layers (a bridge or a two-layer switch).
4. Those which operate at the physical, data link & network layers (a router or a 3-layer switch).
5. Those which can operate at all five layers (a gateway).

1. Passive Hubs

A passive hub is just a connector. It connects the wires coming from different branches. In a star-topology Ethernet LAN, a passive hub is just a point where the signals coming from different stations collide; the hub is the collision point. This type of a hub is part of the media; its location in the Internet model is below the physical layer.

2. Repeaters

A repeater is a device that operates only in the physical layer. Signals that carry information within a network can travel a fixed distance before attenuation endangers the integrity of the data.

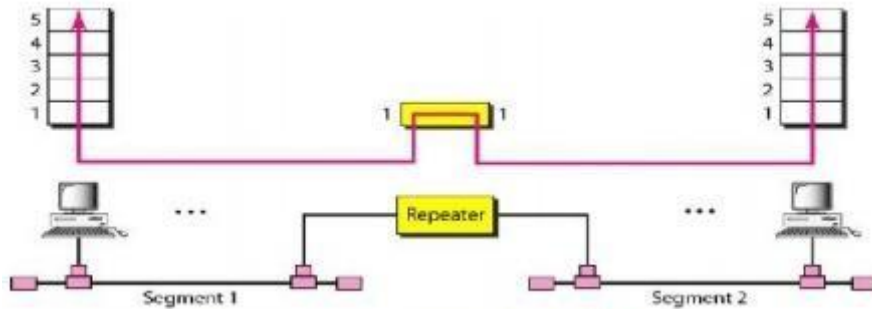


Figure: Repeater connecting two segments of LAN

A repeater receives a signal and, before it becomes too weak or corrupted, regenerates the original bit pattern. The repeater then sends the refreshed signal. A repeater can extend the physical length of a LAN.

3. Active Hubs

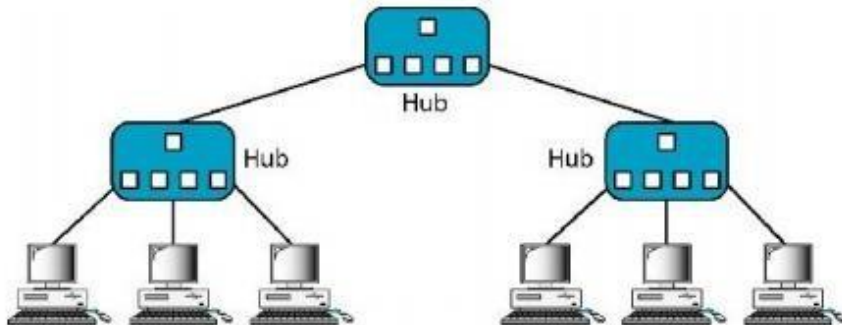
An active hub is actually a multipart repeater. It is normally used to create connections between stations in a physical star topology. However, hubs can also be used to create multiple levels of hierarchy. The hierarchical use of hubs removes the length limitation of 10Base-T (100 m).

4. Bridges

A bridge operates in both the physical and the data link layer. As a physical layer device, it regenerates the signal it receives. As a data link layer device, the bridge can check the physical (MAC) addresses (source and destination) contained in the frame.

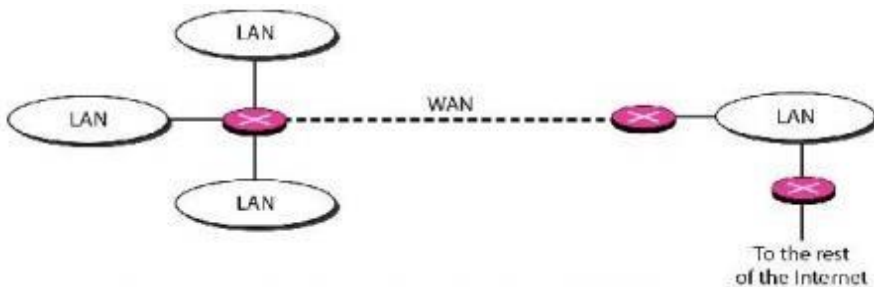
5. Filtering

A bridge has filtering capability. It can check the destination address of a frame and decide if the frame should be forwarded or dropped. If the frame is to be forwarded, the decision must specify the port. A bridge has a table that maps addresses to ports.



6. Routers

A router is a three-layer device that routes packets based on their logical addresses (host-to-host addressing). A router normally connects LANs and WANs in the Internet and has a routing table that is used for making decisions about the route. The routing tables are normally dynamic and are updated using routing protocols.



2.10 VIRTUAL LANS.

In the annals of office history, there exists a forgotten era of networking, a time when thick, yellow cables dictated the digital landscape. These robust conduits, often referred to as "yellow garden hoses," snaked through buildings with little regard for corporate hierarchy. Every computer they encountered was summarily plugged in, creating local area networks (LANs) based purely on geographical proximity. The result was a chaotic patchwork where adjacent offices shared a LAN, irrespective of their departmental affiliations. Geography, it seemed, was the undisputed monarch of network organization.

The 1990s heralded a significant shift with the arrival of twisted pair cabling and hubs. This innovation sparked a considerable—and costly—rewiring effort across countless organizations. The venerable yellow hoses were ripped out, replaced by a meticulously organized system of twisted pairs running from every office to centralized wiring closets. These closets, typically

found at the end of corridors or in dedicated machine rooms, became the nerve centers of the new network. Visionary Vice Presidents in Charge of Wiring opted for Category 5 twisted pairs, anticipating future speeds, while the more fiscally conservative settled for existing Category 3 telephone wiring, a decision often regretted within a few years as fast Ethernet emerged.

Today, while the cables have evolved and hubs have given way to more sophisticated switches, the fundamental wiring pattern remains. This enduring design, however, has unlocked a transformative capability: the ability to configure LANs logically rather than physically. Instead of being bound by the physical layout of the building, organizations can now acquire multiple switches and, by strategically connecting computers, group users onto LANs that align with their organizational structure.

Security: The Digital Moat

One of the most critical drivers for logically structured LANs is security. Imagine an organization with a public-facing LAN hosting web servers and other publicly accessible resources. Alongside it, another LAN safeguards the highly sensitive records of the Human Resources department. Placing all these diverse systems on a single, sprawling LAN would be a security nightmare. By segmenting them, administrators can create digital moats, ensuring that HR data remains isolated and inaccessible from the public network. Management, understandably, takes a dim view of any arrangement that compromises such vital data.

Load Balancing: Preventing Digital Gridlock

Another significant consideration is network load. Not all LANs are created equal in terms of traffic. A research department, for instance, might be constantly running data-intensive experiments, occasionally saturating their LAN. If this impacts the capacity available for, say, management's crucial videoconferencing sessions, tensions are bound to rise. Separating these high-demand LANs helps prevent digital gridlock and ensures that critical operations remain unimpeded. While it might also serve to highlight the need for network upgrades, the immediate benefit is load isolation.

Broadcast Traffic: Taming the Network Chatter

Broadcast traffic presents a third compelling reason for carefully structured LANs. Bridges, when unsure of a destination's location, broadcast frames, and upper-layer protocols frequently utilize broadcasting. Consider a scenario where a user wants to send a packet to an IP address. To determine the correct MAC address for the frame, the system broadcasts a query: "Who owns IP address X?" This broadcast is delivered to every computer on the LAN, consuming more capacity than a regular frame. As LANs expand, so does the volume of broadcast traffic, potentially leading to network congestion. By keeping LANs appropriately sized, the impact of this constant network chatter is significantly reduced.

UNIT III

NETWORK LAYER

Network Layer Services- Network layer performance IPV4 addresses - Internet Protocol (IP) - ICMPv4. Unicast Routing Algorithms: Distance Vector and Link-state routing - Static Routing - Dynamic Routing Protocols: RIP and OSPF IPV6 addressing- IPV6 protocol - Software-defined networks

3.1. NETWORK LAYER SERVICES

Network layer services include addressing (like IP addresses) for unique device identification, routing to find the best path for data, packetization and fragmentation to break data into manageable packets, forwarding to move packets across networks, error detection and handling to manage transmission issues, and congestion control to prevent network overload. These services ensure that data packets are successfully transferred from a source to a destination, potentially across multiple networks.

3.1.1 Major Network layer services:

The **Network Layer** (Layer 3 of the OSI model) is responsible for enabling data transfer between devices across different networks. It provides several essential services to ensure efficient, reliable, and scalable communication.

3.1.1.1. Addressing:

Assigns unique logical addresses (such as IP addresses) to devices on a network, allowing for host-to-host delivery and identifying the source and destination of data packets.

Physical Addresses (MAC Addresses):

These are unique hardware identifiers assigned by the manufacturer to a Network Interface Card (NIC). They are used at the data link layer to identify devices within a local network segment. A MAC address consists of an Organizationally Unique Identifier (OUI) for the manufacturer and a unique device identifier.

Logical Addresses (IP Addresses):

These are assigned by network administrators or automatically via DHCP to allow devices to communicate across different networks.

IPv4: addresses are 32-bit numbers, while IPv6 addresses are 128-bit, providing a larger address space.

They are hierarchical, allowing for efficient routing of data packets.

Port Addresses:

These specify a particular process or application on a destination host.

For example, when you access a website, you are using the port address assigned to the web server.

Specific Addresses:

These are user-friendly names that map to logical (IP) addresses, such as email addresses and URLs

The Domain Name System (DNS) translates these names into IP addresses for communication.

3.2. NETWORK LAYER PERFORMANCE

Network layer performance describes the quality of service provided by a network, assessed through critical metrics such as bandwidth, latency, throughput, packet loss, and jitter. These metrics collectively determine the speed and reliability of data transmission across the network. The performance is influenced by factors including the number of connected devices, network traffic volume, the type of transmission medium, and the overall design of the network.

Performance Metrics

3.2.1. Bandwidth:

The highest volume of data that can be transmitted over the network within a specific time frame.

Latency (Delay):

The total time taken for data to move from its source to the destination. Latency consists of several components:

Propagation delay:

The time it takes for a single bit to travel from one point to another.

Propagation Delay Calculation - Propagation delay is calculated using the formula

$$\text{Propagation Delay} = \frac{\text{Distance}}{\text{Propagation Speed}}$$

Distance: The length between the sender and receiver (usually in meters or kilometers).

Propagation Speed: The speed at which the signal travels through the medium (in meters per second).

For example:

Example:

If the distance between two devices is 500 kilometers, and the signal travels through fiber optic cable at 2×10^8 meters/second:

$$\text{Propagation Delay} = \frac{500,000 \text{ meters}}{2 \times 10^8 \text{ m/s}} = 0.0025 \text{ seconds} = 2.5 \text{ milliseconds}$$

3.2.2. Transmission delay:

The time required to place all bits of a packet onto the communication link.

Transmission Delay Calculation

Transmission delay is the time it takes to push all the bits of a data packet onto the network link.

It depends on the packet size and the bandwidth of the link.

The formula is:

$$\text{Transmission Delay} = \frac{\text{Packet Size (bits)}}{\text{Transmission Rate (bits per second)}}$$

Packet Size: The total number of bits in the packet.

Transmission Rate (Bandwidth): The speed of the link in bits per second (bps).

3.2.3. Processing delay:

The time routers use to examine packet headers and decide the forwarding path.

Processing delay is generally measured rather than calculated because it depends on:

- The router or switch hardware capabilities (CPU speed, memory).
- The complexity of the routing table and algorithms.
- The current load or congestion on the device.

However, if you want to estimate or measure it, network engineers often use:

Processing Delay = Time spent per packet at the router (measured in seconds or milliseconds)

This is usually obtained through network performance tools or simulation software rather than a direct formula.

Example:

If a router takes 2 microseconds (2 μ s) to process each packet, then:

$$\text{Processing Delay} = 2 \times 10^{-6} \text{ seconds}$$

3.2.4. Queuing delay:

The waiting time a packet spends in router queues before transmission, Queuing delay is the time a packet spends waiting in a queue before it can be transmitted onto the network. This delay happens when multiple packets arrive at a router or switch faster than they can be processed or forwarded, causing packets to line up.

Unlike other delays, queuing delay is variable and depends heavily on network congestion and traffic load. When the network is busy, queuing delay increases; when the network is free, it's minimal or zero.

Calculation / Estimation

Queuing delay can be estimated using **queuing theory**, often modeled with formulas like the **M/M/1 queue** (a common model in networking):

$$D_q = L / R \times \rho / 1 - \rho$$

Where:

D_q = Average queuing delay

L = Packet length (bits)

R = Link transmission rate (bits/second)

$\rho = \lambda / \mu$ = Utilization factor (traffic intensity),

where:

λ = Average arrival rate of packets

μ = Average service rate (packets processed per second)

If ρ approaches 1 (meaning the router is almost fully utilized), queuing delay tends to increase dramatically.

Example:

Packet length $L=1,000$ bits

Link transmission rate $R=1,000,000$ bits per second (1 Mbps)

Average arrival rate $\lambda=800$ packets per second

Average service rate $\mu=1,000$ packets per second

Step 1: Calculate utilization ρ :

$$\rho = \frac{\lambda}{\mu} = \frac{800}{1000} = 0.8$$

Step 2: Calculate queuing delay D_q :

$$D_q = \frac{L}{R} \times \frac{\rho}{1 - \rho}$$
$$D_q = \frac{1,000}{1,000,000} \times \frac{0.8}{1 - 0.8} = 0.001 \times \frac{0.8}{0.2} = 0.001 \times 4 = 0.004 \text{ seconds} = 4 \text{ milliseconds}$$

3.2.5. Throughput:

The effective rate at which data is successfully transmitted over the network, often lower than the maximum bandwidth due to various limiting factors. Throughput is the actual rate at which data is successfully transmitted over the network in a given amount of time. It represents the effective data transfer speed experienced by users and applications, which is often less than the theoretical maximum bandwidth due to factors like network congestion, protocol overhead, and errors.

Calculate Throughput

Throughput can be calculated by dividing the total amount of data successfully delivered by the time it took to transfer:

$$\text{Throughput} = \text{Total Data Transferred (bits)} / \text{Total Time (seconds)}$$

Example:

If a file of size 10 megabits (10,000,000 bits) is transmitted successfully in 5 seconds, the throughput would be:

$$\text{Throughput} = \frac{10,000,000 \text{ bits}}{5 \text{ seconds}} = 2,000,000 \text{ bits per second} = 2 \text{ Mbps}$$

Packet Loss:

The count of packets that fail to reach their destination during transmission.

3.2.6. Jitter:

The fluctuation in latency over time, which can affect the consistency of data delivery. Factors Influencing Performance, Jitter refers to the variation or inconsistency in packet delay over time within a network. In other words, it's the fluctuation in latency between consecutive packets being delivered. High jitter can cause problems for real-time applications like video calls, online gaming, and VoIP because it leads to uneven data flow and poor user experience.

Calculate Jitter

Jitter is typically measured as the average deviation from the mean latency.

One common way to express jitter is:

$$\text{Jitter} = \frac{1}{N-1} \sum_{i=1}^{N-1} |D_{i+1} - D_i|$$

Where:

- D_i is the delay of the i^{th} packet
- N is the total number of packets measured
- The formula calculates the average of the absolute differences between delays of successive packets

Example:

Suppose packet delays in milliseconds are:

10 ms, 12 ms, 15 ms, 14 ms, 18 ms

Calculate the jitter:

$$|12 - 10| = 2, \quad |15 - 12| = 3, \quad |14 - 15| = 1, \quad |18 - 14| = 4$$

Average:

$$\frac{2 + 3 + 1 + 4}{4} = \frac{10}{4} = 2.5 \text{ ms}$$

So, the jitter is 2.5 milliseconds.

Number of Devices:

Increased connected devices can raise traffic levels and competition for resources.

3.2.7. Transmission Medium Bandwidth:

The physical capability of the network infrastructure to carry data affects overall speed.

Type of Network Traffic:

Different applications place varied demands on the network, influencing performance needs.

Network Latency:

Elevated latency negatively impacts user experience and network responsiveness.

Transmission Errors:

Errors in data packets can slow down communication and reduce reliability.

Network Design and Configuration:

The structure and setup of the network have a direct effect on how efficiently it operates.

Measuring and Enhancing Network Layer Performance

Utilize Network Monitoring Tools:

These tools gather detailed performance data, aiding in the identification of issues.

Analyze Performance Metrics:

Careful examination of data helps to understand network behavior and locate bottlenecks.

Implement Quality of Service (QoS):

QoS mechanisms allow prioritization of critical network traffic, improving service quality for important applications.

Perform Network Testing:

Simulating different real-world conditions allows evaluation of the network's performance under various scenarios.

3.3. IPV4 ADDRESSES

IPv4 (Internet Protocol version 4) is the fourth version of the Internet Protocol and is the most widely used to identify devices on a network. An **IPv4 address** is a **32-bit** number that uniquely identifies a device on an IP network.

IPv4 addresses are written in **decimal format**, divided into **four octets** (each 8 bits), separated by periods. Each octet can range from 0 to 255.

IPv4 (Internet Protocol version 4) is the fourth iteration of the Internet Protocol and serves as a foundational standard for directing data packets across interconnected networks, including the internet. It utilizes a **32-bit addressing system**, represented as four decimal numbers ranging from 0 to 255 and separated by periods (e.g., **192.0.2.126**), to uniquely identify devices for communication. Although the original design provided approximately **4.3 billion addresses**, the rapid growth of the internet led to address exhaustion. This limitation has prompted the widespread adoption of **Network Address Translation (NAT)** and accelerated the transition to **IPv6**.

Format of IPv4 Address:

xxx.xxx.xxx.xxx (where each xxx is a number from 0 to 255)

Each of the four parts (called **octets**) represents 8 bits of the 32-bit address.\

3.3.3. Classless Networking:

Classless Networking is a method of allocating IP addresses and routing IP packets that **do not rely on the traditional class-based system (Class A, B, C)**. Instead, it allows for more flexible and efficient use of IP address space by using **variable-length subnet masks (VLSM)**.

Need for Classless Network:

- Classful addressing wasted IP addresses: For example, a Class B network had to allocate 65,534 hosts even if fewer were needed.
- Limited number of networks: Classful boundaries restricted how networks were divided.
- Growing Internet demand: More networks and hosts required more flexible IP address assignment.
- Reduce routing table size: CIDR aggregates routes to simplify routing.

Types:

3.3.3.1 Fixed-Length Subnet Mask (FLSM)

The Fixed-Length Subnet Mask (FLSM) method divides an IP network into several equal-sized subnets, all using the same subnet mask. This straightforward approach simplifies network planning by providing consistent subnet capacities but can lead to significant IP address wastage, as smaller segments receive the same large subnet size as larger ones. FLSM is also known as classful or traditional subnetting and is beneficial for its simplicity and for creating a predictable number of subnets.

Example Scenario:

You have a 192.168.1.0/24 network and want to divide it into 4 equal subnets.

Step 1: Determine how many bits to borrow.

/24 = 256 IPs

Need 4 subnets $\rightarrow 2^2 = 4 \rightarrow$ Borrow 2 bits

New subnet mask : /26 (24 + 2 bits)

Each subnet : $2^6 = 64$ IPs (62 usable)

Step 2: Subnet breakdown:

Subnet	Network Address	First Usable	Last Usable	Broadcast
1	192.168.1.0/26	192.168.1.1	192.168.1.62	192.168.1.63
2	192.168.1.64/26	192.168.1.65	192.168.1.126	192.168.1.127
3	192.168.1.128/26	192.168.1.129	192.168.1.190	192.168.1.191
4	192.168.1.192/26	192.168.1.193	192.168.1.254	192.168.1.255

3.3.3.2 Variable Length Subnet Mask (VLSM)

Variable Length Subnet Mask (VLSM) is a subnetting technique that allows the use of **subnet masks of different lengths** within the same network. It enables more **efficient utilization of IP address space** by assigning subnet sizes based on the specific number of hosts required in each subnet. Unlike **Fixed Length Subnet Mask (FLSM)** where all subnets are of equal size, **VLSM allows creating subnets of varying sizes**, making it ideal for networks with diverse host requirements.

Example

To better understand **Variable Length Subnet Masking (VLSM)**, let's consider an example. Suppose our company is assigned the IP address **192.168.1.0/24**. We need to divide this address space among four departments: **Engineering, Sales, Accounting, and HR**.

The user requirements for each department are:

Engineering	: 60 users
Sales	: 40 users
Accounting	: 20 users
HR	: 10 users

Using VLSM, we can efficiently allocate IP addresses by creating subnets that match each department's size, minimizing wasted addresses and making the best use of the available IP

space.



Here, we will use the below subnet masks and **CIDR values** for each department:

Engineering (60 users) /26 (64 available address) **255.255.192**

Sales (40 users) /26 (64 available address) **255.255.192**

Accounting (20 users) /27 (32 available address) **255.255.224**

HR (10 users) /28 (16 available address) **255.255.240**

Engineering (/26 255.255.255.192)

Network Address : **168.1.0**

First Host Address : **168.1.1**

Last Host Address : **168.1.62**

Broadcast Address : **168.1.63**

Sales (/26 255.255.255.192)

Network Address : **168.1.64**

First Host Address : **168.1.65**

Last Host Address : **168.1.126**

Broadcast Address : **168.1.127**

Accounting (/27 255.255.255.224)

Network Address : **168.1.128**

First Host Address : **168.1.129**

Last Host Address : **168.1.158**

Broadcast Address : **168.1.159**

Engineering (/28 255.255.255.240)

Network Address : **168.1.160**

First Host Address : **168.1.161**

Last Host Address : **168.1.174**

Broadcast Address : 168.1.175

3.3.4 IPv4 Header Format

IPv4 Header Definition

The IPv4 header is a structured portion of an IPv4 packet that contains essential control information required for routing and delivering the packet across networks. It precedes the payload (data) and provides details such as source and destination addresses, packet size, fragmentation, and error checking.

The IPv4 header has a minimum length of 20 bytes and can be extended up to 60 bytes if optional fields are included. It consists of multiple fields, each serving a specific function to ensure proper handling, forwarding, and delivery of the packet.

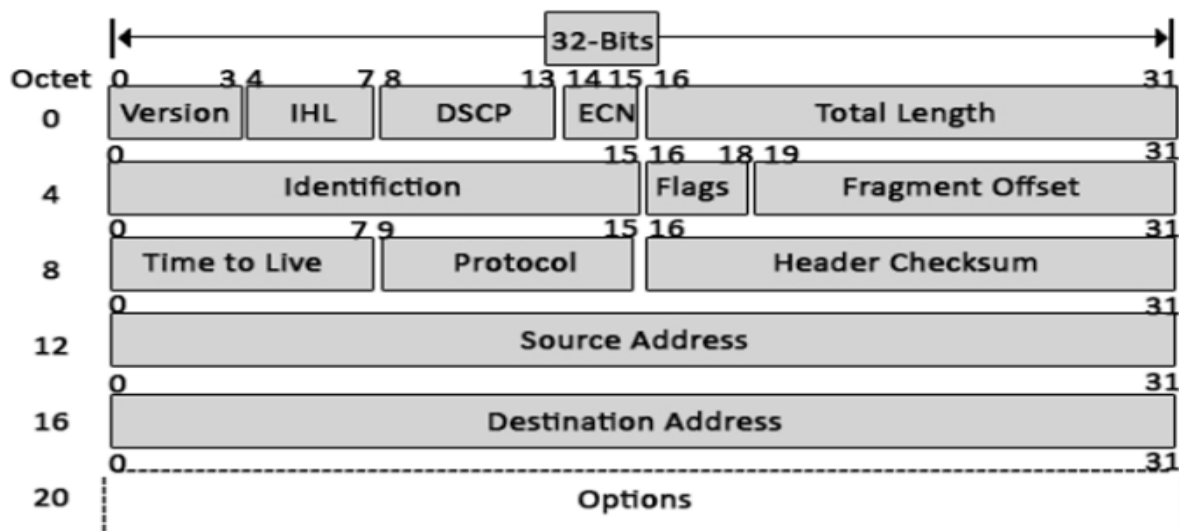


Fig No: 3.3.4.1 IPv4 Header Format

Version (4 bits): Specifies the IP protocol version. For IPv4, this value is 4.

Header Length (HLEN) (4 bits): Indicates the length of the IP header in 32-bit words. The minimum value is 5 (20 bytes), and the maximum is 15 (60 bytes).

Type of Service (8 bits): Specifies quality of service parameters such as **Low Delay**, **High Throughput**, and **High Reliability**.

Total Length (16 bits): Represents the combined length of the header and data in bytes. It ranges from a minimum of 20 bytes to a maximum of 65,535 bytes.

Identification (16 bits): A unique identifier for the packet, used to group fragments belonging to the same original IP datagram.

Flags (3 bits): Three control bits:

1. **Reserved bit:** Must always be set to zero.
2. **Don't Fragment (DF) flag:** Indicates if the packet should not be fragmented.
3. **More Fragments (MF) flag:** Indicates if more fragments follow this fragment.

Fragment Offset (13 bits): Specifies the position of this fragment's data within the original datagram. It is measured in units of 8-byte blocks, allowing a maximum offset of 65,528 bytes.

Time To Live (TTL) (8 bits): Limits the lifetime of the datagram by restricting the number of hops it can take through the network. This prevents packets from circulating indefinitely.

Protocol (8 bits): Identifies the protocol of the encapsulated data carried in the payload, such as TCP, UDP, or ICMP.

Header Checksum (16 bits): A checksum used for error-checking the integrity of the IP header.

Source IP Address (32 bits): The IP address of the sender of the packet.

Destination IP Address (32 bits): The IP address of the intended recipient.

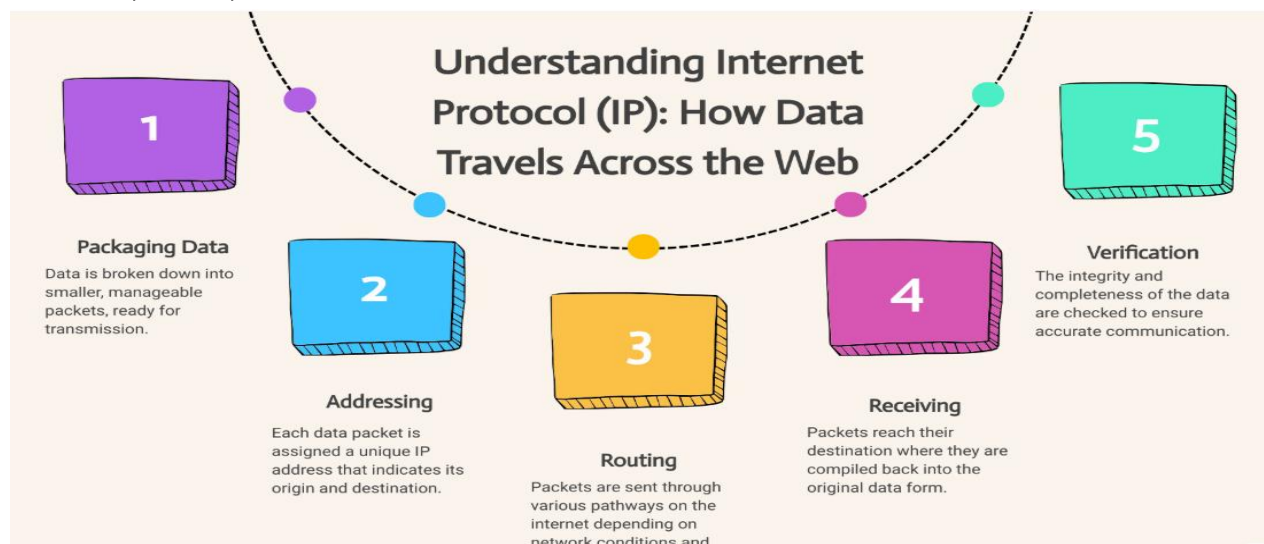
Options (variable length): Optional fields for additional information such as source routing and record routing. These are used mainly by network administrators to troubleshoot or verify the path of packets.

3.4 INTERNET PROTOCOL (IP)

The **Internet Protocol (IP)** is the **primary network layer protocol** in the TCP/IP suite, responsible for delivering **packets of data** from the source host to the destination host based on their **IP addresses**. IP is a **connectionless, best-effort delivery** protocol, meaning it does not guarantee delivery, order, or error checking. It relies on upper-layer protocols (like TCP) to ensure reliability.

3.4.1 Functions of Internet Protocol (IP)

The Internet Protocol (IP) performs several critical functions that enable data communication across interconnected networks. These functions ensure that data is correctly addressed, routed, and delivered from the source device to the destination device.



Addressing

IP assigns logical addresses (IP addresses) to devices on a network.

These addresses uniquely identify each host, allowing packets to be sent to the correct destination.

IP addressing supports hierarchical structure (network and host portions) for efficient routing.

Packetization

IP encapsulates data received from the transport layer (e.g., TCP, UDP) into IP packets (also called datagrams).

It adds an IP header containing control information necessary for routing and delivery.

Routing

IP determines the best path for packets to travel from source to destination across multiple networks.

Routers use IP addresses and routing tables to forward packets through interconnected networks.

Fragmentation and Reassembly

Networks may have different Maximum Transmission Units (MTUs) (largest packet size supported).

IP fragments large packets into smaller fragments to fit the MTU of the network. The destination reassembles these fragments into the original packet.

Encapsulation

IP encapsulates upper-layer protocol data units (such as TCP segments or UDP datagrams) within IP packets for transmission.

This enables transport protocols to operate independently of the underlying physical network.

Connectionless Delivery

IP operates in a connectionless manner; it does not establish a dedicated end-to-end connection before sending packets.

Each packet is routed independently, allowing for flexible and scalable communication.

Best-Effort Delivery

IP provides best-effort delivery, meaning it does not guarantee packet delivery, ordering, or error correction.

Reliability and error handling are managed by transport layer protocols (like TCP).

3.5 ICMPv4 (Internet Control Message Protocol for IPv4)

ICMPv4 is a Network Layer protocol that works alongside IPv4 to provide error reporting and network diagnostic functions. It plays a crucial role in maintaining efficient and reliable communication by delivering messages such as "Destination Unreachable" or "Time Exceeded". Instead of being transmitted directly over the data link layer, ICMPv4 messages are encapsulated within IP datagrams for delivery across the network.

What is ICMP?

The Internet Control Message Protocol (ICMP) is a supporting network protocol used primarily for reporting errors and handling management queries. It enables network devices such as routers to send notifications about operational issues and error conditions. For example, ICMP can report when a requested service is unavailable or when a host or router cannot be reached.

Since the IP protocol itself does not have built-in mechanisms for error reporting or correction, ICMP acts as a communication channel to convey such information. When a packet is sent to a destination, it may encounter problems along the way—such as being dropped or delayed due to network congestion—but without ICMP, the sender might remain unaware of these issues.

ICMP helps by sending back messages that inform the sender about problems like unreachable destinations, network congestion, or packet loss. This feedback is crucial for diagnosing network problems and enables devices to adjust communication routes or take corrective actions, ensuring smoother and more reliable network operations.

How Does ICMP Work?

ICMP is a crucial protocol within the IP suite, but unlike transport layer protocols such as TCP or UDP, it does not rely on them. ICMP operates as a connectionless protocol, meaning it does not establish a connection with the destination device before sending messages.

This behavior contrasts with TCP, which is a connection-oriented protocol requiring both sender and receiver to establish a connection via a handshake before exchanging data.

ICMP messages are transmitted as datagram's—self-contained packets consisting of an IP header and ICMP-specific data. Each ICMP datagram functions as an independent unit of communication, allowing error and control messages to be sent quickly and efficiently without the overhead of connection management.

3.5.1 Functions of ICMPv4

1. Error Reporting

ICMPv4 informs the source device about network issues, such as:

Destination host or network unreachable

TTL (Time to Live) expiration

Fragmentation required but not permitted

These error messages help identify and respond to delivery problems that IP alone cannot detect.

2. Network Diagnostics

ICMPv4 enables essential diagnostic tools, including:

Ping: Verifies connectivity and measures round-trip time using ICMP Echo Request and Echo Reply messages.

Traceroute: Determines the path packets take to a destination by triggering ICMP Time Exceeded messages from intermediate routers.

3. Operational Messaging

In addition to error messages, ICMPv4 communicates **operational information** between devices to support proper network behavior and monitoring.

3.5.2 ICMP Packet Format

ICMP header comes after IPv4 and IPv6 packet header.

Type (8-bit)	Code (8-bit)	Checksum (16-bit)
--------------	--------------	-------------------

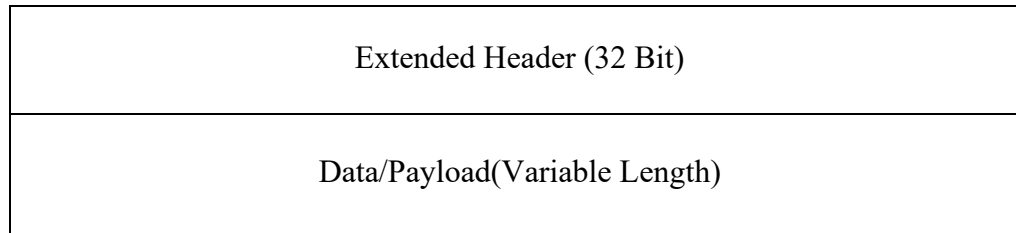


Fig No: 3.5.2.1 - ICMP Packet Format

Type (8 bits):

The first 8 bits specify the message type, which indicates the nature of the ICMP message. This helps the receiving device understand what kind of message it has received and how to process it. Common ICMP message types include:

- Type 0 — Echo Reply
- Type 3 — Destination Unreachable
- Type 5 — Redirect Message
- Type 8 — Echo Request
- Type 11 — Time Exceeded
- Type 12 — Parameter Problem

Code (8 bits):

The next 8 bits provide **additional details** related to the message type, offering more specific information about the error or control message.

Checksum (16 bits):

The final 16 bits of this section are reserved for the checksum, which verifies the integrity of the entire ICMP message. It helps ensure that the data has not been corrupted during transmission.

Following these fields, the next 32 bits from the Extended Header, which identifies the location of the issue within the IP message? A pointer indicates the exact byte position where the problem occurred, allowing the receiving device to pinpoint the cause.

3.5.3 Types of ICMP Messages

Source Quench Message

A Source Quench message is a request sent to reduce the traffic rate of packets being sent to a destination host. When the receiving host detects that packets are arriving too quickly and may cause congestion or loss, it sends a Source Quench message back to the sender. This message instructs the sender to slow down the transmission rate to prevent packet loss.

When a router or host discards packets due to congestion, ICMP uses the source IP address from those discarded packets to send a Source Quench message to the sender. The sender then reduces its transmission speed, helping to relieve congestion in the network.

If the congested router is located far from the source, ICMP may send hop-by-hop Source Quench messages, prompting each router along the path to decrease the transmission rate, thereby easing congestion gradually across the network.

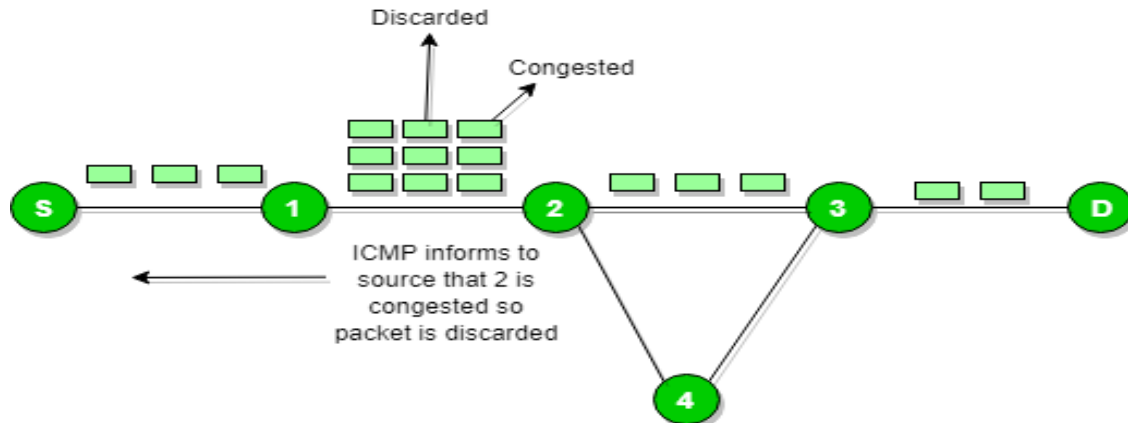


Fig.No. 3.5.3.1 Source Quench Message

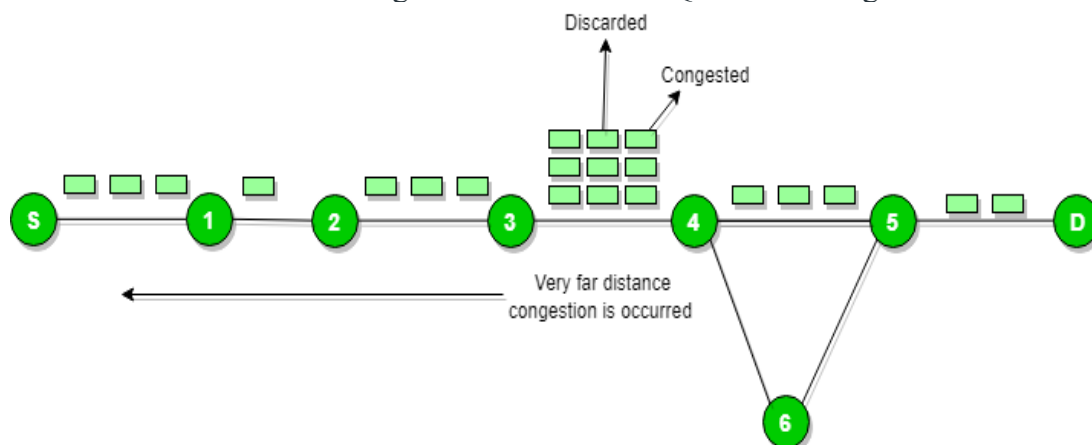


Fig.No. 3.5.3.2 Source Quench Message with Reduced Speed

Parameter Problem Message

When a router receives a packet, it verifies the IP header checksum. If the calculated checksum does not match the received checksum, the router discards the packet.

In such cases, ICMP sends a Parameter Problem message to the source IP address found in the discarded packet, notifying the sender of the issue so corrective action can be taken.

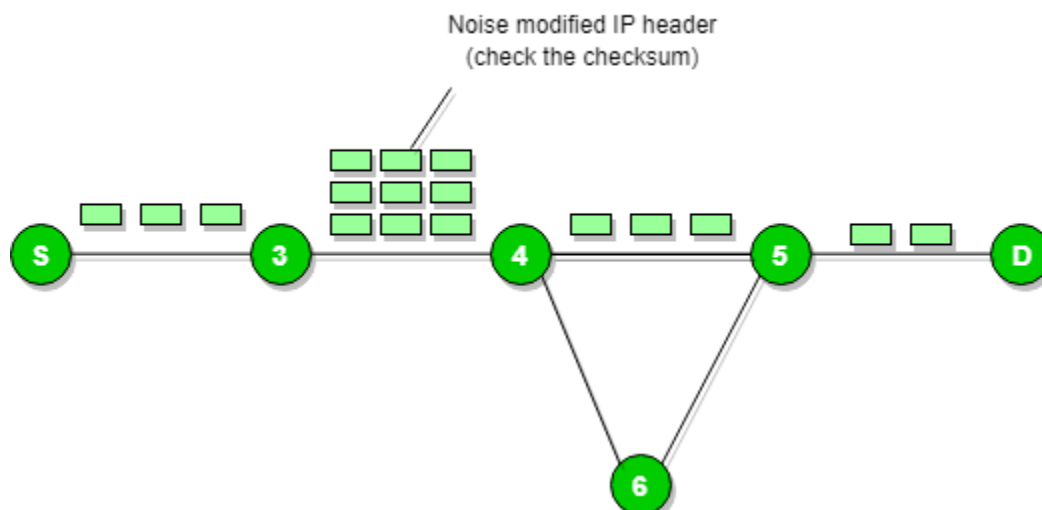


Fig.No. 3.5.3.3 Parameter Problem

Time Exceeded Message

The Time Exceeded message is generated by routers or gateways when a packet's Time To Live (TTL) field reaches zero before reaching its destination. This message informs the sender that the packet has been discarded due to exceeding its allowed lifetime in the network.

Understanding the IP header is important to fully grasp this message, as the TTL is a field within the IP header controlling how many hops a packet can make.

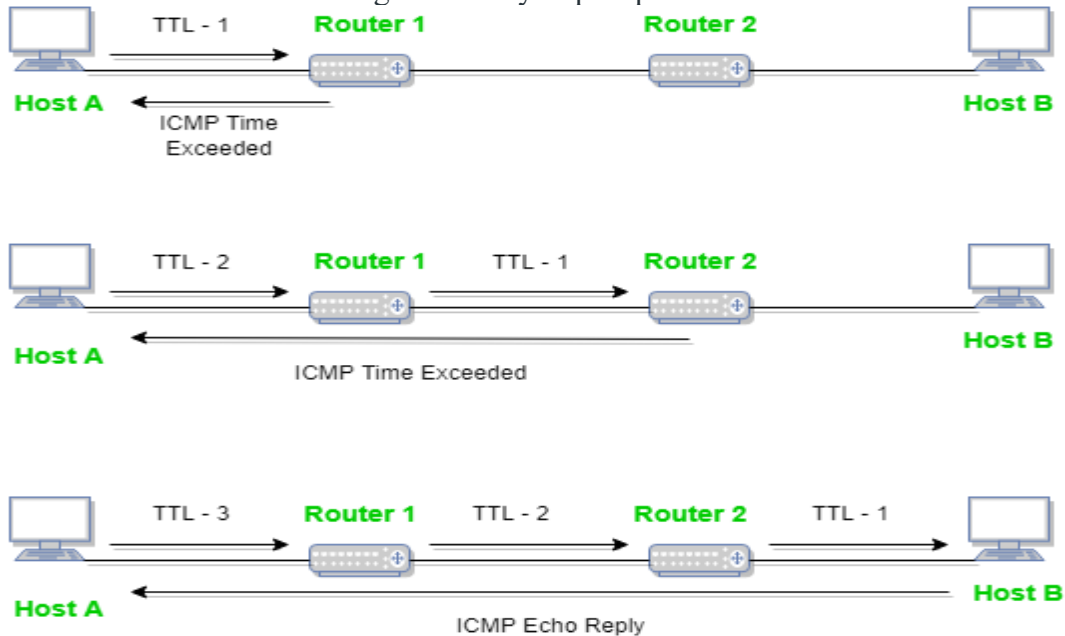


Fig. No. 3.5.3.4 Time Exceeded Message

Destination Unreachable Message

A Destination Unreachable message is sent by either the destination host or an inbound gateway to notify the sender that the destination cannot be reached for some reason.

This message can be generated due to various failures such as link issues, hardware failures, or port problems. Not only routers but also the destination host can send this ICMP error message to inform the sender about connectivity problems.

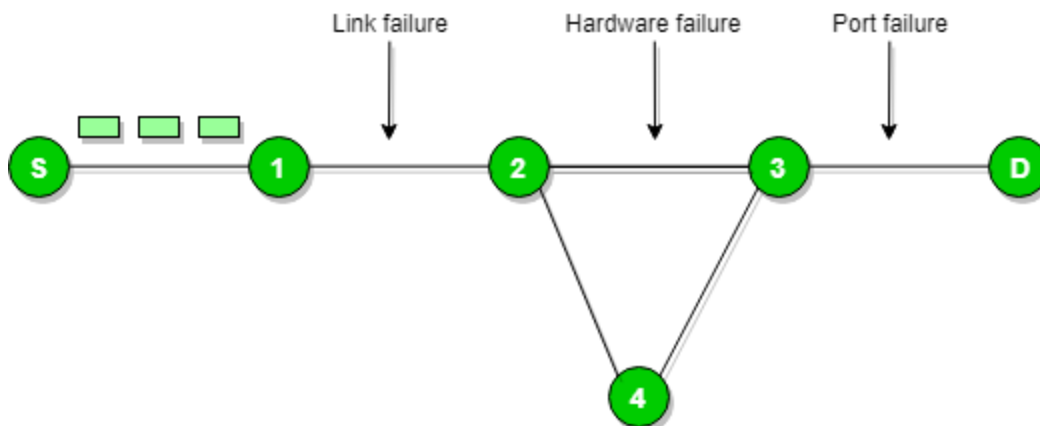


Fig. No. 3.5.3.5 Destination Un-reachable

Redirect Message

The Redirect message informs a host to update its routing information by sending packets via an alternate, more efficient route.

For example, if a host sends data to router R1, and R1 forwards the packet to router R2, but there exists a more direct path from the host to R2, then R1 will send a Redirect message to the host. This instructs the host to send future packets directly to R2, which will then forward them to the destination.

However, Redirect messages are only sent by routers (gateways) and not by end hosts. Also, if the packet contains routing information, the Redirect message is not sent, even if a better route exists.

Redirect messages also occur when packets are initially forwarded in a suboptimal direction and later redirected to the correct path.

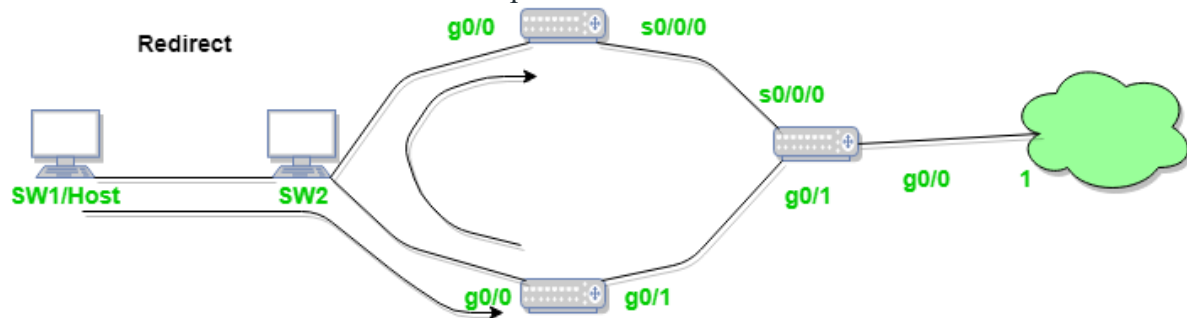


Fig No: 3.5.3.6 Redirection Message

3.6 UNICAST ROUTING ALGORITHMS: DISTANCE VECTOR AND LINK-STATE ROUTING

Introduction

Unicast routing algorithms determine the best path for sending data packets from a single source to a single destination across a network. Unlike multicast or broadcast routing, where packets are sent to multiple destinations, **unicast** involves one-to-one communication. These algorithms help routers decide how to forward packets efficiently, ensuring data reaches the intended recipient with minimal delay and optimal resource usage.

Types of Unicast Routing Algorithms

Unicast routing algorithms generally fall into two main categories:

1. **Distance-Vector Routing Algorithms**
2. **Link-State Routing Algorithms**

Each has unique characteristics, advantages, and trade-offs.

3.6.1 Distance-Vector Routing Algorithms

Overview:

Distance-vector algorithms work by having each router maintain a table (vector) of the best-known distance to every possible destination and the direction (next hop) to send packets. Routers periodically exchange these tables with their directly connected neighbors.

How It Works:

Each router sends its distance vector (routing table) to its neighbors. Upon receiving neighbors' vectors, the router updates its own table by choosing the shortest path based on a metric, usually hop count. This process continues until all routers have consistent views of the network.

Key Concepts:

Bellman-Ford Algorithm: The mathematical basis for many distance-vector protocols.

Routing by Rumor:

Routers only know what their neighbors tell them, not the entire network topology.

Periodic Updates:

Distance vectors are exchanged at regular intervals.

Common Protocols:

Routing Information Protocol (RIP): Uses hop count as the metric and has a maximum limit of 15 hops.

Interior Gateway Routing Protocol (IGRP): Cisco proprietary, uses multiple metrics like bandwidth and delay.

Advantages:

- Simple to implement.
- Requires less computational power.

Disadvantages:

- Slow convergence: Changes in topology take time to propagate.
- Susceptible to routing loops (count-to-infinity problem).
- Limited scalability due to hop count limits.

3.6.2 Link-State Routing Algorithms

Overview:

Link-state algorithms require each router to have a complete map of the network topology. Routers exchange link-state information (about their immediate neighbors and the cost to reach them) with all other routers in the network.

How It Works:

Each router discovers its neighbors and measures the cost (e.g., delay, bandwidth) to each.

Routers broadcast link-state advertisements (LSAs) to all other routers.

Each router builds a topology database from the LSAs.

Using algorithms like Dijkstra's Shortest Path First (SPF), routers calculate the shortest path to every destination.

Routing tables are then updated accordingly.

Key Concepts:

Link-State Advertisement (LSA): Information about a router's links.

Topology Database: A map of the network created from LSAs.

Dijkstra's Algorithm: Calculates shortest paths.

Common Protocols:

Open Shortest Path First (OSPF): Widely used, supports hierarchical routing.

Intermediate System to Intermediate System (IS-IS): Similar to OSPF but used mainly in service provider networks.

Advantages:

- Faster convergence than distance-vector.
- More scalable and robust.
- Reduces routing loops.

Disadvantages:

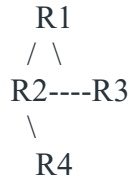
- Higher memory and CPU usage due to topology maintenance.
- More complex to implement.

Link-State Routing Algorithms

Example of Link-State Routing Algorithm: OSPF

Network Setup:

Consider a network of four routers — R1, R2, R3, and R4 — connected as follows:



Each link has a cost representing the “distance” or “metric” to traverse that link:

Link	Cost
R1 - R2	2
R1 - R3	5
R2 - R3	1
R2 - R4	3

Step 1: Each Router Discovers Its Neighbors

Each router learns the routers it’s directly connected to and measures the cost.

For example, R1 knows it is connected to R2 (cost 2) and R3 (cost 5).

Step 2: Routers Flood Link-State Advertisements (LSAs)

Each router creates an LSA describing its directly connected neighbors and the cost to reach them.

Example: R2 creates an LSA that says “I am connected to R1 (cost 2), R3 (cost 1), and R4 (cost 3).”

These LSAs are flooded to all routers in the network so every router learns the complete topology.

Step 3: Building the Topology Database

After receiving LSAs from all routers, each router builds a complete map of the network, showing all routers and links with their costs.

For example, R1’s topology database will include:

R1-R2 (2), R1-R3 (5), R2-R3 (1), R2-R4 (3)

Step 4: Calculating the Shortest Path Using Dijkstra’s Algorithm

Each router runs Dijkstra’s Shortest Path First (SPF) algorithm on the topology map.

For example, R1 calculates the shortest path to all routers:

Destination	Possible Paths Total	Cost	Selected Path
R2	R1-R2	2	R1-R2
R3	R1-R3 (cost 5)	5	R1-R2-R3 (2+1=3)
R4	R1-R2-R4 (2+3=5)	5	R1-R2-R4

R1 chooses R1- R2 - R3 for R3 because cost 3 < cost 5.

Step 5: Update Routing Table

Based on the SPF calculation, each router updates its routing table with the next hop for each destination.

For R1:

Destination	Next Hop	Cost
R2	R2	2
R3	R2	3

3.7 STATIC ROUTING

Static routing involves manually configuring fixed paths—known as static routes—for data packets to reach their destinations, instead of using dynamic routing protocols that automatically discover and update routes.

This method is typically used in simpler network environments, for managing specific traffic paths, or as backup routes. However, because any network changes require manual updates, static routing can be challenging to maintain in larger, more complex networks. Each static route is defined by a destination network address, subnet mask, and either a next-hop IP address or an exit interface.

Static routing is especially well-suited for stub networks, where only one route to a destination exists.

Static routing refers to the manual configuration of routing entries in a router's routing table.

Unlike dynamic routing, it does not adapt to network changes automatically. There are **several types of static routing**, depending on how and where they're applied.

Here are the **main types of static routing**:

1. Standard Static Route

Definition: A route manually configured to reach a specific network via a specific next-hop IP address or exit interface.

Use Case: Simple, small networks with few paths.

Example:

```
ip route 192.168.2.0 255.255.255.0 10.0.0.2
```

2. Default Static Route (Gateway of Last Resort)

Definition: A catch-all route used when no other specific route matches the destination IP address.

Use Case: Often used in edge routers to send all unknown traffic to the internet or a central router.

Example:

```
ip route 0.0.0.0 0.0.0.0 192.168.1.1
```

3. Floating Static Route

Definition: A backup static route with a higher administrative distance than the primary route.

Use Case: Provides redundancy by taking over only if the main route fails.

Example:

```
ip route 192.168.2.0 255.255.255.0 10.0.0.2 200
```

4. Recursive Static Route

Definition: A route that specifies a next-hop IP address, which must be resolved to an exit interface using another route.

Use Case: Useful when the next-hop is reachable via a different route.

Example:

```
ip route 10.10.10.0 255.255.255.0 192.168.1.2
```

5. Fully Specified Static Route

Definition: Specifies both the next-hop IP address **and** the outgoing interface.

Use Case: Used when recursive lookup is undesirable or unsupported.

Example:

```
ip route 192.168.3.0 255.255.255.0 Serial0/0 10.0.0.1
```

Summary Table:

Type	Purpose	Key Feature
Standard Static Route	Manual path to a specific destination	Direct next-hop or interface specified
Default Static Route	Route for unknown destinations	0.0.0.0/0 destination
Floating Static Route	Backup route	Higher administrative distance
Recursive Static Route	Next-hop needs resolution	Uses routing table lookup
Fully Specified Route	Explicitly defines both next-hop & interface	Avoids recursive lookup

3.8 DYNAMIC ROUTING PROTOCOLS: RIP and OSPF

3.8.1 RIP (Routing Information Protocol)

RIP (Routing Information Protocol) is one of the oldest distance-vector routing protocols used in local and wide area networks. It helps routers dynamically share information about network topology to determine the best path for forwarding packets.

RIP (Routing Information Protocol) configuration guide for Cisco routers. RIP is a distance-vector routing protocol.

Key points:

Type: Distance-vector routing protocol.

Metric: Uses **hop count** as the routing metric (the number of routers a packet must pass through).

Hop Limit: Maximum hop count allowed is **15**; any destination beyond 15 hops is considered unreachable.

Updates: Routers send full routing table updates to neighbors every 30 seconds.

Versions: There are two main versions:

- **RIPv1:** Classful routing, no subnet mask information.
- **RIPv2:** Classless routing, supports subnet masks and multicast updates.

Purpose: Helps routers dynamically learn and share routes to reach different networks without manual configuration of each route.

Basic RIP Configuration Steps (Cisco IOS)

Enter global configuration mode:

```
Router> enable
```

```
Router# configure terminal
```

Enable RIP routing protocol:

```
Router(config)# router rip
```

Specify the RIP version (optional, but recommended to use RIP version 2):

```
Router(config-router)# version 2
```

Add the networks that RIP should advertise and listen on:

```
Router(config-router)# network [network-address]
```

Example: If your router has interfaces in networks 192.168.1.0/24 and 10.0.0.0/8, add:

```
Router(config-router)# network 192.168.1.0
```

```
Router(config-router)# network 10.0.0.0
```

Note: You specify the classful network address (not subnet mask).

(Optional) Disable automatic route summarization if you use discontinuous subnets:

```
Router(config-router)# no auto-summary
```

Exit configuration mode and save the configuration:

```
Router(config-router)# exit
```

```
Router(config)# exit
```

```
Router# write memory
```

3.8.2 OSPF (Open Shortest Path First):

OSPF (Open Shortest Path First) is a widely used **link-state** routing protocol designed for IP networks. It dynamically discovers and maintains routing information, providing fast convergence and scalability for large and complex networks.

Key Features of OSPF:

Type: Link-state routing protocol.

Metric: Uses **cost** as its routing metric, which is usually based on interface bandwidth.

Hierarchical Design: Supports **areas** to optimize traffic and reduce routing overhead.

Convergence: Fast convergence — reacts quickly to network changes.

Protocol: Uses the **Dijkstra shortest path first (SPF)** algorithm to calculate the best path.

Supports VLSM and CIDR: Fully supports variable-length subnet masks and classless routing.

Multivendor: OSPF is an open standard (defined in RFC 2328), so it works across different vendors' equipment.

Authentication: Supports authentication for routing updates to improve security.

Updates: Only sends updates when there is a change, rather than periodic full updates (unlike RIP).

How OSPF Works:

Neighbor Discovery: Routers discover OSPF neighbors using hello packets.

Database Synchronization: Routers exchange link-state advertisements (LSAs) to build a consistent map of the network.

SPF Calculation: Each router runs the Dijkstra algorithm on the collected LSAs to compute the shortest path to each network.

Routing Table Update: The best routes are placed into the routing table.

Advantages of OSPF:

Scales well in large networks.

Fast convergence and loop-free topology.

Efficient bandwidth use due to incremental updates.

Supports complex topologies via areas.

Flexible and highly configurable.

3.9. IPV6 ADDRESSING

IPv6 is the latest version of the Internet Protocol, designed to replace IPv4. It provides a much larger address space and improved features for modern networking.

IPv6 Address Format

Length: 128 bits (compared to IPv4's 32 bits).

Representation: Written as eight groups of four hexadecimal digits, separated by colons.

Example:

2001:0db8:85a3:0000:0000:8a2e:0370:7334

IPv6 Address Compression Rules

To simplify notation:

Leading zeros in any 16-bit block can be omitted:

2001:0db8:0000:0000:0000:0000:1428:57ab

Becomes

2001:db8:0:0:0:0:1428:57ab

Types of IPv6 Addresses

Address Type	Description	Example
Unicast	One-to-one communication	2001:db8::1
Multicast	One-to-many communication	ff02::1
Anycast	One-to-nearest communication (special use)	Same as unicast but assigned to multiple devices

3.9.1. IPv6 PROTOCOL

Header Format

The IPv6 header is designed to be **simpler and more efficient** than the IPv4 header. It has a fixed size of **40 bytes**, which helps speed up packet processing.

Version	Traffic Class	Flow Label	
Payload Length		Next Header	Hop Limit
Source Address			
Destination Address			

Fig No.3.9.1 - IPv6 Header Format

- Version (4 bits)** : Always set to 6.
- Traffic Class (8 bits)** : Used for packet priority and QoS.
- Flow Label (20 bits)** : Enables special handling of packets, like real-time traffic.
- Payload Length (16 bits)** : Size of the payload (data + extension headers).
- Next Header (8 bits)** : Indicates the type of the next header or protocol (e.g., TCP=6, UDP=17, ICMPv6=58).
- Hop Limit (8 bits)** : Same as TTL in IPv4; prevents infinite looping.
- Source Address (128 bits)** : The IPv6 address of the sender.
- Destination Address (128 bits)**: The IPv6 address of the intended recipient.

Version (4 bits):

The Version field is 4 bits long and always set to the value 6 in IPv6 packets, indicating the protocol version. This field helps devices and routers quickly identify the packet as an IPv6 packet, ensuring it is processed correctly according to the IPv6 standards rather than IPv4 or any other protocol.

Traffic Class (8 bits):

The Traffic Class field is used to classify and prioritize packets for quality of service (QoS) purposes. It allows routers and switches to differentiate between types of traffic, such as giving higher priority to real-time applications like voice or video streaming, and lower priority to bulk data transfers, thereby managing network congestion and ensuring efficient delivery of time-sensitive data.

Flow Label (20 bits):

The Flow Label field identifies packets that belong to the same communication flow, which is a sequence of packets sent from a source to a destination requiring special handling, such as real-time traffic. By labeling these flows, routers can recognize and apply consistent treatment to packets without inspecting deeper headers, improving performance for applications that need steady and predictable network behavior.

Payload Length (16 bits):

This field specifies the size of the data payload following the IPv6 header, including any extension headers and the upper-layer data like TCP or UDP segments. It enables the receiving device to know where the IPv6 packet ends, with a maximum value of 65,535 bytes, while larger payloads use special jumbo payload options to handle sizes beyond this limit.

Next Header (8 bits):

The Next Header field indicates the type of the header immediately following the IPv6 base header, which could be a transport layer protocol like TCP or UDP, or an extension header such as routing or fragmentation information. This field guides the processing of the packet by identifying the structure of the data that follows, ensuring correct interpretation and routing.

Hop Limit (8 bits):

The Hop Limit field limits the lifetime of a packet by specifying the maximum number of routers (hops) it can pass through before being discarded. Each router that forwards the packet decreases this value by one, and when it reaches zero, the packet is dropped to prevent it from circulating indefinitely, similar to the Time to Live (TTL) field in IPv4.

Source Address (128 bits):

The Source Address field contains the IPv6 address of the device that originally sent the packet. This address is used by the destination device to know where the packet came from, enabling it to send replies or acknowledgments back to the sender. It can be a unicast address uniquely identifying the sender on the network.

Destination Address (128 bits):

The Destination Address specifies the IPv6 address of the intended recipient of the packet. This address directs routers and switches in the network to forward the packet towards the correct endpoint and can represent a single device (unicast), a group of devices (multicast), or the nearest device in a group (anycast), depending on the addressing type used.

3.10. SOFTWARE-DEFINED NETWORKS

Software-Defined Networking is a revolutionary approach to managing and controlling computer networks. Traditionally, each network device like a switch or router makes its own decisions about where to send data (this is called the **control plane**), and it also forwards the actual data packets (**data plane**). In SDN, these two functions are separated.

Key Components

An SDN system typically includes:

SDN Controller: A logically centralized software entity that functions as the “brain” of the network, managing and orchestrating the entire infrastructure.

Southbound APIs: Interfaces that allow the SDN controller to communicate with and configure individual network devices such as switches, routers, and access points. OpenFlow is a common example of a southbound protocol.

Northbound APIs: Interfaces that facilitate communication between the SDN controller and external applications, policy engines, or orchestration tools.

SDN Works

Separation of Planes: Unlike traditional networks where control and data planes are combined within each device, SDN separates these functions.

Centralized Control: The SDN controller centrally decides network traffic routing and policy enforcement.

Programmable Network: Network administrators can dynamically adjust network behavior, traffic flows, and security settings via APIs from a single point, eliminating the need to configure each device individually.

Benefits of SDN

Agility and Flexibility: Networks can be quickly adapted to evolving business needs.

Simplified Management: Centralized control streamlines network administration and automation.

Enhanced Security: Centralized policy enforcement allows consistent security measures across the entire network.

Cost Efficiency: Improved resource utilization and simpler management reduce operational expenses.

Innovation: Open platforms and APIs enable faster rollout of new protocols and services.

3.10.1. SDN Architecture

In traditional networks, each switch contains both its own control plane and data plane. Switches share topology information with each other to build forwarding tables that determine where to send data packets. In contrast, Software-Defined Networking (SDN) separates the control plane from the switches and assigns it to a centralized SDN controller. This centralization enables network administrators to manage traffic through a single interface rather than configuring each switch individually. The data plane remains within the switches and is responsible for forwarding packets according to flow tables programmed by the controller. These flow tables include match criteria (such as input port and packet headers) and instructions (such as forwarding, dropping, or modifying packets). When a packet does not match any existing flow entry, the switch queries the controller, which then installs a new flow entry to guide the packet's handling. A standard SDN architecture is organized into three distinct layers.

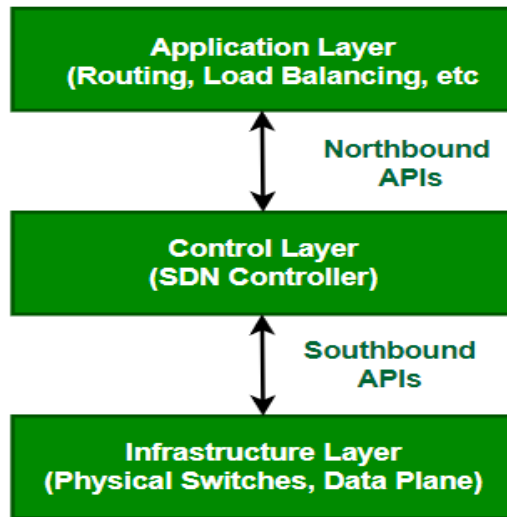


Fig. No: SDN Architecture

Application Layer: This layer includes typical network applications such as intrusion detection systems, firewalls, and load balancers.

Control Layer: This layer contains the SDN controller, which serves as the network's brain. It also provides hardware abstraction, allowing applications above it to interact with the network without needing to manage the underlying devices directly.

Infrastructure Layer: Composed of physical switches that form the data plane, this layer handles the actual forwarding of data packets.

These layers communicate through specific interfaces: **northbound APIs**, which connect the application layer to the control layer, and **southbound APIs**, which link the control layer to the infrastructure layer.

UNIT IV

TRANSPORT LAYER

Introduction- Transport layer Protocols-Simple-Stop-and-Wait-Go-Back-N-Selective Repeat-Piggy backing-UDP-TCP-SCTP-Quality of Services: Data Flow Characteristics-Flow control to Improve QoS

INTRODUCTION:

Transport layer is the 4th layer of the OSI model and the foundation of the Internet model. It accepts service requests from the session layer and forwards requests to the network layer. Ensures transparent data transfer between host systems. Provides end-to-end communication and maintains the required quality of service for applications. It is the first true end-to-end layer and is implemented in all End Systems (ES). Together with the network layer, the transport layer forms the backbone of the protocol hierarchy. While the network layer is responsible for delivering packets from one host to another—either through datagrams or virtual circuits—the transport layer ensures that data is reliably transferred from a specific process on the source machine to the correct process on the destination machine.

TRANSPORT LAYER PROTOCOLS:

The transport service defines how the transport layer supports the application layer. To make this more concrete, we will first look at some basic service primitives (operations available to applications) and then study the socket interface commonly used on the Internet.

Services Provided to the Upper Layers:

The main purpose of the transport layer is to deliver efficient, reliable, and cost-effective communication between processes running on different hosts. To achieve this, it relies on the network layer but adds its own mechanisms for error detection, retransmission, and reliability.

The component that actually carries out these tasks is called the transport entity. Depending on the system design, the transport entity may be implemented in the operating system kernel, a library used by network applications, a separate user-level process, or even on the network interface card.

Just as the network layer can provide connection-oriented or connectionless services, the transport layer also offers both.

- Connection-oriented service follows three phases: connection establishment, data transfer, and connection release.
- Connectionless service works without setting up a connection, but it is less efficient if the underlying network is connection-oriented.
- If a network connection breaks halfway during a transfer, the transport layer can establish a new one and resume from where it left off.

This ability makes the transport layer crucial—it shields applications from the imperfections of the network.

Thanks to transport primitives (implemented as standard library calls), application developers can write programs that work across different networks without needing to worry about varying network interfaces or reliability levels.

Thus, the transport layer is often seen as the boundary between service providers (layers 1–4) and service users (layers above 4).

Transport Service Primitives

To make the transport service accessible to applications, the transport layer exposes a set of primitives (operations). While each protocol may have its own set, the basic idea is similar everywhere: applications need a way to create a connection, send data, receive data, and release the connection.

Consider a simple transport service with five primitives:

- LISTEN – Wait until a connection request is received.
- CONNECT – Actively try to establish a connection.
- SEND – Transmit data to the peer process.
- RECEIVE – Block until data is received.
- DISCONNECT – Release the connection.

These primitives allow applications to set up a session, exchange data, and close the session once finished. For example, in a client–server model:

- The server waits using LISTEN.
- The client initiates communication using CONNECT.
- Once connected, both sides use SEND and RECEIVE.
- Finally, either side can call DISCONNECT.

Behind the scenes, the transport entities handle complexities like acknowledgements, retransmissions, timers, and error control. To applications, the connection simply looks like a reliable bit pipe—whatever data is sent arrives in the same order, error-free.

Berkeley Sockets A practical implementation of transport primitives can be seen in the socket API, introduced in Berkeley UNIX (4.2BSD) in 1983. Sockets quickly became the standard interface for Internet programming and are still widely used today, with equivalents such as Winsock in Windows.

The main socket primitives are:

- SOCKET – Create a new communication endpoint.
- BIND – Attach a local address (IP + port) to the socket.
- LISTEN – Announce willingness to accept connections and set a queue size.
- ACCEPT – Passively wait for and establish an incoming connection.
- CONNECT – Actively initiate a connection.
- SEND / RECEIVE – Exchange data once connected.
- CLOSE – Release the connection.

For servers, the typical sequence is: SOCKET → BIND → LISTEN → ACCEPT.

For clients, it is: SOCKET → CONNECT.

Once connected, both sides use SEND and RECEIVE, and eventually CLOSE the connection.

Sockets are most commonly used with TCP, providing a reliable byte stream. However, they can also work with connectionless services like UDP, in which case primitives such as SENDTO and RECEIVEFROM are used.

Over time, new protocols like SCTP (Stream Control Transmission Protocol) and SST (Structured Stream Transport) have extended the socket concept to support multiple related streams, mixed connection types, and multipath communication.

A Simple Client–Server Example Using Sockets

To make the transport service more concrete, let us examine a minimal client–server application that transfers files across a network using TCP sockets. This example demonstrates how two processes can establish a connection, exchange messages, and terminate communication in a controlled way.

The Server

The server plays a passive role—it waits for clients to connect and responds to their requests. Its operation can be summarized as follows:

- Initialize communication structures. The server begins by preparing a data structure that will hold its IP address and port number. Functions such as `htonl` and `htons` are used to convert values into a standard representation.
- Create and configure a socket. A TCP socket is created. The server enables the port reuse option so that it can be restarted immediately.
- Bind and listen. The socket is bound to the local address and port, after which the server announces its willingness to accept incoming connections by calling `listen`. A backlog queue is set up for multiple connection requests.
- Accept connections. The `accept` call blocks until a client requests a connection. When this happens, the server obtains a new socket dedicated to that client.
- Process the request. Once connected, the server reads the name of the requested file from the client. If the file exists, the server reads it block by block and sends the data back across the socket. After the file is transmitted, both the file and the client connection are closed.
- Repeat forever. The server then returns to its main loop, ready to handle the next client.

The Client

The client is the active participant. Its responsibilities are simpler but equally important:

- Check arguments. The client must be invoked with two arguments: the server’s hostname and the name of the file to be fetched.
- Resolve the server address. Using the Domain Name System (DNS), the client translates the server’s name into its corresponding IP address.
- Create a socket and connect. A TCP socket is created, and the client attempts to connect to the server at the specified port.
- Send the file request. The client transmits the desired filename to the server, making sure to include a terminating character.
- Receive and display the file. After sending the request, the client enters a loop where it reads data blocks from the socket and writes them directly to standard output. When the file transfer is complete, the program exits.

How It Works Together

Suppose the server is already running on a machine called `flits.cs.vu.nl`. A typical client invocation might look like this:

```
client flits.cs.vu.nl /usr/tom/filename > outputfile
```

Here, the client requests `/usr/tom/filename` from the server, and the retrieved contents are redirected into `outputfile`. The server remains running, so the client can repeat this process to request other files.

Observations and Limitations

- Portability: Functions like `htons` and `htonl` ensure that the software works correctly across different hardware architectures.
- Error handling: The example includes only basic error reporting.
- Sequential operation: Because this server handles only one client at a time, its performance is limited.
- Security and robustness: The implementation assumes that filenames fit into the buffer and that requests arrive atomically.

Despite its simplicity, this program illustrates the essential mechanics of a working Internet file server.

4.3 The Simple Stop-and-Wait Protocol

4.3.1 The Foundation of Reliable Transfer

The Stop-and-Wait protocol is the simplest automatic repeat request (ARQ) mechanism used for reliable data transfer. It is a fundamental concept that ensures a sender does not overwhelm a receiver with data. The core principle is straightforward: the sender transmits a single frame (or packet) and then waits for an acknowledgment (ACK) from the receiver before sending the next one.

How it Works:

1. Sender's Side:

- The sender sends one frame to the receiver.
- After sending the frame, the sender starts a timer and waits for an ACK.
- If the ACK is received before the timer expires, the sender proceeds to transmit the next frame in the sequence.
- If the timer expires before the ACK is received (indicating the frame or the ACK was lost), the sender retransmits the same frame.

2. Receiver's Side:

- The receiver waits for a frame to arrive.
- Upon receiving a frame, it sends an ACK back to the sender to confirm its successful reception.
- The receiver then waits for the next frame.

This process ensures that frames are delivered in the correct order and provides a basic level of error control. To handle duplicate frames (which can occur if an ACK is lost and the sender retransmits), frames and ACKs are numbered. In the simplest case, sequence numbers 0 and 1 are used alternately. The receiver only accepts a frame with the sequence number it is expecting.

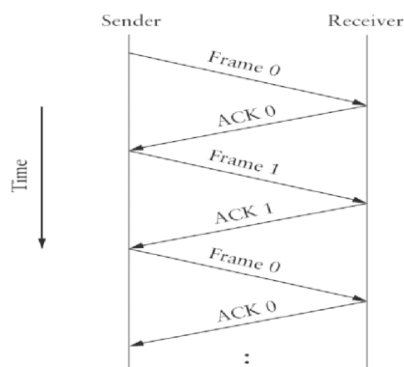


Figure 4.1: Basic Stop-and-Wait Operation. This diagram shows the ideal sequence where the sender sends Frame 0, receives ACK 0, then sends Frame 1 and receives ACK 1.

4.3.2 Handling Errors in Stop-and-Wait

Error Scenarios in Stop-and-Wait

The Stop-and-Wait protocol must be robust enough to handle several potential error scenarios that can occur in an unreliable network.

Scenario 1: Lost Frame

If a data frame sent by the sender is lost in transit, the receiver will never receive it.

- **Sender Action:** The sender's timer for the lost frame will expire.
- **Recovery:** Upon timeout, the sender retransmits the same frame. The receiver, which was still waiting for that frame, will receive it and proceed as normal.

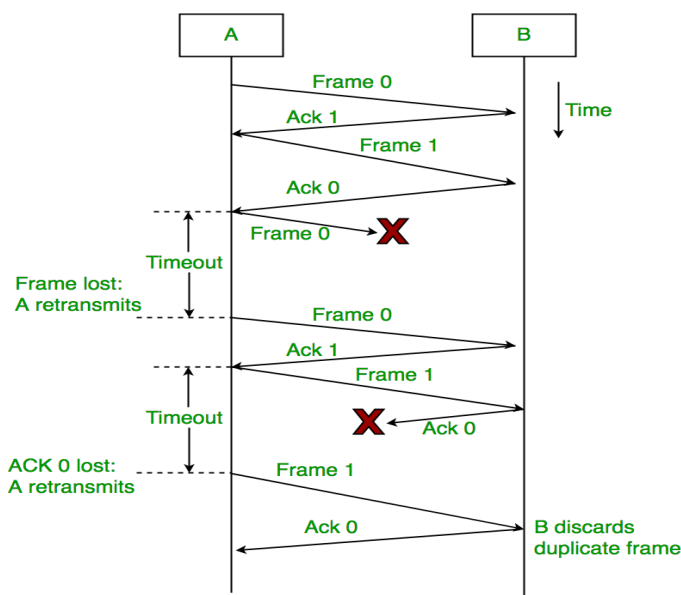


Figure 4.2: Lost Frame. This diagram shows the sender transmitting a frame that is lost. The sender's timer expires, prompting a retransmission which is then successfully received.

Scenario 2: Lost Acknowledgment (ACK)

If the data frame is successfully received, but the ACK sent back by the receiver is lost.

- **Sender Action:** The sender's timer will expire because it never received the ACK. The sender, assuming the frame was lost, will retransmit it.
- **Receiver Action:** The receiver will get a duplicate copy of the frame. Because the frame has a sequence number the receiver has already processed, it recognizes it as a duplicate, discards it, and retransmits the ACK for that frame.

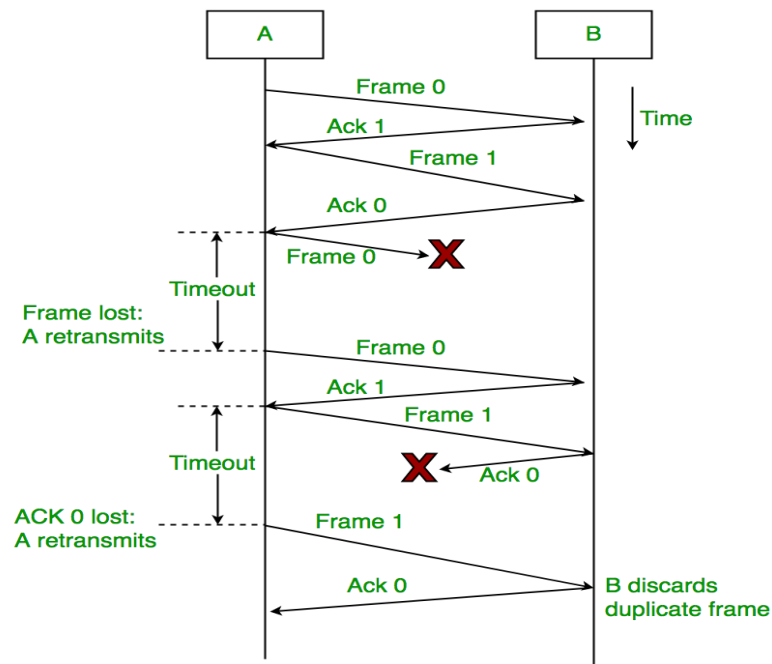


Figure 4.3: Lost Acknowledgment. This diagram shows the receiver successfully getting a frame but its ACK is lost. The sender times out, retransmits the frame, and the receiver discards the duplicate and resends the ACK.

The efficiency is heavily dependent on the propagation delay. In networks with a long propagation delay (like satellite communications) or very high data rates (where T_t is small), the efficiency of Stop-and-Wait becomes extremely low.

The product of bandwidth and delay, often called the **Bandwidth-Delay Product**, represents the number of bits that can be "in flight" on the network link. In Stop-and-Wait, we are severely underutilizing this capacity. This inefficiency was the primary motivation for developing more advanced protocols like Go-Back-N and Selective Repeat.

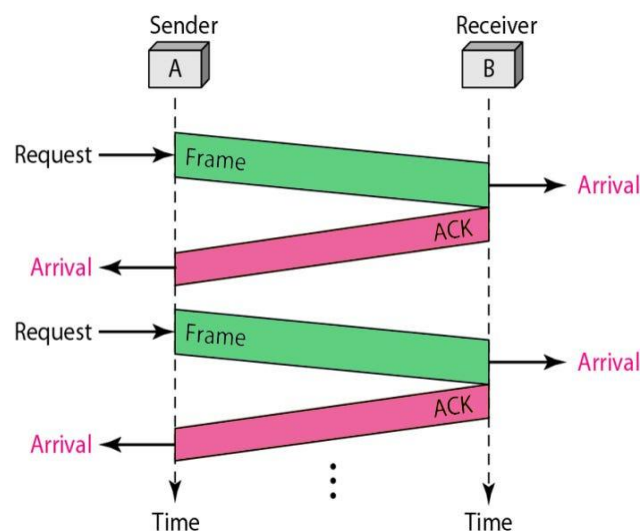


Figure 4.4: Channel Utilization. A visual representation showing the sender being active for a short period (T_t) and then idle for a long period ($2 * T_p$) while waiting for the ACK.

4.4 Go-Back-N ARQ Protocol

Improving Efficiency with Pipelining: Go-Back-N ARQ

To overcome the inefficiency of Stop-and-Wait, the Go-Back-N (GBN) ARQ protocol was developed. GBN allows the sender to transmit multiple frames without waiting for an acknowledgment for each one. This technique is known as **pipelining**.

4.4.1 The Sliding Window Concept:

Go-Back-N uses a "window" of a fixed size, N , which represents the maximum number of unacknowledged frames the sender can have in transit.

- **Sender's Window:** The sender maintains a window of size N . It can transmit all the frames within this window. The window slides forward as ACKs are received. For example, if the window size is 4, the sender can send frames 0, 1, 2, and 3. When it receives an ACK for frame 0, the window slides to now include frames 1, 2, 3, and 4.
- **Receiver's Window:** In Go-Back-N, the receiver maintains a simple window of size 1. It only looks for one specific frame at a time—the next one in the sequence. Any frame that arrives out of order is discarded.

How it Works:

- **Sender:** The sender can transmit up to N frames. It maintains a timer for the oldest unacknowledged frame.
- **Receiver:** The receiver expects frames to arrive in order. When frame i is received correctly, it sends an ACK for i . This ACK is **cumulative**, meaning it acknowledges the receipt of frame i and all frames before it.
- **Out-of-Order Frames:** If the receiver gets frame $i+2$ but was expecting frame $i+1$, it discards $i+2$. It continues to send ACKs for the last correctly received in-order frame (i).

4.4.2 Error Handling in Go-Back-N

Go-Back-N: Handling Lost Frames and Timeouts

The name "Go-Back-N" comes from its behavior when an error occurs. If a frame is lost or an ACK is delayed, the protocol "goes back" and retransmits all frames starting from the lost or unacknowledged one.

Scenario: Lost Data Frame

Suppose the sender sends frames 0, 1, 2, 3, 4, 5. Frame 2 gets lost.

- **Receiver Action:** The receiver gets frames 0 and 1 and sends ACKs for them. It never receives frame 2. When frame 3 arrives, the receiver discards it because it is out of order (it was expecting frame 2). It also discards frames 4 and 5 as they arrive.
- **Sender Action:** The sender has a timer running for frame 2 (the oldest unacknowledged frame). This timer will eventually expire.
- **Recovery:** When the timer for frame 2 expires, the sender **goes back** and retransmits frame 2, and all subsequent frames that were already sent (frames 3, 4, and 5).

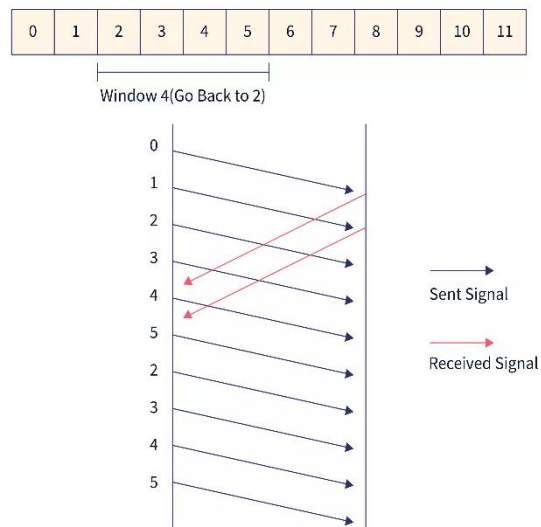


Figure 4.5: Lost Frame in Go-Back-N. This timeline shows frame 2 being lost. The receiver discards subsequent frames (3, 4, 5). The sender's timer for frame 2 expires, and it re-sends frame 2 and all the frames after it.

Disadvantage of this Approach:

The main drawback of Go-Back-N is that it can lead to the retransmission of a large number of frames, many of which may have been successfully received the first time. This is particularly problematic on networks with high error rates or long delays.

4.4.3 Go-Back-N Window Size

Title: Determining the Optimal Window Size for Go-Back-N

The size of the sender's window (N) is a critical parameter. It must be large enough to keep the communication pipe full but is limited by the number of available sequence numbers. If we use m bits for the sequence number, there are 2^m possible sequence numbers (from 0 to $2^m - 1$).

The maximum window size for Go-Back-N is $N = 2^m - 1$.

Why not 2^m ?

Consider a case with $m=2$ (sequence numbers 0, 1, 2, 3). If the window size N was 4 (2^2), a dangerous ambiguity could occur:

1. Sender sends frames 0, 1, 2, 3.
2. Receiver gets all frames and sends cumulative ACKs for all of them.
3. **All ACKs are lost.**
4. The sender's timer expires and it retransmits frame 0.

5. The receiver has already moved its window forward, expecting the *next* cycle of frames (0, 1, 2, 3). It accepts the retransmitted frame 0 as a **new** frame, not a duplicate. This violates the protocol's correctness.

By limiting the window size to $2^m - 1$, this ambiguity is avoided.

4.5 Selective Repeat ARQ

Optimizing Retransmissions with Selective Repeat ARQ

Selective Repeat (SR) ARQ is an enhancement over Go-Back-N that addresses its major inefficiency. As its name implies, Selective Repeat only retransmits those frames that are actually lost or corrupted.

Key Differences from Go-Back-N:

- **Receiver's Window:** The receiver in SR also has a window of size N. This allows it to accept and **buffer** out-of-order frames.
- **Acknowledgments (ACKs):** ACKs in SR are **individual**, not cumulative. Each ACK acknowledges a single, specific frame.
- **Sender's Timers:** The sender maintains a separate timer for each unacknowledged frame.

How it Works:

Sender: Transmits a window of N frames, starting a timer for each.

Receiver:

- Accepts any frame that falls within its receiving window, regardless of order.
- Sends a specific ACK for each correctly received frame.
- Buffers any out-of-order frames until the missing frames arrive.
- Once a contiguous block of frames is assembled, it delivers them to the application layer and slides its window forward.

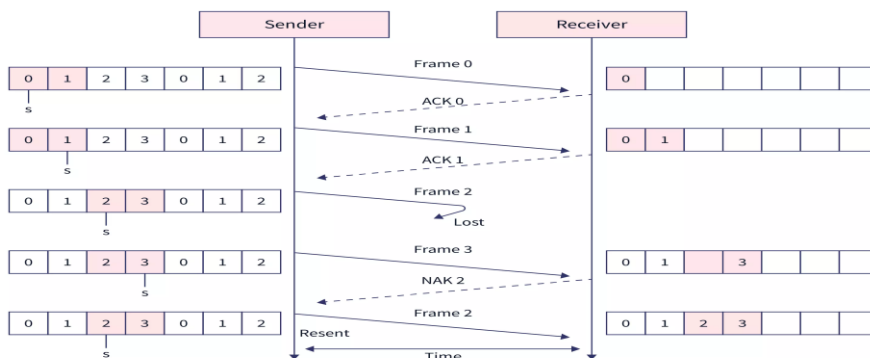


Figure 4.6 : Selective Repeat Operation. This timeline shows frame 2 being lost. The receiver gets and buffers frames 3 and 4, sending individual ACKs for them. The sender's timer for frame 2 expires, and it retransmits *only* frame 2.

4.6 Piggybacking

Piggybacking: Maximizing Efficiency in Bidirectional Communication

In many network applications, such as the client-server model, data flows in both directions. In these scenarios, sending dedicated acknowledgment (ACK) frames for every received data frame can be inefficient. **Piggybacking** is an optimization technique that combines an acknowledgment with an outgoing data frame.

The Core Mechanism:

Instead of sending a separate ACK, the acknowledgment is "piggybacked" onto a data frame that is already heading in the other direction. This is possible because the headers of most transport layer protocols have specific fields for both sequence numbers and acknowledgment numbers.

The process works as follows:

1. Host A sends a data frame with sequence number X to Host B.
2. Host B receives this frame correctly.
3. Shortly after, Host B has its own data to send to Host A.
4. Host B constructs its data frame with its own sequence number Y. Before sending, it places the acknowledgment for A's frame into the acknowledgment field of its outgoing data frame's header.
5. Host A receives the frame from B. It accepts the data with sequence number Y and, by inspecting the acknowledgment field, also learns that its frame X was received successfully.

The Role of Timers and Standalone ACKs:

A key question arises: what happens if a host receives data but has no data of its own to send back? It cannot wait indefinitely to piggyback the ACK, because the original sender's retransmission timer might expire.

To solve this, a receiving host uses an **acknowledgment timer**.

- When a data frame arrives, the receiver starts this timer.
- If an outgoing data frame becomes available before the timer expires, the ACK is piggybacked.
- If the timer expires and there is still no data to send, the receiver sends a **separate, standalone ACK frame**. This ensures that the acknowledgment is sent in a timely manner.

4.7.1 UDP

UDP, which stands for User Datagram Protocol, gives applications a direct way to send packets, called datagrams, over an IP network without needing to establish a formal connection first. It is officially described in the document RFC 768.

User Datagram Protocol (UDP) is a connectionless, unreliable transport protocol. UDP adds process-to-process communication to best-effort service provided by IP.

- ✓ UDP is a very simple protocol using a minimum of overhead.
- ✓ UDP is a simple demultiplexer, which allows multiple processes on each host to communicate.
- ✓ UDP does not provide flow control, reliable or ordered delivery.
- ✓ UDP can be used to send small message.

- ✓ Sending a small message using UDP takes less interaction between the sender and receiver.
- ✓ UDP allow processes to indirectly identify each other using an abstract locator called port or mailbox

4.7.2 The UDP Header

A UDP transmission consists of a simple 8-byte header followed by the application's data (the payload). The primary advantage of using UDP over raw IP is that it adds source and destination "ports".

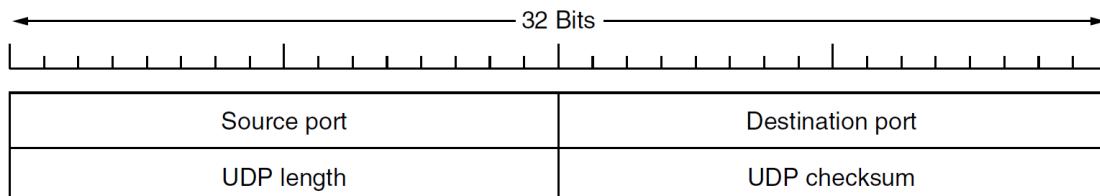


Figure 4.8 UDP Header

The header contains the following fields:

Source Port & Destination Port: Ports are used to identify the specific application that sent the data and the specific application that should receive it. Think of an IP address as a building's street address; the port number is the apartment number inside that building. Without ports, the operating system wouldn't know which application to deliver an incoming packet to. The source port is important so the receiving application knows where to send a reply.

UDP Length: This field specifies the total length of the UDP segment, which includes the 8-byte header and the data itself. The maximum length is 65,515 bytes.

UDP Checksum: This is an optional field used to check for errors in the packet. It calculates a value based on the UDP header, the data, and a special "pseudoheader" from the IP layer. While you can turn it off, it's not recommended unless the data's integrity doesn't matter (like for some real-time digitized speech)

The IP Pseudoheader

The checksum calculation includes a conceptual "pseudoheader" that contains information from the IP layer, such as the source and destination IP addresses and the protocol number (17 for UDP). Including this helps detect if a packet was delivered to the wrong machine by mistake.

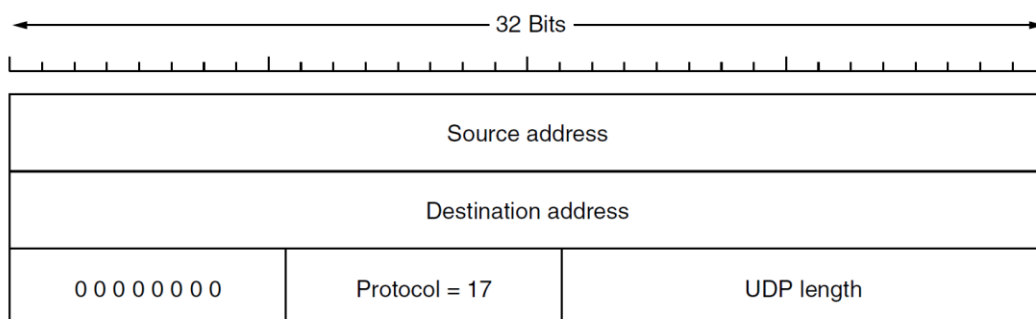


Figure 4.9 The IPv4 pseudoheader included in the UDP checksum

It shows fields for Source Address, Destination Address, a zero-padded field, the Protocol number (17), and the UDP Length.]

What UDP Does and Doesn't Do

UDP is intentionally minimal. Its main job is to provide a way for applications on different machines to talk to each other using ports and to offer an optional check for data corruption.

Here's what UDP does not do:

- Flow Control (it won't slow down if the receiver is overwhelmed)
- Congestion Control (it won't slow down if the network is busy)
- Retransmission (it won't resend a packet that gets lost)
- Applications that need these features must implement them on their own.

4.7.3 When is UDP Useful?

UDP is ideal for applications where speed is critical and where losing a packet isn't a disaster. It works very well in client-server scenarios that involve a short request and a short reply. If a request or reply is lost, the client can simply time out and send the request again.

A classic example is

DNS (Domain Name System). To look up the IP address for `www.google.com`, your computer sends a small UDP packet with the name to a DNS server. The server then sends back a UDP packet with the corresponding IP address. It's a quick, two-message exchange without the overhead of setting up a formal connection.

RPC (Remote Procedure Call). The idea is to make a request to a computer across a network look and feel just like a regular function call within a program. This hides the complexity of networking from the programmer.

The process works like this, using helper code called "stubs":

1. The client application makes a normal function call to a local procedure called the client stub.
2. The client stub packs the function's parameters into a message (a process called marshaling) and sends it over the network using UDP.
3. The message travels across the network to the server machine.
4. The server stub on the server machine receives the packet.
5. The server stub unpacks the parameters and calls the actual server procedure, making it seem like the call came from a local source.

The reply follows the same path back to the client.

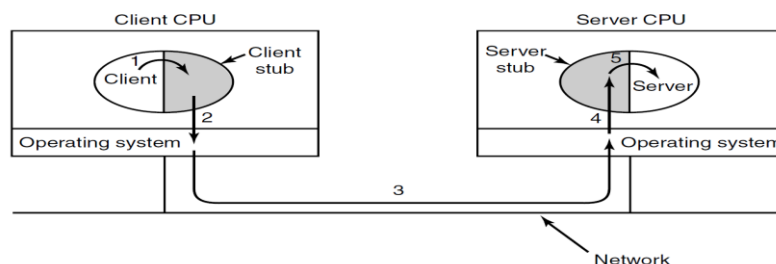


Figure 4.10 Steps in making a remote procedure call. The stubs are shaded.

It shows the Client calling the Client Stub (1), the stub sending a message via the OS (2), the message crossing the network (3), the Server Stub receiving the message from its OS (4), and the Server Stub calling the Server procedure (5).

However, RPC has some challenges:

Pointers: You cannot pass memory pointers because the client and server have different memory spaces.

Parameter Types: In flexible languages like C, it can be impossible for the stub to know the size and type of all parameters to be sent.

Global Variables: If a procedure depends on global variables, it will fail when run on a remote machine where those variables don't exist.

Despite these issues, RPC is widely used. UDP is a good choice for simple RPCs, but the application must handle timeouts and retransmissions in case of lost packets.

RTP (The Real-time Transport Protocol)

RTP's main function is to carry real-time audio and video data. Since it uses UDP, it doesn't get any special guarantees of delivery; packets can be lost, delayed, or corrupted.

RTP adds a special header with features to help with real-time streams:

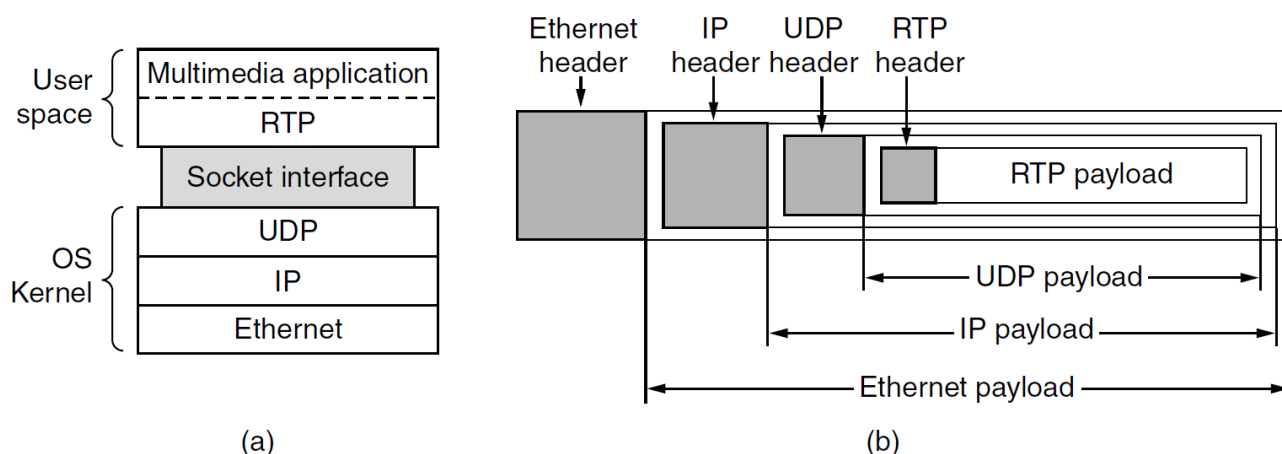


Figure 4.11 (a) The position of RTP in the protocol stack. (b) Packet nesting.

It shows fields for Version, P, X, CC, M, Payload type, Sequence number, Timestamp, Synchronization source identifier, and Contributing source identifier.

Sequence Number: Each RTP packet is numbered sequentially. This allows the receiver to detect if any packets have been lost. Retransmitting a lost packet is often not practical for live media, as it would arrive too late.

Timestamp: The source adds a timestamp to each packet, indicating when the sound or video frame was originally sampled. This allows the receiver to play back the media smoothly, even if packets arrive at irregular intervals. It also helps synchronize separate audio and video streams.

Payload Type: This field indicates the media encoding format being used (e.g., MP3 audio). This allows the format to be changed mid-stream if network conditions change.

Synchronization Source Identifier: This tells the receiver which stream a packet belongs to (e.g., to distinguish the audio stream from the video stream).

RTP is almost always used with a companion protocol,

RTCP (Real-time Transport Control Protocol). RTCP does not carry any media data. Instead, it handles:

- **Feedback:** It sends reports back to the source about network conditions like delay, packet loss, and jitter. This allows the source to adapt, for example by reducing video quality if the network becomes congested.
- **Synchronization:** It helps keep different streams (like audio and video) synchronized, even if they use different internal clocks.
- **Source Naming:** It can carry user-identifiable names (e.g., "John Smith") so an application can display who is currently speaking.

Playout with Buffering and Jitter Control

Packets traveling across the internet experience varying amounts of delay. This variation is known as jitter. If a media player were to play packets the instant they arrived, the result would be choppy audio and jerky video.

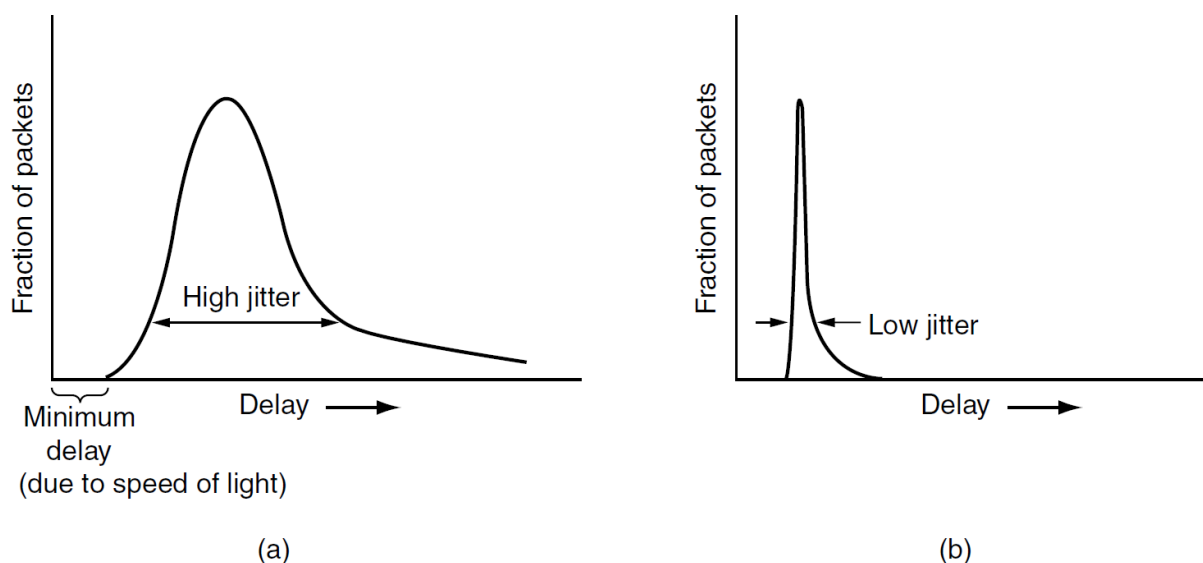


Figure 4.12 (a) High jitter. (b) Low jitter

High and low jitter packet delay distributions, similar to Figure in the source document. Graph (a) shows a wide, spread-out curve representing high jitter. Graph (b) shows a tall, narrow spike representing low jitter.

The solution is to use a playout buffer at the receiver.

- As packets arrive, they are placed in a buffer instead of being played immediately.
- The application waits for a short period to build up a small reserve of packets in the buffer.
- Playback begins by pulling packets from the buffer at a perfectly smooth and steady rate, using the RTP timestamps to determine the correct timing.

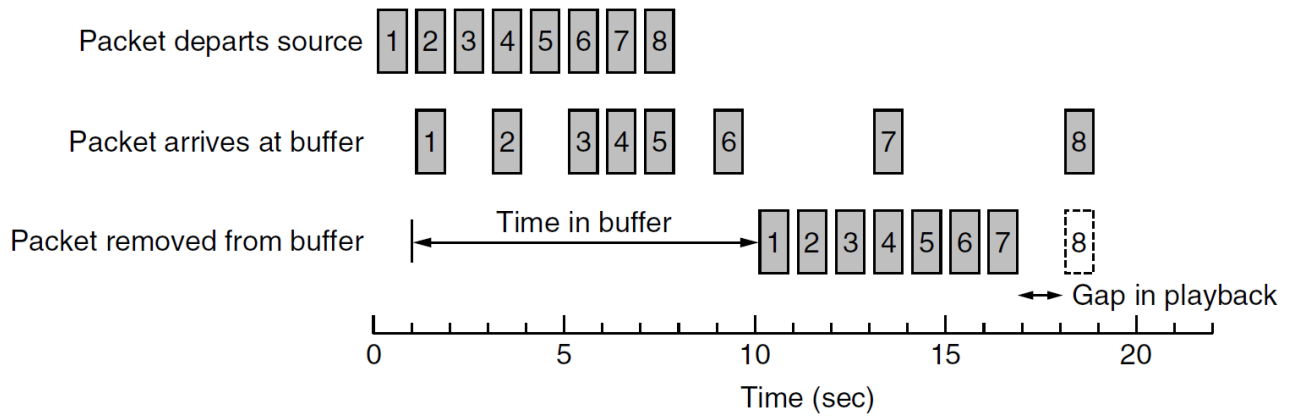


Figure 4.13 Smoothing the output stream by buffering packets.

A timeline showing how a playout buffer smooths an output stream, similar to Figure in the source document. It shows packets departing the source at regular intervals, arriving at the receiver's buffer with jitter, and then being removed from the buffer for playback at regular intervals after an initial delay.

The size of this buffer is a trade-off.

Large Buffer: A buffer of several seconds can handle significant jitter and prevent gaps in playback, which is good for streaming a movie.

Small Buffer: Live applications like video conferencing require a very short buffer to keep the delay low and the conversation responsive. This means they are more vulnerable to packets arriving too late and having to be skipped.

Advantages of UDP

- **Speed:** UDP is faster than TCP because it does not have the overhead of establishing a connection and ensuring reliable data delivery.
- **Lower latency:** Since there is no connection establishment, there is lower latency and faster response time.
- **Simplicity:** UDP has a simpler protocol design than TCP, making it easier to implement and manage.
- **Broadcast support:** UDP supports broadcasting to multiple recipients, making it useful for applications such as video streaming and online gaming.
- **Smaller packet size:** UDP uses smaller packet sizes than TCP, which can reduce network congestion and improve overall network performance.
- **User Datagram Protocol (UDP)** is more efficient in terms of both latency and bandwidth.

Disadvantages of UDP

- **No reliability:** UDP does not guarantee delivery of packets or order of delivery, which can lead to missing or duplicate data.
- **No congestion control:** UDP does not have congestion control, which means that it can send packets at a rate that can cause network congestion.
- **No flow control:** UDP does not have flow control, which means that it can overwhelm the receiver with packets that it cannot handle.

- Vulnerable to attacks: UDP is vulnerable to denial-of-service attacks, where an attacker can flood a network with UDP packets, overwhelming the network and causing it to crash.
- Limited use cases: UDP is not suitable for applications that require reliable data delivery, such as email or file transfers, and is better suited for applications that can tolerate some data loss, such as video streaming or online gaming.

4.8 Transmission Control Protocol (TCP)

Transmission Control Protocol (TCP) is a type of protocol that creates a connection to allow communication between devices, enabling them to send and receive messages over a network. It is one of the main protocols in the TCP/IP suite. In the OSI model, TCP operates at the transport layer which is located between the Application Layer and the Network Layer. This position enables TCP to provide dependable delivery of data. The Internet Protocol (IP) outlines how data packets are transmitted between computers, works alongside TCP to ensure efficient communication across networks.

FLOW CONTROL

Flow control is a method that prevents the sender from sending more data than the receiver can handle. Its main purpose is to align the speed at which data is sent with the receiver's ability to process it, avoiding situations where the receiver's buffer becomes full. TCP uses the Sliding Window Protocol to manage flow control. In this method, the receiver specifies a window size in each data packet, indicating how many bytes it is ready to receive. The sender then ensures that the amount of data being sent but not yet acknowledged does not exceed this window size. This approach helps maintain steady data transfer, makes efficient use of network resources and stops data from being lost because the receiver's buffer is already full.

2. CONGESTION CONTROL

Congestion control refers to the techniques TCP uses to avoid overloading the network with too much traffic, which can lead to packet loss, delays or even network collapse. Its usage is critical because congestion occurs when the combined demand from all hosts exceeds network capacity. TCP implements congestion control through algorithms such as Slow Start, Congestion Avoidance, Fast Retransmit and Fast Recovery.

3. ERROR DETECTION

Error detection in TCP ensures that the data received is the same as the data sent, maintaining reliable communication over unreliable networks. Its usage is essential because data can get corrupted during transmission due to noise, hardware faults, or network errors. TCP implements error detection using a 16-bit checksum in the header that covers both header and data. The sender calculates the checksum before transmission and the receiver recalculates it upon receiving. If the values don't match, the segment is discarded and retransmitted. This mechanism guarantees data integrity and reliability for higher-layer applications.

4.8.2 The TCP Service Model

TCP service is obtained by both the sender and the receiver creating end points, called sockets. Each socket has a socket number (address) consisting of the IP address of the host and a 16-bit number local to that host, called a port. A port is the TCP name for a TSAP. For TCP service to be obtained, a connection must be explicitly established between a socket on one machine and a socket on another machine.

A socket may be used for multiple connections at the same time. In other words, two or more connections may terminate at the same socket. Connections are identified by the socket identifiers at both ends, that is, (socket1, socket2). No virtual circuit numbers or other identifiers are used. Port numbers below 1024 are reserved for standard services that can usually only be started by privileged users (e.g., root in UNIX systems). They are called well-known ports. For example, any process wishing to remotely retrieve mail from a host can connect to the destination host's port 143 to contact its IMAP daemon. The list of well-known ports is given at www.iana.org. Over 700 have been assigned.

A few of the better-known ones are listed

Port	Protocol	Use
80	HTTP	World Wide Web
443	HTTPS	Secure Web (HTTP over SSL/TLS)
22	SSH	Remote login, replacement for Telnet
25	SMTP	Email
110	POP-3	Remote email access
143	IMAP	Remote email access
20, 21	FTP	File transfer
543	RTSP	Media player control
631	IPP	Printer sharing

Other ports from 1024 through 49151 can be registered with IANA for use by unprivileged users, but applications can and do choose their own ports. For example, the BitTorrent peer-to-peer file-sharing application (unofficially) uses ports 6881–6887, but may run on other ports as well.

It would certainly be possible to have the FTP daemon attach itself to port 21 at boot time, the SSH daemon attach itself to port 22 at boot time, and so on. However, doing so would clutter up memory with daemons that were idle most of the time. Instead, what is commonly done is to have a single daemon, called `inetd` (Internet daemon) in UNIX, attach itself to multiple ports and wait for the first incoming connection. When that occurs, `inetd` forks off a new process and executes the appropriate daemon in it, letting that daemon handle the request. In this way, the daemons other than `inetd` are only active when there is work for them to do. `Inetd` learns which ports it is to use from a configuration file. Consequently, the system administrator can set up the system to have permanent daemons on the busiest ports (e.g., port 80) and `inetd` on the rest.

All TCP connections are full duplex and point-to-point. Full duplex means that traffic can go in both directions at the same time. Point-to-point means that each connection has exactly two end points. TCP does not support multicasting or broadcasting.

A TCP connection is a byte stream, not a message stream. Message boundaries are not preserved end to end. For example, if the sending process does four 512-byte writes to a TCP stream, these data may be delivered to the receiving process as four 512-byte chunks, two 1024-byte chunks, one 2048-byte chunk or some other way. There is no way for the receiver to detect the unit(s) in which the data were written, no matter how hard it tries.

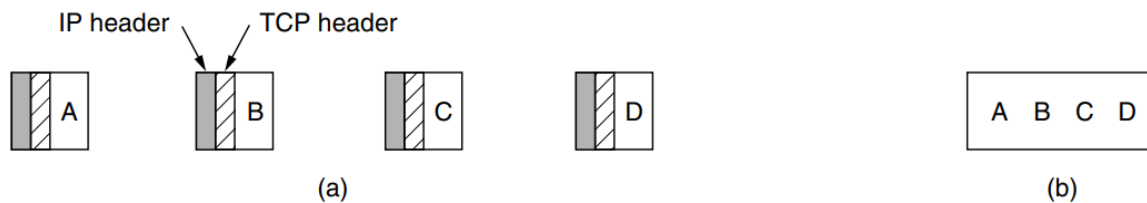


Figure 4.13 (a) Four 512-byte segments sent as separate IP datagrams. (b) The 2048 bytes of data delivered to the application in a single READ call.

4.8.3 TCP SEGMENT STRUCTURE

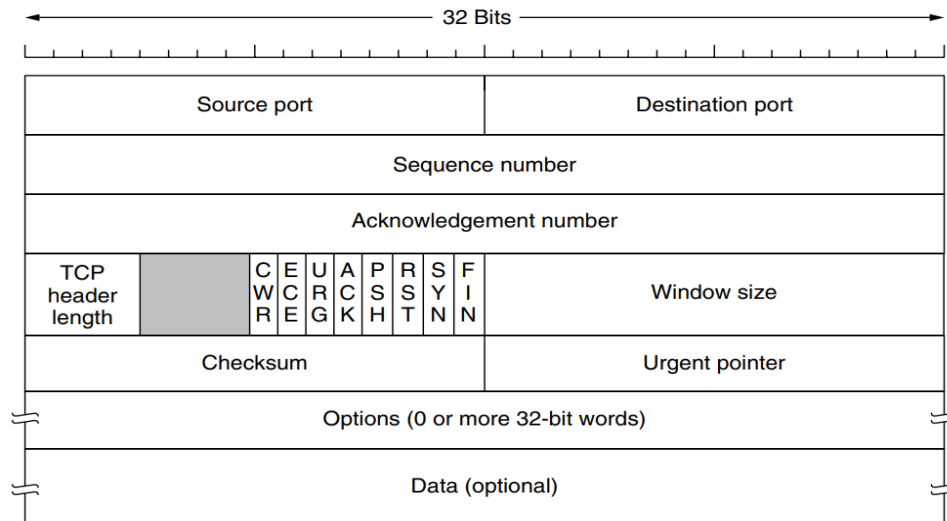


Figure 4.15 The TCP header

- **Source Port (16 bits):** Stores the port number of the sending process, allowing the receiver to identify which application generated the data.
- **Destination Port (16 bits):** Contains the port number of the receiving process so the data can be delivered to the correct application on the destination system.
- **Sequence Number (32 bits):** Assigns a unique number to each byte in the stream, ensuring that segments are reassembled in the correct order at the receiving end.
- **Acknowledgment Number (32 bits):** Specifies the next expected sequence number, thereby confirming successful receipt of previous segments.
- **Header Length (4 bits):** This field indicates where the actual TCP data starts and tells how long the TCP header is.
- **Control Flags (9 bits):** A set of binary indicators that manage TCP connections. Key flags include:
 - **SYN (Synchronize):** Used to initiate a connection between sender and receiver.
 - **ACK (Acknowledgment):** Confirms that data has been successfully received.
 - **FIN (Finish):** Signals that the sender has completed data transmission and wants to close the connection.
 - **RST (Reset):** Terminates or resets a connection, usually when errors occur.
 - **PSH (Push):** Requests immediate delivery of data to the application without waiting in the buffer.
 - **URG (Urgent):** Highlights that the urgent pointer field contains valid data requiring priority handling.

- **Window Size (16 bits):** Specifies the size of the receiver's available buffer, controlling the flow of incoming data.
- **Checksum (16 bits):** Provides error-checking for the TCP header and data to ensure integrity during transmission.
- **Urgent Pointer (16 bits):** Identifies the location of urgent data within the segment.
- **Options (Variable length):** Offers additional parameters for TCP communication, such as maximum segment size or timestamps.
- **Data Payload:** Holds the actual user data that is being carried over the TCP connection.

4.9 Stream Control Transmission Protocol (SCTP)

Stream Control Transmission Protocol (SCTP) is a transport layer protocol designed for modern internet applications. It uniquely blends the characteristics of both TCP and UDP. Like TCP, SCTP is a reliable, connection-oriented protocol that provides congestion and flow control. However, like UDP, it is message-oriented, meaning it preserves the boundaries of each message sent by an application. SCTP is also capable of detecting lost, duplicated, or out-of-order data to ensure reliability.

Core Services Provided by SCTP

SCTP offers several key services that enhance data communication:

1. **Process-to-Process Communication:** SCTP enables communication between specific processes running on the source and destination hosts by using port numbers, similar to TCP.
2. **Full-Duplex Communication:** Data can flow in both directions simultaneously within a single SCTP connection.
3. **Reliable Data Transfer:** To ensure data arrives successfully, SCTP uses an acknowledgement mechanism.
4. **Multi-Stream Facility:** A standout feature of SCTP is its ability to handle multiple independent streams of data within a single connection, which is called an "association". This prevents head-of-line blocking; if one stream is blocked waiting for a lost piece of data, other streams can continue to deliver their data without delay.

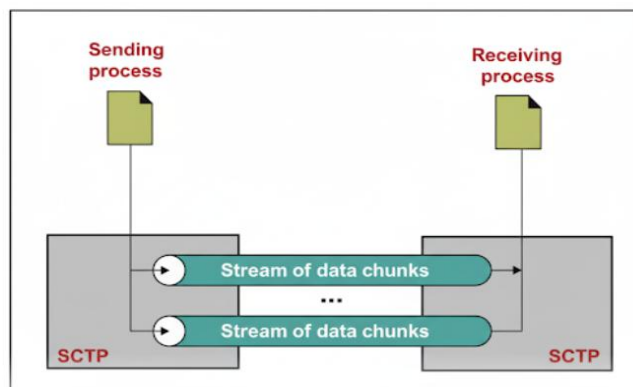


Figure 4.16 Multi-Stream Facility

Connection- Oriented Service: The SCTP is a connection-oriented protocol, just like TCP with the only difference that, it is called association in SCTP. If User1 wants to send and receive message from user2, the steps are:

- Step1: The two SCTPs establish the connection with each other.
- Step2: Once the connection is established, the data gets exchanged in both the directions.
- Step3: Finally, the association is terminated.

4.9.1 Key Features of SCTP

SCTP incorporates several features that contribute to its robustness and efficiency:

Multihoming: This feature allows a single SCTP association to be established across multiple IP addresses for both the sender and the receiver. One IP address is designated as the primary path for communication, while the others serve as alternative paths for backup and fault tolerance.

Transmission Sequence Number (TSN): The fundamental unit of data in SCTP is called a data chunk. To manage data transfer, every data chunk is assigned a unique Transmission Sequence Number (TSN).

Stream Identifier (SI): Within an association, data chunks are organized into different streams. Each chunk's header contains a 16-bit Stream Identifier (SI) to identify which stream it belongs to, ensuring it is processed correctly upon arrival.

Packet and Chunk Structure: In SCTP, information is organized into "chunks." Application data is carried in data chunks, while protocol-level information is carried in control chunks. These chunks are then bundled together into a single SCTP packet for transmission.

The general-header format of the SCTP

Source port address 16 bits	Destination port address 16 bits
Verification tag 32 bits	
Checksum 32 bits	

Figure 4.17 SCTP Header

Source Port Address (16 bits): Identifies the port number of the sending process.

Destination Port Address (16 bits): Identifies the port number of the receiving process.

Verification Tag (32 bits): A value used to validate that the packet is from the correct sender and belongs to the current association.

Checksum (32 bits): Used for error detection to ensure the integrity of the packet's contents.

4.9.2 Establishing an Association: The Four-Way Handshake

To begin communication, SCTP establishes a connection between a client and server using a four-way handshake protocol. This process is more secure than TCP's three-way handshake and works as follows:

INIT: The client initiates the process by sending an INIT data chunk to the server.

INIT-ACK: The server responds with an INIT-ACK chunk. This chunk contains a "cookie," which is a piece of data that securely defines the state of the proposed association.

COOKIE-ECHO: The client sends a COOKIE-ECHO chunk, which simply contains the cookie it received from the server. SCTP allows this packet to carry user data, enabling data exchange to begin earlier than in TCP.

COOKIE-ACK: The server receives the echoed cookie, verifies it, and completes the association by sending a COOKIE-ACK chunk. This final packet can also carry user data.

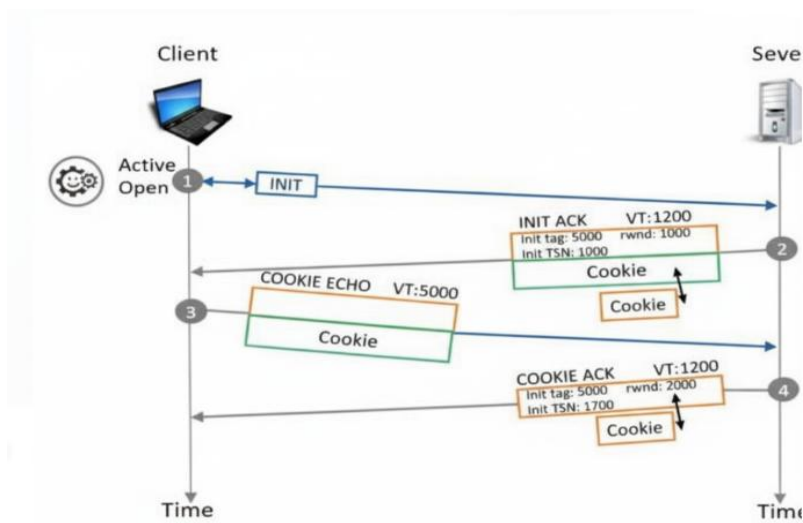


Figure 4.18 The Four-Way Handshake

A sequence diagram showing the four-way handshake process for establishing an SCTP association between a client and server over time. The client sends INIT, the server replies with INIT-ACK (containing a Cookie), the client sends COOKIE-ECHO, and the server finishes with COOKIE-ACK, as illustrated in the source document's diagram.

Data Transfer and Termination

Once an association is established, data can be transferred bidirectionally. When an application passes a message to SCTP, it is converted into one or more data chunks, each with a header and a TSN. In some cases, to improve efficiency, the receiver may only send one acknowledgement for the last data chunk received in a sequence.

When the communication is finished, either the client or the server can initiate the termination of the association. Once one side terminates the connection, the other end automatically stops sending new data.

4.10 Quality of Service (QoS)

In a digital world where billions of devices exchange information every second, the smooth and predictable flow of data is as essential as electricity or water. This requirement is addressed through a concept known as Quality of Service, often abbreviated as QoS. In simple terms, QoS is like a traffic management system on a busy highway. Imagine ambulances, buses, and cars trying to move on the same road. Without rules, congestion and delays are unavoidable. Similarly, in computer networks, different types of data—such as video calls, text messages, and software downloads—must be carefully organized so that critical information reaches on time without unnecessary delay.

QoS does not magically make the internet faster, but it ensures that important services, such as real-time video conferencing, online classes, or cloud-based business applications, get preference over less urgent traffic. This balance allows users to experience reliability and continuity even when the network is under stress. Without QoS, even a minor congestion could disrupt a doctor consulting a patient online or cause a stock market trader to miss a crucial update.

A straightforward way to ensure good service quality is by designing a network with sufficient capacity to handle any level of traffic. This approach is commonly referred to as overprovisioning.

To maintain quality of service, four key aspects need attention:

- Identifying the requirements of applications from the network.
- Controlling and managing the flow of traffic entering the network.
- Allocating resources at routers to ensure reliable performance.
- Determining if the network can handle additional traffic without issues.

4.10.1 Architectures for Achieving QoS

The Internet's default "best-effort" model, which treats all traffic equally, is insufficient for the needs of inelastic applications. To address this, two primary service models have been developed to differentiate and prioritize traffic:

1. Integrated Services (IntServ) IntServ is a fine-grained, stateful architecture that aims to provide end-to-end service guarantees for individual data flows.

Operation: This model relies on a signaling protocol, most notably the Resource Reservation Protocol (RSVP), to reserve network resources before a data flow begins. An application explicitly requests a specific level of service from every router along its path.

Admission Control: Before a new flow is permitted, each device performs an admission control check to ensure it has the capacity to meet the request without negatively impacting pre-existing reservations. If the request is denied at any point along the path, the flow is not admitted, preventing oversubscription.

Limitations: The primary drawback of IntServ is its poor scalability. The need for every router to maintain and manage state information for every individual flow (which could number in the tens of thousands on a high-speed backbone) makes this approach unviable for large-scale public networks.

2. Differentiated Services (DiffServ) DiffServ is a more scalable, class-based architecture that provides QoS guarantees for aggregated groups of traffic rather than individual flows.

Operation: In a DiffServ domain, traffic is marked at the network's edge with a Differentiated Services Code Point (DSCP) in the IP packet header. This marking classifies the packet into one of a few traffic classes, such as Expedited Forwarding (EF) for low-delay, premium traffic or Assured Forwarding (AF) for traffic that receives preferential treatment but no strict guarantees.

Congestion Management: Core routers within the network are configured to implement different "per-hop behaviors" (PHBs) for each traffic class, such as prioritizing EF traffic in their queues. If congestion occurs, packets from lower-priority classes are dropped first to protect the service quality of higher-priority classes.

Advantages: The main benefit of DiffServ is its scalability. It shifts the complexity to the network's edge, requiring core routers to manage only a small number of queues for different classes, rather than a massive number of per-flow states. This makes it a practical and widely-adopted solution for modern networks.

4.10.2 Data Flow Characteristics

When data travels through a network, it behaves much like water flowing through a pipeline. Network engineers study several characteristics of this flow to determine whether the service is functioning smoothly. The major characteristics are explained below in everyday language.

- **Effective Data Rate:**

Often referred to as throughput in technical language, this is the real speed at which information reaches its destination. A connection may advertise high bandwidth, but what truly matters is the portion of data that successfully arrives without loss. It is similar to a pipe that looks wide, but due to leaks, the actual water reaching the tank is much less.

- **Time Lag:**

Known traditionally as latency, this refers to the waiting period between sending and receiving information. When you click a link and wait for a website to appear, that waiting time is the lag. In live applications such as gaming or virtual classrooms, even milliseconds of lag can affect performance.

- **Timing Irregularity:**

This is the inconsistency in the arrival of data packets, commonly termed jitter. It is like a bus service where buses are expected every 10 minutes, but sometimes they arrive after 2 minutes and at other times after 20 minutes. Such unpredictability can make a video call choppy.

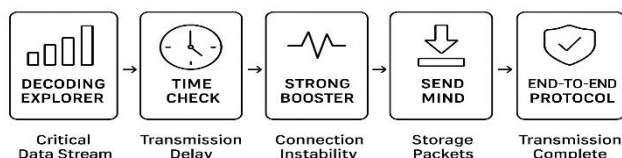
- **Data Drop:**

During transmission, certain packets may never reach their target, which is called packet loss. It can be compared to a postal service where some letters are misplaced before delivery. High data drop makes it impossible to stream videos smoothly or hold clear conversations online.

- **Delivery Assurance:**

Beyond speed, reliability matters. It is not enough for data to reach its destination—it must arrive complete and in the right order. Delivery assurance ensures that digital files, financial transactions, and business communications are not corrupted during transfer.

DATA FLOW IN A NETWORK



FLOW CONTROL TO ENHANCE QOS

Managing how fast information is sent and received is crucial for maintaining quality. This technique is called flow control, or in simpler words, transmission pacing. If data is sent too quickly, the receiver may be overloaded; if sent too slowly, the network's capacity is wasted.

Different strategies exist to manage pacing:

- **Stop-and-Wait:**

Data is sent one piece at a time, and the sender waits until confirmation is received before sending more. Although safe, it is relatively slow, much like a teacher dictating one word at a time to ensure every student writes correctly.

- **Sliding Window:**

Instead of waiting after every piece, multiple pieces of data are sent at once. If something goes wrong, only the faulty portion is resent. This method balances safety and speed, similar to a teacher dictating a paragraph and asking only students who missed a line to correct it.

- **Adaptive Pacing:**

Modern networks use intelligent methods that adjust their sending speed based on current network load. When the path is clear, more data flows; when congestion builds, the system automatically slows down. This is like a driver slowing on a crowded road and speeding up again on an empty highway.

By applying these flow control mechanisms, networks avoid congestion, minimize losses, and ensure smooth service delivery. Together with QoS policies, flow control ensures that essential services are uninterrupted.

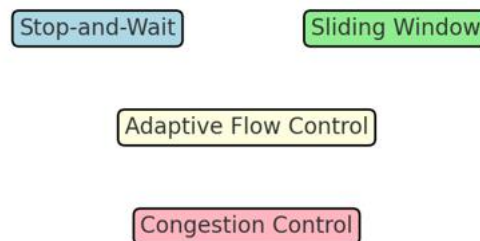


Figure 4.20 Flow Control Mechanism Flowchart

Quality of Service and flow control are two sides of the same coin. While QoS defines priorities and sets rules for traffic handling, flow control ensures that these priorities are respected in practice by regulating the rate of data flow. With the rise of 5G networks, smart cities, and real-time applications such as remote surgery or autonomous vehicles, their importance has grown significantly. A well-designed system that combines both ensures that the digital highway remains efficient, reliable, and ready for future demands.

4.10.3 Foundational Properties of Data Traffic

To provide effective QoS, network engineers must manage several key characteristics of data transmission. These attributes dictate an application's tolerance for network fluctuations and form the basis for resource allocation:

- **Throughput (Effective Bandwidth):** While a link may have a high theoretical bandwidth, the actual rate at which useful data is successfully delivered is known as throughput. For QoS, the objective is to secure a baseline data rate that is consistently available for mission-critical applications.
- **Packet Delay:** The time a packet takes to traverse the network from its source to its destination. Delays are influenced by the distance of the path, processing time at each network device, and the time a packet spends waiting in queues.
- **Delay Variation (Jitter):** The inconsistency in packet arrival times. When packets in a single stream arrive with varying delays, the receiver must use buffering to re-sequence them. Excessive jitter can overflow these buffers or cause playback irregularities, making it particularly damaging to real-time audio and video.
- **Information Loss:** The percentage of data packets that are corrupted or fail to reach their destination. In a congested network, routers and switches may discard packets to alleviate stress, leading to information loss. Some applications can recover from this via retransmission, but real-time streams often cannot and must discard late packets, effectively losing them.

Applications can be broadly classified based on their sensitivity to these parameters. **Inelastic applications** (e.g., live video conferencing, voice over IP) have strict requirements for low delay and jitter. They are typically intolerant of late arrivals but can sometimes handle a small amount of data loss. By contrast, **elastic applications** (e.g., file transfers, web browsing) can adapt to a wide range of delays and throughput. They are less sensitive to network conditions because they can use retransmission to guarantee data integrity, a luxury that inelastic traffic cannot afford due to time constraints.

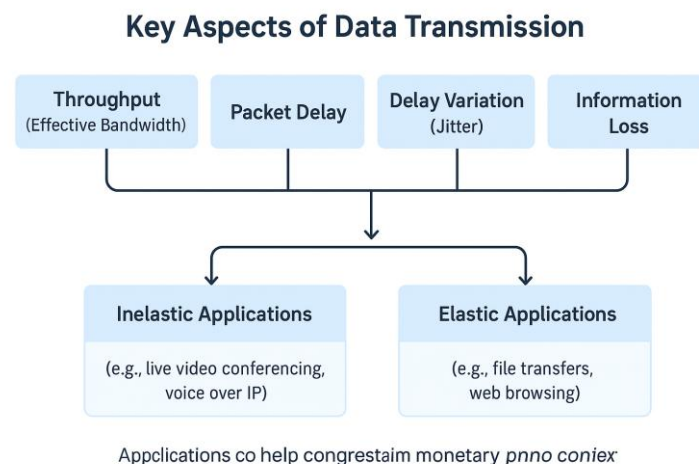


Figure 4.21 Data Transmission

UNIT 5

APPLICATION LAYER

World Wide Web and Hyper Text Protocol-File Transfer Protocol-NFS-Electronic Mail-Telnet-Secure Shell-Domain Name System-5G Network Utilization-CoAP-MQTT

The Application Layer is the topmost layer in the OSI (Open Systems Interconnection) model and the TCP/IP protocol suite. It provides an interface between the user (or application software) and the underlying network. Unlike lower layers, which focus on data transmission and routing, the application layer deals with network services that directly support user applications. It enables end-user processes to communicate over a network by providing services such as email, file transfer, web browsing, and remote login. This layer defines protocols that allow software applications to interact with each other and exchange data seamlessly, regardless of differences in hardware or operating systems.

Common Protocols at the Application Layer

HTTP/HTTPS – Web browsing and secure communication.

FTP – File transfer between systems.

SMTP/IMAP/POP3 – Email services.

DNS – Domain name resolution (converting names to IP addresses).

Telnet/SSH – Remote login and secure communication.

The Application Layer acts as a bridge between user applications and the network, ensuring users can interact, share, and access resources effectively.

5.1 World Wide Web (WWW) AND HYPER TEXT PROTOCOL

The World Wide Web (WWW) is a distributed information system that operates over the Internet. It provides a universal platform where documents, multimedia, and interactive content can be stored, accessed, and interconnected through hyperlinks. Unlike the Internet itself, which is a global network of interconnected computers and devices, the WWW is a service that utilizes the Internet as its foundation for data transfer. It is one of the most significant developments in the field of networking, having transformed the way information is created, shared, and consumed across the globe.

Nature of the WWW

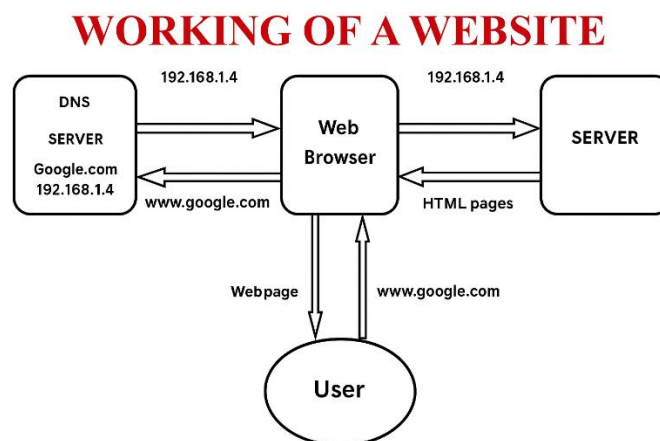


Figure 5.1 Working of a website

The World Wide Web is not a separate physical network. Instead, it is an application-level service that runs on top of the Internet infrastructure. Its nature lies in providing an easy-to-use and standardized interface for accessing diverse types of information, including text, images, audio, video, and interactive applications.

In essence, the WWW acts as a virtual library where each resource is stored in distributed servers across the world but can be accessed by anyone through a web browser. The openness of its design, combined with the use of common protocols like HTTP, makes it universally accessible regardless of the device, operating system, or geographical location of the user.

5.1.2. Components of the WWW

The WWW functions with the help of several key components. These can be divided into structural components, which define the basic building blocks, and semantic components, which define the meaning and interoperability of web resources.

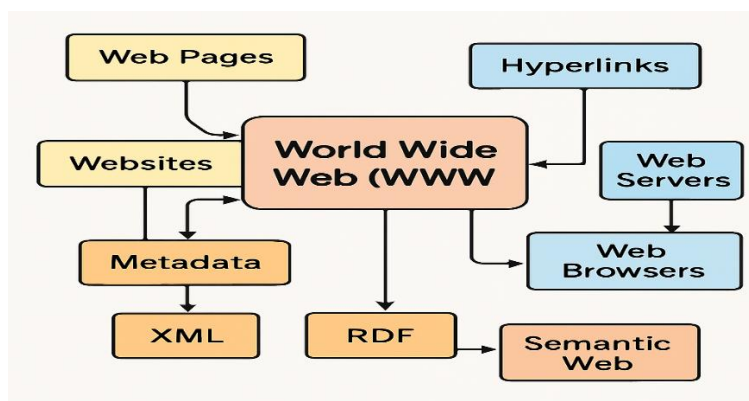


Figure 5.2 components of WWW

Structural Components

1. Web Pages:

A web page is the fundamental unit of the WWW. It is a document written in HTML (Hypertext Markup Language), which can contain formatted text, images, tables, audio, video, and links to other pages. Web pages are retrieved from servers and displayed by browsers.

2. Websites:

A website is a collection of interrelated web pages maintained under a single domain name. For example, www.example.com may consist of multiple pages such as the homepage, about page, contact page, and so on. Websites serve as organizational structures that group related resources.

3. Hyperlinks:

Hyperlinks are embedded references in a web page that allow users to navigate from one page to another, either within the same website or across different websites. This linking capability is what differentiates the WWW from earlier isolated document systems.

4. Web Servers:

A web server is a computer that stores and delivers web pages to clients upon request. It listens for incoming requests on specific ports (typically port 80 for HTTP and port 443 for HTTPS) and responds with the appropriate resource.

5. Web Browsers:

A web browser is a client application, such as Chrome, Firefox, or Edge, that enables users to access and interact with web resources. Browsers interpret HTML and other content formats, rendering them in a graphical user interface for the user to view and navigate.

Semantic Components

1. Metadata:

Metadata provides descriptive information about web resources, such as the author, creation date, language, or subject. It helps search engines and indexing services in organizing and retrieving information efficiently.

2. XML and RFD Standards:

Extensible Markup Language (XML) and Resource Description Framework (RDF) are widely used for structuring and describing data on the web. These standards enable interoperability between different systems and applications.

3.Semantic Web:

The Semantic Web is an extension of the current WWW where data is structured in a way that is not only human-readable but also machine-readable.

This allows intelligent agents and applications to process information meaningfully, paving the way for advanced applications such as AI-powered search engines and automated decision-making systems.

5.1.4. Architecture of the WWW

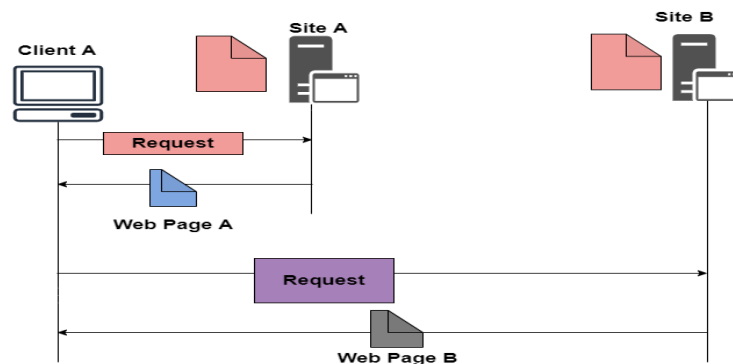


Figure 5.3 Architecture of WWW

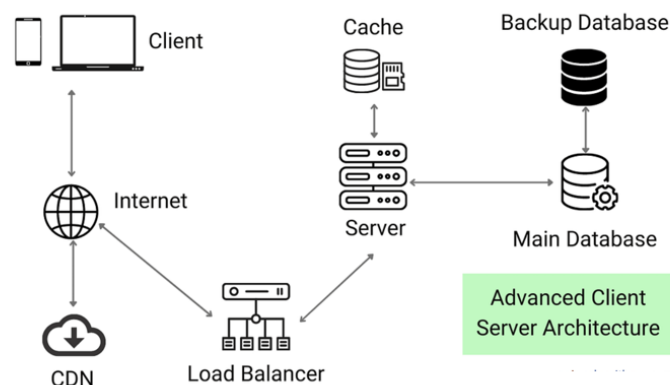


Figure 5.4 Advanced Client Server Architecture

The WWW is based on a distributed client-server architecture, where clients and servers interact over the Internet using well-defined protocols.

1. Client (Web Browser):

The client is responsible for initiating requests. It takes input from the user in the form of URLs and sends HTTP requests to the appropriate server. Once the response is received, the browser interprets the data (HTML, CSS, scripts) and presents it to the user.

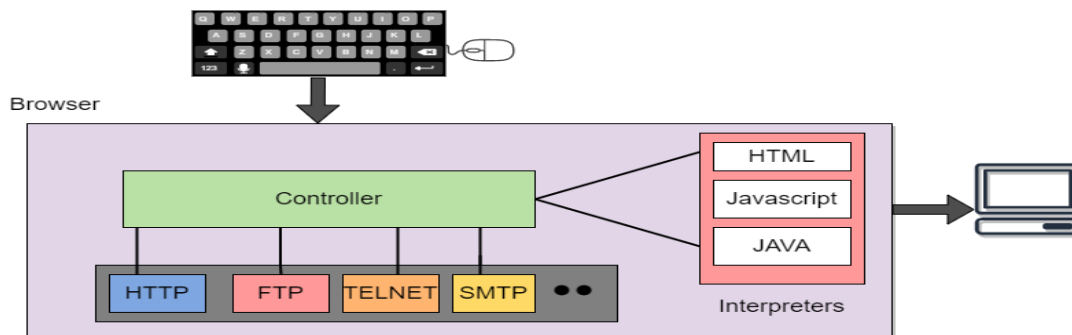


Figure 5.5 Client

2. Web Server:

A server hosts web pages and resources. Upon receiving a request from a client, it retrieves the requested document and transmits it back over the network. Servers are capable of handling multiple simultaneous requests using mechanisms like multithreading and multiprocessing.

3. Communication Protocols:

The interaction between client and server occurs using the Hypertext Transfer Protocol (HTTP) or its secure version (HTTPS). At a lower level, these rely on TCP/IP to provide reliable delivery. This architecture ensures that resources stored in different parts of the world can be accessed transparently by any user, creating the illusion of a single, integrated information system.

Architectural Overview:

From the users' point of view, the Web consists of a vast, worldwide collection of content in the form of Web pages, often just called pages for short. Each page may contain links to other pages anywhere in the world. Users can follow a link by clicking on it, which then takes them to the page pointed to. This process can be repeated indefinitely. The idea of having one page point to another, now called hypertext, was invented by a visionary M.I.T. professor of electrical engineering, Vannevar Bush, in 1945 (Bush, 1945). This was long before the Internet was invented. In fact, it was before commercial computers existed although several universities had produced crude prototypes that filled large rooms and had less power than a modern pocket calculator.

Pages are generally viewed with a program called a browser. Firefox, Internet Explorer, and Chrome are examples of popular browsers. The browser fetches the page requested, interprets the content, and displays the page, properly formatted, on the screen. The content itself may be a mix of text, images, and formatting commands, in the manner of a traditional document, or other forms of content such as video or programs that produce a graphical interface with which users can interact.

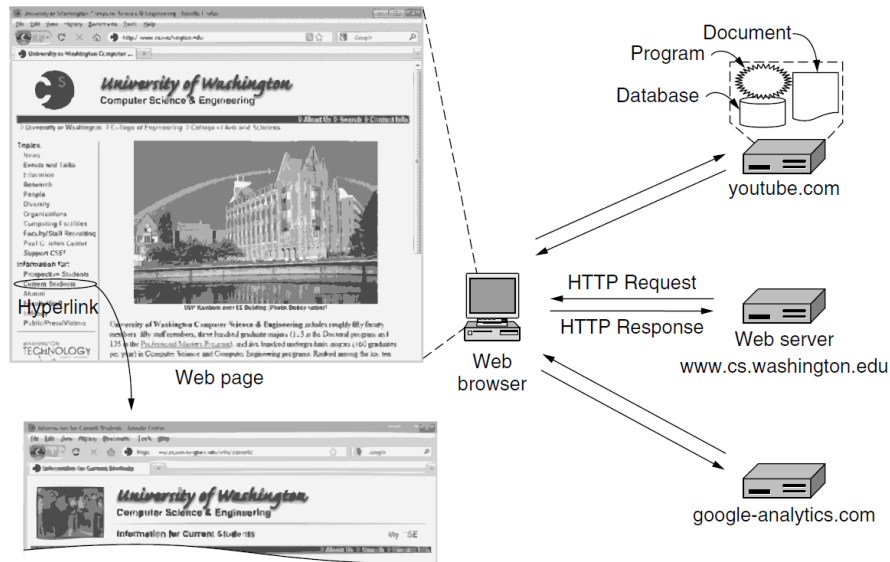


Figure 5.6 Architecture of the Web

5.1 6. HTML (Hypertext Markup Language)

HTML is the primary language for creating web pages. It defines both the structure of the content (headings, lists, tables, forms) and its presentation (images, embedded multimedia, hyperlinks). HTML is interpreted by browsers, which render it into visually interactive pages.

XML (Extensible Markup Language)

XML is used to describe structured data in a standardized format. Unlike HTML, which is focused on presentation, XML emphasizes data description and exchange. It is particularly important for applications that require machine-to-machine communication, such as web services, RSS feeds, and semantic web technologies.

5.1.7. Operational Flow of the WWW

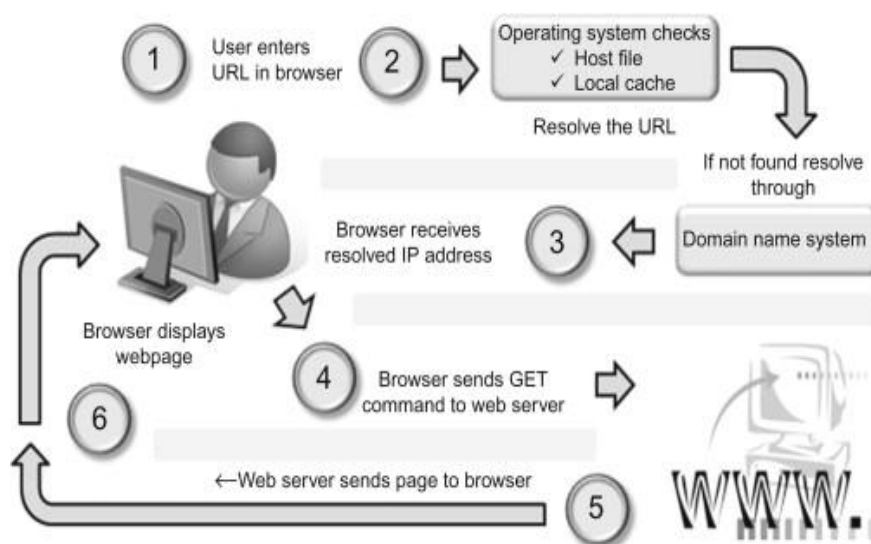


Figure 5.7 Operational flow of WWW

The working of the WWW can be summarized in the following steps:

1. A user enters a URL in the browser's address bar.
2. The browser queries the DNS (Domain Name System) to resolve the domain name into an IP address.

3. The browser establishes a TCP connection with the corresponding server.
4. An HTTP request (e.g., GET) is sent to the server.
5. The server processes the request, retrieves the resource, and sends it back as an HTTP response.
6. The browser interprets the response and renders the web page for the user to view.

This cycle forms the basis of all web interactions, whether simple static pages or complex dynamic applications.

5.1.8. Applications of the WWW

The WWW supports a wide range of applications in different sectors:

- **Education:** Online courses, e-learning platforms, digital libraries, and research databases.
- **Commerce:** Online shopping, electronic banking, and digital marketplaces.
- **Media and Entertainment:** Streaming platforms, gaming, and digital publishing.
- **Social Networking:** Platforms like Facebook, Instagram, and Twitter allow people to interact and share content globally.
- **Information Access:** Online encyclopedias, news portals, and knowledge bases provide instant access to diverse information sources.

5.1.9. Features of the WWW

The WWW possesses several defining features that make it a unique global information system:

- **Distributed System:** Information is stored on millions of servers worldwide, yet accessible from anywhere.
- **Cross-Platform Compatibility:** Content can be accessed from any device or operating system.
- **Dynamic Nature:** Content can be updated in real-time, providing freshness and relevance.
- **Interactivity:** Users can interact with applications through forms, comments, and multimedia interfaces.
- **Support for Multimedia:** Integration of text, audio, video, and graphics enhances communication.
- **Open Standards:** Built on standardized protocols (HTTP, HTML, XML) which ensure global interoperability.

5.2 Hypertext Transfer Protocol (HTTP)

The Hypertext Transfer Protocol (HTTP) is the fundamental communication standard used on the World Wide Web (WWW). It defines how clients (typically web browsers) and servers exchange requests and responses. HTTP is an application-layer protocol that operates on top of the TCP/IP stack and provides a mechanism for transmitting hypertext documents, multimedia, and structured data between distributed systems. Without HTTP, the modern web's interactive and interconnected nature would not be possible.

2. Nature of HTTP

HTTP is a **stateless, request-response protocol**. Every interaction is initiated by a client, which sends a request to a server. The server processes this request and returns a response. Since HTTP is stateless, the server does not retain information about previous requests; each exchange is independent.

To compensate for this limitation, mechanisms such as **cookies, sessions, and tokens** are layered on top of HTTP to maintain continuity in user interactions, enabling features like login persistence, shopping carts, and user-specific preferences.

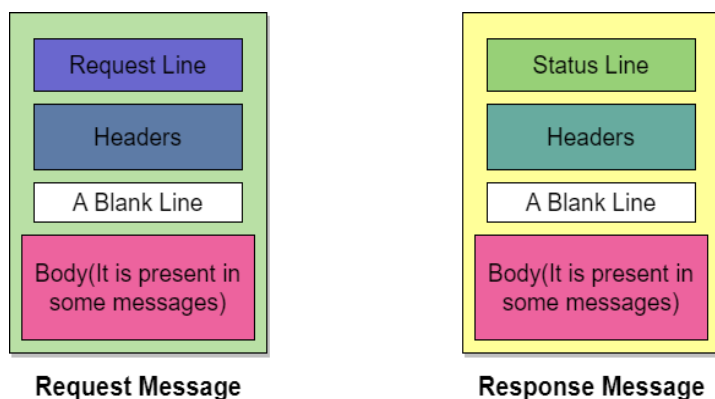
5.2.3 HTTP Methods

HTTP defines several request methods, each specifying a particular action to be performed on a resource:

- **GET:** Retrieves information from the server. It is the most common method, typically used to fetch web pages and resources.
- **POST:** Submits data to the server, often through forms. Used for actions such as user registration or search queries.
- **PUT:** Uploads or updates a resource on the server, replacing existing content with new data.
- **DELETE:** Requests that a specified resource be removed from the server.
- **HEAD:** Similar to GET but returns only the headers, useful for checking metadata without transferring the body.
- **OPTIONS, TRACE, CONNECT:** Specialized methods used for diagnostics, testing, or creating secure tunnels (e.g., HTTPS).

These methods provide a consistent vocabulary for client-server interaction and form the basis of web services and RESTful APIs.

5.2.4. HTTP Message Format



Communication in HTTP occurs through **request messages** (from client to server) and **response messages** (from server to client).

Request Message

- **Request Line:** Contains the method, the resource path, and the protocol version (e.g., GET /index.html HTTP/1.1).
- **Headers:** Provide metadata such as content type, length, accepted formats, and authentication tokens.
- **Body (Optional):** Contains data being sent to the server, typically used with POST or PUT requests.
- **Response Message**
- **Status Line:** Contains the protocol version, a status code, and a descriptive phrase (e.g., HTTP/1.1 200 OK).
- **Headers:** Provide details such as content type, caching instructions, or cookies.

- Body: Contains the requested content (HTML, images, JSON data, etc.).

This well-structured format ensures interoperability between clients and servers, regardless of their implementation.

5.2.5. Status Codes

HTTP uses three-digit **status codes** in responses to indicate the outcome of a request:

- 1xx Informational: Acknowledges receipt of the request (e.g., 100 Continue).
- 2xx Success: Indicates the request was successful (e.g., 200 OK, 201 Created).
- 3xx Redirection: Informs the client that further action is needed, such as following a new URL (e.g., 301 Moved Permanently, 302 Found).
- 4xx Client Errors: Indicates an error caused by the client (e.g., 404 Not Found, 403 Forbidden).
- 5xx Server Errors: Indicates the server failed to process a valid request (e.g., 500 Internal Server Error, 502 Bad Gateway).

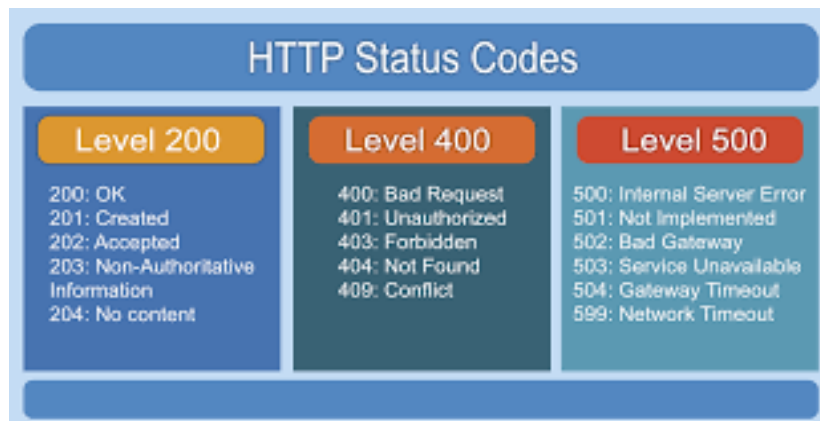


Figure 5.8 HTTP Status Code

These codes standardize communication and help developers and users understand request outcomes quickly.

5.2.6. Connection Management

HTTP communication relies on TCP connections, and connection management is crucial for efficiency:

- Non-Persistent Connections (HTTP/1.0): A separate TCP connection is established for each request-response pair. This increases overhead, especially for web pages with multiple embedded resources.
- Persistent Connections (HTTP/1.1 and later): Multiple requests and responses can be exchanged over a single TCP connection. This reduces latency and resource consumption, significantly improving performance.

Persistent connections paved the way for modern web efficiency, complemented later by HTTP/2's multiplexing capability.

5.2.7. Characteristics of HTTP

- Stateless: Each request is independent; servers do not track previous interactions.
- Human-Readable (Text-based in HTTP/1.x): Requests and responses are plain text, making debugging easy.
- Connectionless (in early versions): Connections close after each response, unless persistence is enabled.

- Extensible: New headers, methods, and features can be added without breaking compatibility.
- Secure Variant (HTTPS): Adds SSL/TLS encryption to HTTP, ensuring confidentiality, integrity, and authentication.

5.2.9. Features of HTTP

- Simplicity: Easy to implement and understand.
- Flexibility: Can carry multiple content types (HTML, JSON, XML, images, audio, video).
- Proxy Support: Can operate through caching servers, firewalls, and load balancers.
- Error Reporting: Provides status codes for diagnostics without closing the connection.
- Cross-Platform: Works across operating systems, devices, and networks.

5.3 File Transfer Protocol (FTP)

File Transfer Protocol (FTP) is a **standard network protocol** used to transfer files between a client (your computer) and a server (remote computer) over the Internet or any TCP/IP-based network.

It was designed to **upload, download, and manage files** efficiently. FTP works on the **application layer** of the TCP/IP model and uses a **client-server architecture**.

Key Points about FTP:

- Uses Port 21 for control connection (commands).
- Uses another port (Port 20 or dynamically assigned) for data transfer.
- Supports authentication using a username and password, or anonymous login.
- Can transfer large files and perform operations like renaming, deleting, or listing files on the server.

Limitation:

FTP transfers data in plain text (not secure). That's why secure alternatives like FTPS (FTP Secure) and SFTP (SSH File Transfer Protocol) are widely used today.

5.3.2 Working principle of FTP:

FTP works on a **client-server model** and uses **two separate connections** between the client and the server:

1. Control Connection (Port 21)

- Established first when the client connects to the server.
- Used for sending **commands** (like login, upload, download) and receiving **responses**.
- Remains open throughout the session.

2. Data Connection (Port 20 or Dynamic Port)

- Used for transferring the **actual files** (uploading/downloading).
- Opened separately whenever a file transfer request is made.
- Closes after the transfer is complete.

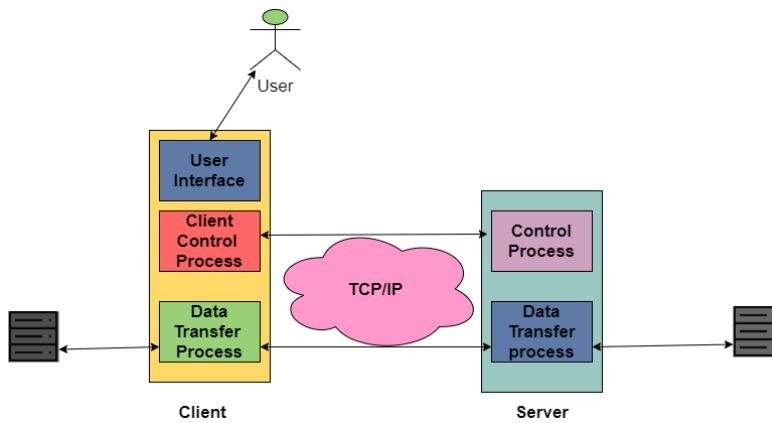


Figure 5.11 Data Connection

Step-by-Step Process:

Connection Setup: The client opens a connection to the FTP server on Port 21.

Authentication: User provides a username and password, or connects anonymously (using “anonymous” as username).

Command Communication: The client sends commands like LIST (to list files), STOR (to upload), or RETR (to download).

Data Transfer: The server establishes a data connection to send or receive the file. File is transferred in binary mode (for images, software, etc.) or ASCII mode (for text files).

Session Termination: After all transfers are complete, the client sends the QUIT command, and the server closes the connection.

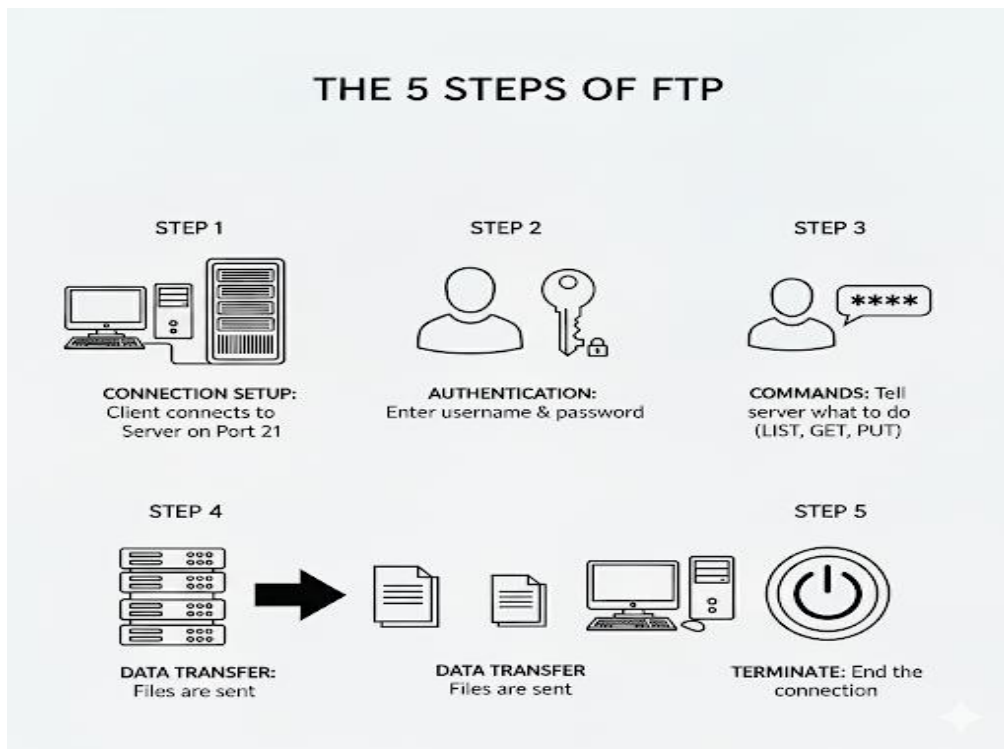


Figure 5.12 Step of FTP

Modes of Operation:

Active Mode: Client opens a port and waits for the server to connect back.

Passive Mode: Server opens a port and client connects to it (commonly used today because it works with firewalls).

5.4 NETWORK FILE SYSTEM (NFS)

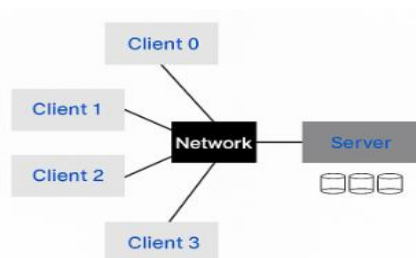
A Network File System (NFS) allows remote hosts to mount file systems over a network and interact with those file systems as though they are mounted locally. This enables system administrators to consolidate resources onto centralized servers on the network.

A Network File System (NFS) is a distributed file system protocol that allows a client to access files over a network as if they were on a local drive, enabling seamless file sharing and centralized data management across multiple machines.

Developed by Sun Microsystems, NFS uses Remote Procedure Calls (RPCs) over TCP/IP to facilitate client-server communication, allowing users to perform operations like reading, writing, and creating files on remote servers.

Currently, there are three versions of NFS. NFS version 2 (NFSv2) is older and widely supported.

NFS version 3 (NFSv3) supports safe asynchronous writes and is more robust at error handling than NFSv2; it also supports 64-bit file sizes and offsets, allowing clients to access more than 2Gb of file data.



NFS version 4 (NFSv4) works through firewalls and on the Internet, no longer requires an rpcbind service, supports ACLs, and utilizes stateful operations.

All versions of NFS can use Transmission Control Protocol (TCP) running over an IP network, with NFSv4 requiring it. NFSv2 and NFSv3 can use the User Datagram Protocol (UDP) running over an IP network to provide a stateless network connection between the client and server.

5.4.1 Architecture of Network File System (NFS)

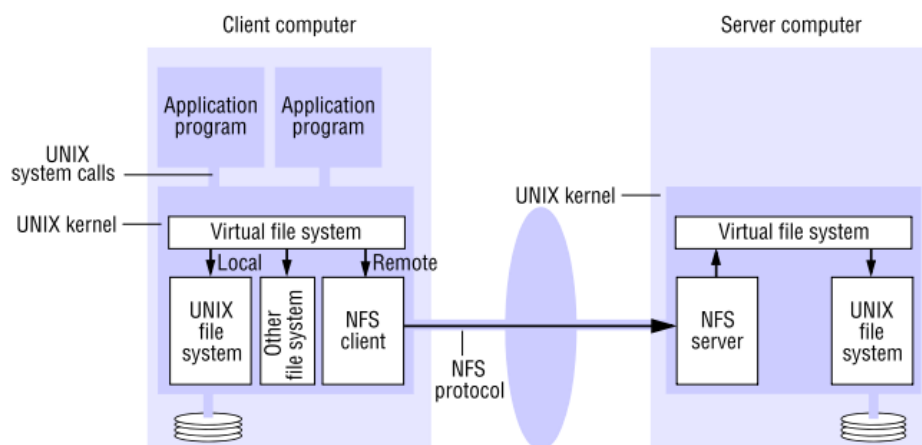


Figure 5.13 Architecture of Network File System (NFS)

1. Client Side:

Application programs: These are user applications that need to read or write files. They make file-related requests.
UNIX system calls: The applications interact with the file system by making system calls (like `open()`, `read()`, `write()`).
UNIX kernel: Provides the operating system services, including file system handling.
Virtual File System (VFS): A layer inside the kernel that gives a uniform interface for all types of file systems (local or remote).

2. Communication (Middle Part):

NFS protocol: The standard protocol used for communication between the client and server. It allows file operations like open, read, write, and close to work across the network.

3. Server Side:

UNIX kernel: The server also runs a UNIX-based operating system with its own kernel.
Virtual File System (VFS): Similar to the client, it abstracts the local file system structure.
NFS server: Listens for file operation requests from NFS clients. It interprets the NFS protocol messages and translates them into local file system operations.
UNIX file system: The actual file system where the files are stored on the server's disk.

4. Flow of Operation

Application on the client requests a file operation (via system call). The client's VFS decides whether it's a local or remote request. For remote requests, the NFS client sends the request via the NFS protocol to the server. The NFS server receives the request, processes it, and accesses the UNIX file system. The response (data or acknowledgment) is sent back to the client through the same path. To the application, it looks like the file is local—even though it is actually on the remote server.

5.5 Electronic Mail

Email has been around for more than 30 years. It is faster and cheaper than paper mail. Before 1990, email was mostly used in academic circles, but during the 1990s it became popular among the general public and grew rapidly. Today, the number of daily emails is much greater than the number of paper letters. Other communication methods like instant messaging and voice-over-IP have also grown, but email is still the main form of Internet communication.

Email is widely used in industries for communication within companies and to help employees in different parts of the world work together on projects. However, about 90% of email traffic is spam or junk mail.

Email communication is informal and easy to use. People often send emails even to very important persons, something they might not do with phone calls or letters. Email reduces the importance of rank, age, or gender, so discussions focus more on ideas than on status. This means a student's good idea can carry more weight than a bad idea from an executive.

5.5.1 The Architecture of an Email System

An email system is fundamentally composed of two main types of components: user agents and message transfer agents (mail servers). These components work together to facilitate the sending and receiving of electronic messages.

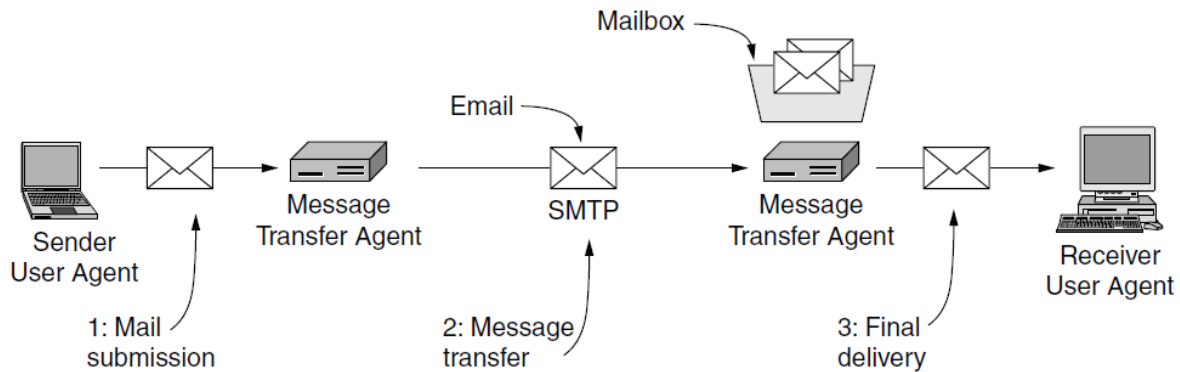


Figure 5.15 Architecture of an Email System

1. User Agent (UA): This is the program that an end-user interacts with to compose, read, reply to, forward, save, and manage their emails. Popular examples include Gmail, Microsoft Outlook, Mozilla Thunderbird, and Apple Mail. User agents provide a graphical interface or a text-based command interface. They typically have a menu-driven or touch-sensitive interface on mobile devices. User agents often allow for creating and manipulating mailboxes and message folders.

User agent responsibilities include preparing the message, interacting with mail servers, and displaying messages to the user. They can also handle automatic replies, such as those from a "vacation agent"

2. Message Transfer Agent (MTA) / Mail Server: These servers are responsible for receiving messages from user agents, storing them, and forwarding them to other mail servers or delivering them to the recipient's mailbox. Mail servers run continuously in the background. The Simple Mail Transfer Protocol (SMTP) is the primary protocol used by message transfer agents for sending and delivering email.

3. Mailbox: A mailbox is essentially a storage area on a mail server where incoming emails for a specific user are held. User agents retrieve messages from this mailbox. Mailboxes can contain various message folders like "Inbox," "Junk Mail," and user-created folders.

5.6 TELNET:

TELNET is basically the short form for TErminAl NETwork. It is basically a TCP/IP protocol that is used for virtual terminal services and was mainly proposed by International Organization for Standards (ISO).

It is an application-layer protocol that operates using the **client-server model**. The client is the system initiating the connection, while the server is the system being accessed remotely.

Telnet provides users with the ability to log in to a remote computer and execute commands as though they were physically present at that system. It uses **port 23** by default and supports full-duplex communication, meaning data can flow simultaneously in both directions.

Telnet was introduced in **1969 as part of the ARPANET project**, making it one of the oldest internet protocols. It played a critical role in the early days of networking by enabling remote system management and troubleshooting. Its simplicity and lightweight design made it an essential tool for system administrators.

However, one major drawback of Telnet is that it **transmits data in plain text** without any form of encryption. This includes sensitive information such as usernames, passwords, and all command data. As a result, Telnet is highly vulnerable to **cyberattacks** like **packet sniffing** and **man-in-the-middle attacks**, making it unsuitable for modern public networks.

Example:

telnet mail.example.com 25

This command checks whether the email server is accepting connections on port 25.

Working of Telnet

Telnet establishes a session through the following steps:

- **Connection Establishment** – The Telnet client initiates a connection to the Telnet server using **TCP port 23**.
- **Option Negotiation** – The client and server negotiate settings like terminal type, character set, and display options using Telnet control codes.
- **Authentication** – The user provides login credentials (username and password) in plain text.
- **Session Creation** – After authentication, the user gains access to the command-line interface of the remote system.
- **Command Execution** – Commands entered on the client are executed on the remote system.
- **Termination** – The session ends when the user logs out or disconnects.

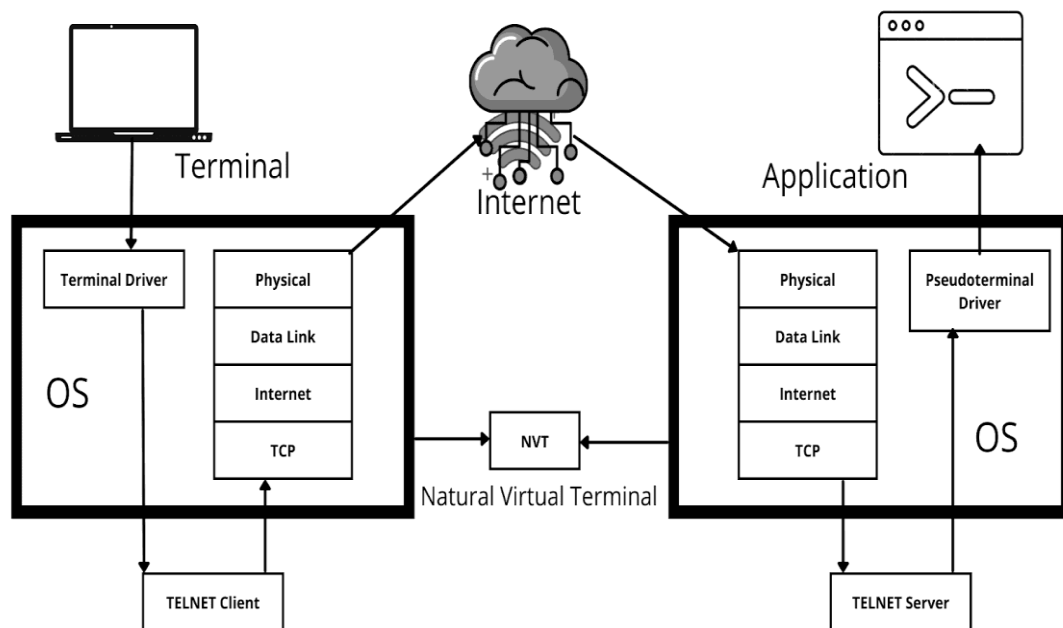


Figure 5.16 Architecture of TELNET using Natural Virtual Terminal (NVT)

Logging

As in the timesharing environment, the users are a part of the system having some rights in order to access the resources. It is important to note that each user has an identification along with the password. This user identification mainly indicates that the user is a part of the system.

In order to access the system the user logs in to the system by the **user-id**. The system also includes **password checking** in order to prevent unauthorized access to the resources of a system.

The logging process can be further categorized into two:

1. Local log-in
2. Remote log-in

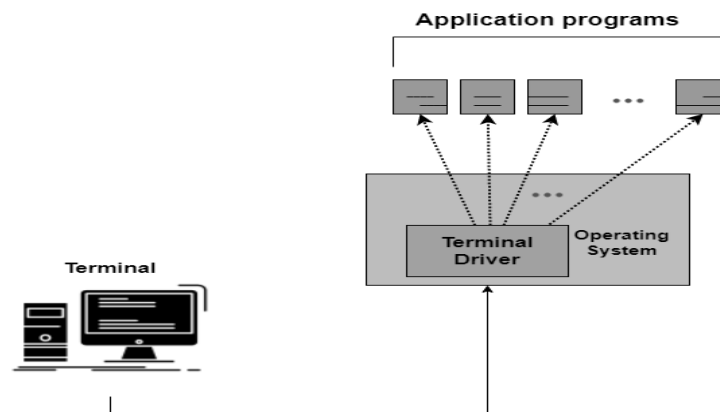


Figure 5.17 Local Log-in

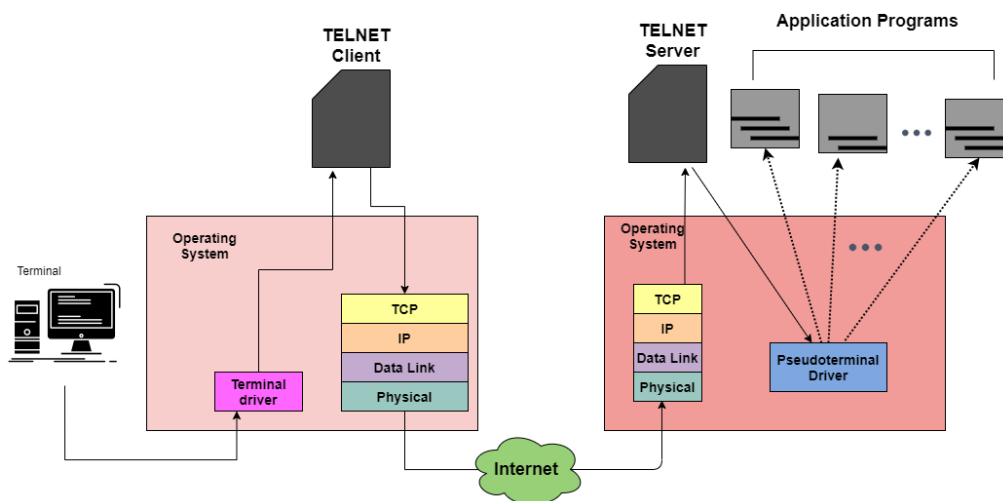


Figure 5.18 Remote Log-in

5.7 Secure Shell (SSH)

Secure Shell (SSH) is a secure communication protocol that enables encrypted interaction between two systems over an untrusted network, such as the Internet. Unlike earlier protocols that transmitted information in plain text, SSH ensures privacy and data protection by encrypting every exchange. This makes it a preferred tool for remote administration, file transfer, and system management, especially for administrators and security professionals.

Importance of SSH:

- **Security:**
SSH uses advanced encryption standards like AES or ChaCha20 to secure communication, preventing unauthorized interception..
- **Authentication:**
SSH verifies both the client and server before establishing a connection. Authentication can be done using passwords or cryptographic key pairs, ensuring that only authorized users can access the server.

- **Data Integrity:**
SSH ensures that data is not altered during transmission using hashing algorithms like SHA-2. Any tampering or corruption of data can be detected immediately.
- **Confidentiality:**
All information exchanged through SSH remains private, protecting sensitive data from unauthorized access.

5.7.1 How SSH Works (Step by Step):

Client-Server Architecture:

- **Client:** Device initiating the connection (e.g., laptop).
- **Server:** Device receiving the connection (e.g., remote server).

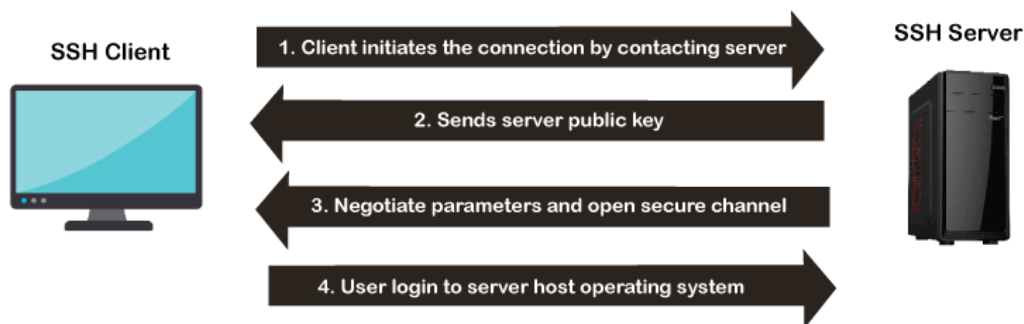


Figure 5.20 Client-Server Architecture

Connection Steps:

1. **Handshake:** The client sends a connection request, and the server provides its host key for verification.
2. **Authentication:** The client authenticates using a password or key pair.
3. **Session Establishment:** After successful authentication, an encrypted session is created.
4. **Data Transfer:** All commands, files, and communication are securely transmitted through this encrypted session.

BEFORE SSH:

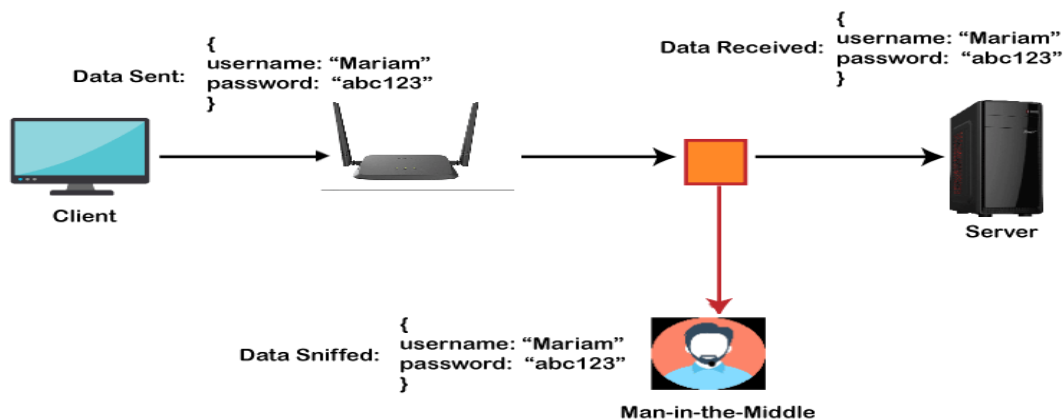


Figure 5.22 Before SSH

After SSH:

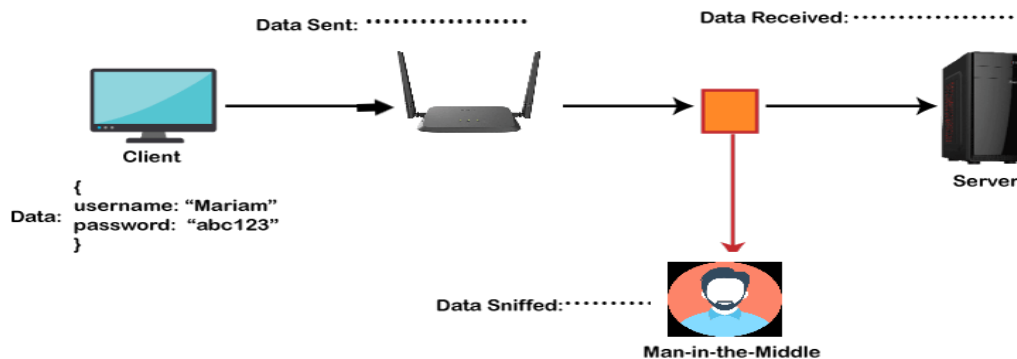


Figure 5.23 After SSH

5.7.2 The architecture of SSH Protocol

The SSH architecture is made-up of three well-separated layers. These layers are:

1. Transport Layer
2. User-authentication layer
3. Connection Layer

The SSH protocol architecture is an open architecture; hence it provides great flexibility and enables SSH use for many other purposes instead of only a secure shell. In the architecture, the transport layer is similar to the transport layer security (TLS). The User-authentication layer can be used with the custom authentication methods, and the connection layer allows multiplexing different secondary sessions into a single SSH connection.

Transport Layer

The transport layer is the top layer of the TCP/IP protocol suite. For SSH-2, this layer is responsible for handling initial key exchange, server authentication, set up encryption, compression, and integrity verification. It works as an interface for sending and receiving plaintext packets with sizes up to 32, 768bytes.

User authentication Layer

As its name suggests, the user authentication layer is responsible for handling client authentication and provides various authentication methods. The authentication is done at the client-side; hence when a prompt occurs for a password, it usually for an SSH client rather than a server, and the server responds to these authentications.

This layer includes various methods of authentication; these methods are:

- **Password:** Password authentication is a straightforward way of authentication. It includes the feature to change the password for easy access. But it is not used by all the applications.

- **Public-key:** The public-key is a public key-based authentication method, which supports DSA, ECDSA, or RSA keypairs.
- **Keyboard-interactive:** It is one of the versatile authentication methods. In this, the server sends a prompt to enter information & the client sends it back with keyed-in responses by the user. It is used to provide a one-time password or OTP authentication.
- **GSSAPI:** In this method, the authentication is performed by external methods such as Kerberos 5 or NTLM, which provide the single sign-on capability to SSH sessions.

Connection Layer

The connection layer defines various channels through which SSH services are provided. It defines the concept of channels, channel requests, and global requests. One SSH connection can host different channels simultaneously and can also transfer data in both directions simultaneously. Channel requests are used in the connection layer to relay out-of-band channel-specific data, for example, the altered size of a terminal window or the exit code of a server-side process. The standard channel types of connection layer are:

shell: It is used for terminal shells, SFTP, and exec requests.

direct-tcpip: It is used for the client-to-server forwarded connections.

forwarded-tcpip: It is used for the server-to-client forwarded connections.

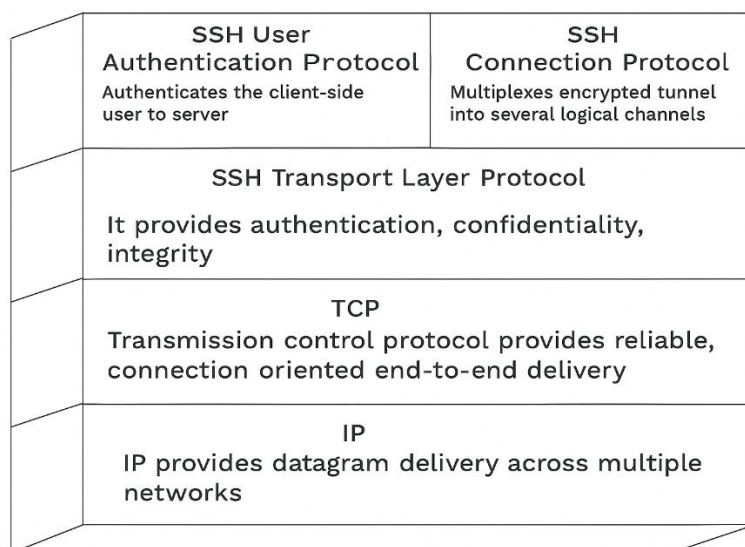


Figure 5.24 Layered Architecture of SSH Protocol

5.8 DNS (Domain Name System)

DNS (Domain Name System) is a hierarchical and distributed naming system that translates human-readable domain names (like *example.com*) into machine-readable IP addresses (like *192.0.0.1*), enabling users to access websites and other resources on the Internet.

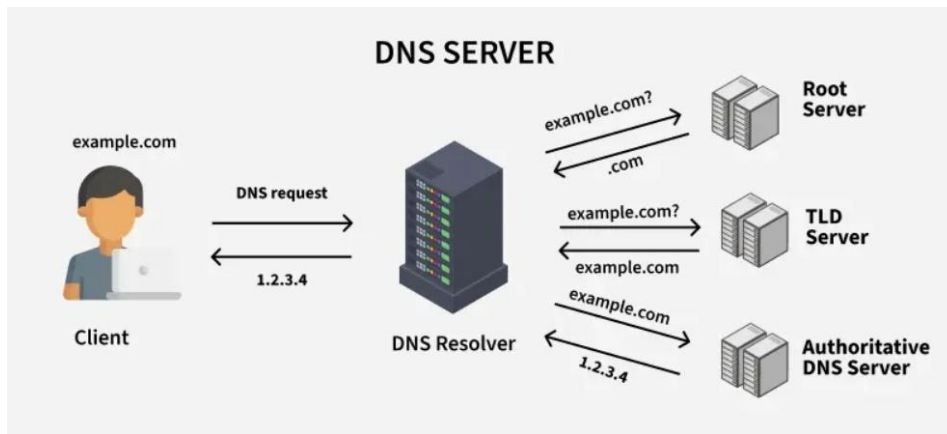


Figure 5.25 DNS Server

Client (User's computer)

- The user types `example.com` in the browser.
- A DNS query is first sent to the DNS Resolver.

DNS Resolver

- The resolver is responsible for locating the IP address of a domain, such as `example.com`.
- It queries other DNS servers in sequence until it gets the final answer.

Root Server

- The resolver first asks the Root Server: "Do you know `example.com`?"
- The root server doesn't know the IP but replies with a reference to the TLD Server (e.g., `.com`).

TLD (Top-Level Domain) Server

- The resolver then asks the TLD Server about `example.com`.
- The TLD (Top-Level Domain) server then replies with the details of the Authoritative DNS Server for that domain.

Authoritative DNS Server

- The resolver finally asks the Authoritative DNS Server: "What is the IP of `example.com`?"
- This server responds with the actual IP address (e.g., `1.2.3.4`).

Back to Client

- Finally, the resolver retrieves the IP address (e.g., `1.2.3.4`) from the authoritative server and returns it to the client.
- Now the client can directly connect to the website server.

5.8.1 Structure Of DNS

The Domain Name System (DNS) functions using a layered structure, which helps maintain both scalability and dependability across the global internet. Its organization works as follows:

- **Root DNS Servers:** Positioned at the top of the hierarchy, these servers maintain information about where to locate the TLD servers. They play a vital role in directing DNS requests to the appropriate level.
- **TLD Servers:** These servers handle top-level domains such as *.com*, *.org*, *.net*, *.edu*, *.gov*, and others. Their role is to guide DNS queries toward the appropriate authoritative servers.
- **Authoritative DNS Servers:** They store the verified DNS records of domains and provide the exact IP addresses required to reach the correct websites.

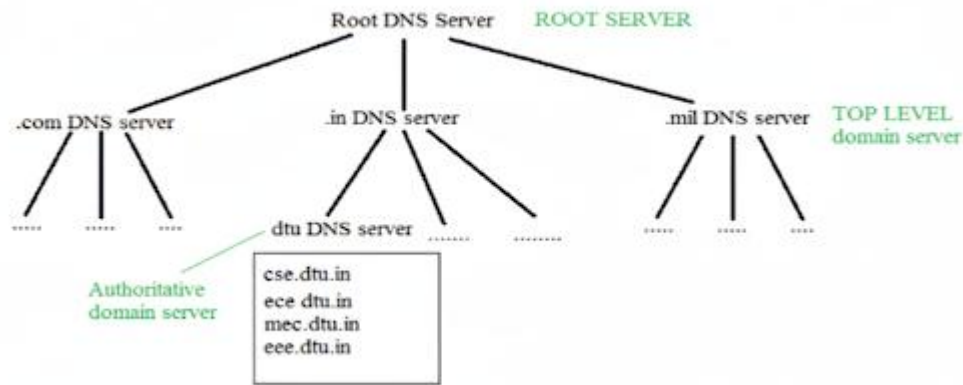
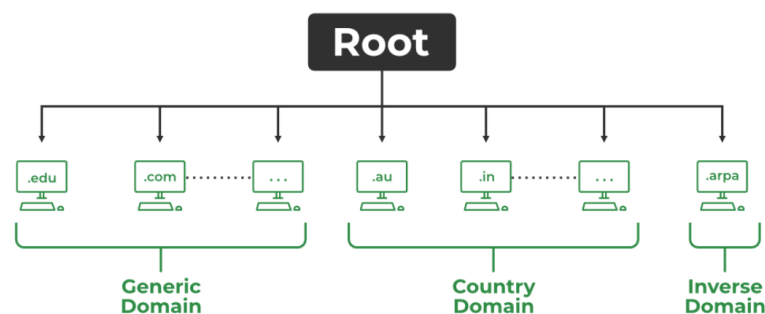


Figure 5.26 Structure of DNS

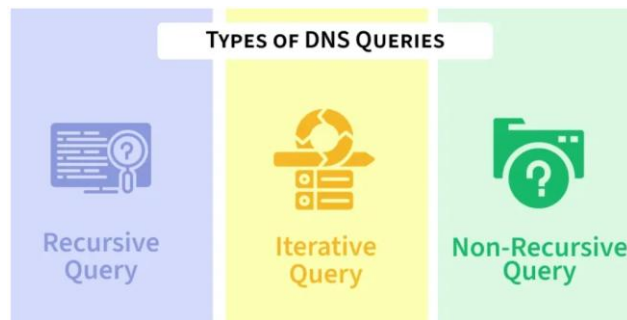
Types of Domain

The Domain Name System (DNS) organizes different types of domains to manage the huge number of websites available on the internet. The main types include:

- **Generic Domains:** These cover common top-level domains such as *.com*, *.org*, *.net*, and *.edu*. They are globally recognized and widely used.
- **Country Code Domains:** These represent particular countries or regions. Examples include *.in* for India, *.us* for the United States, *.uk* for the United Kingdom, and *.jp* for Japan.
- **Inverse Domains:** These are specifically used for reverse DNS lookups, where an IP address is translated back into its corresponding domain name.



Types of DNS Queries



Recursive Query: In this type of query, the DNS client relies on the DNS server to deliver a full answer. The server continues searching until it either finds the requested record or returns an error.

Iterative Query: With an iterative query, the DNS client asks the server for the best possible answer. Instead of searching further, the server responds with the most relevant information it has, which may include a referral to another DNS server closer to the answer.

Non-Recursive Query: A non-recursive query occurs when the DNS resolver already has the required record stored in its cache. Since the information is readily available, the DNS server can immediately reply without contacting other servers.

5G Network Utilization

Fifth Generation (5G) mobile networks mark a revolutionary leap in wireless communication. Unlike earlier generations, which focused primarily on improving voice and data services, 5G is designed to enable entirely new classes of applications through high speed, ultra-low latency, and massive device connectivity. The utilization of 5G extends far beyond smartphones—it forms the backbone for smart cities, healthcare innovations, Industry 4.0, and autonomous systems. Compared to existing 4G networks, 5G delivers enhanced speed, low latency, and improved reliability, positioning it as a transformative technology for internet-based services. Applications such as autonomous driving, high-performance gaming, and real-time multimedia streaming—which rely heavily on ultra-fast and stable connectivity—are projected to experience significant advancements through 5G integration.

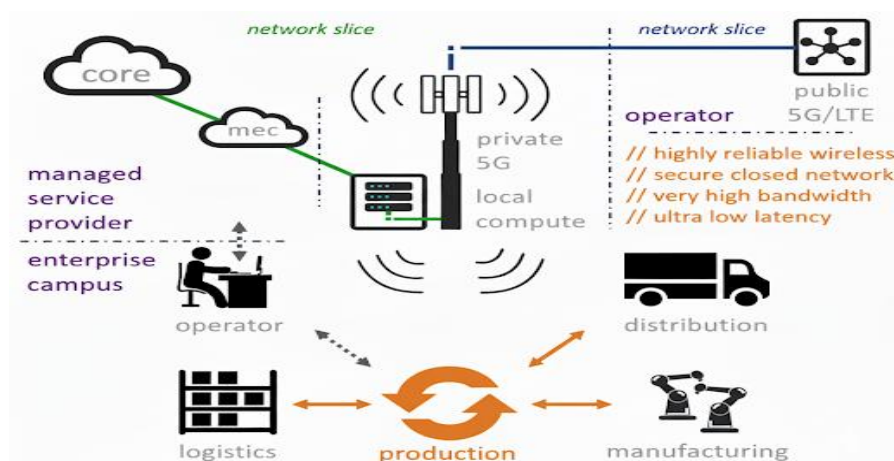


Figure 5.27 Private 5G Network Architecture for Enterprise Applications

5.9.1 Core Architectural Principles of 5G Networks

The fifth generation of cellular technology, 5G, represents a significant architectural evolution over its predecessors like 4G/LTE. While it continues to utilize a cellular model of sites transmitting data via radio waves, 5G introduces fundamental changes at both the physical and network layers to achieve its design goals of higher bandwidth, lower latency, and massive device connectivity. The core technological pillars enabling this transformation include a new radio interface, a denser network topology, and a virtualized network core.

Orthogonal Frequency Division Multiplexing (OFDM)

A cornerstone of the 5G New Radio (NR) interface is Orthogonal Frequency Division Multiplexing (OFDM). OFDM is a sophisticated digital modulation scheme that encodes data across multiple closely spaced, orthogonal sub-carrier frequencies. This approach offers several advantages over the modulation formats used in LTE: Spectral Efficiency: It allows for the efficient use of wide frequency bands, including the high-frequency millimeter wave (mmWave) spectrum, which was largely incompatible with previous cellular technologies.

Lower Latency: The flexible and scalable nature of the OFDM frame structure enables significantly reduced transmission time intervals, which is critical for achieving the ultra-low latency required by real-time applications.

Enhanced Flexibility: The 5G standard builds upon OFDM with configurable numerologies, allowing the network to adapt sub-carrier spacing and symbol duration dynamically to suit different services and deployment scenarios.

Dense Network Infrastructure and Small Cells

In contrast to 4G networks that relied primarily on large, high-power macro-cell towers, 5G architecture embraces a heterogeneous network (HetNet) model. This model heavily utilizes small cells—low-power, compact base stations that can be deployed on existing infrastructure such as streetlights, utility poles, and building facades. This dense deployment strategy is essential for two primary reasons:

Coverage for High-Frequency Bands: Higher frequency signals (e.g., mmWave) offer vast bandwidth but have a much shorter range and are easily obstructed. A dense grid of small cells is necessary to provide consistent coverage.

Capacity and Performance: By bringing the network access point closer to the user, small cells increase network capacity, reduce latency, and deliver the gigabit-per-second speeds promised by 5G, especially in areas with high user density.

Network Slicing

Perhaps the most revolutionary feature at the network layer is network slicing. This is a form of network virtualization that allows a mobile network operator to partition a single physical network infrastructure into multiple, independent, end-to-end virtual networks. Each "slice" is logically isolated and can be provisioned with its own unique set of resources and performance characteristics (e.g., bandwidth, latency, reliability, and security).

This enables operators to customize network capabilities for specific use cases or business verticals, such as:

Enhanced Mobile Broadband (eMBB): A slice optimized for high bandwidth to support applications like 4K video streaming and virtual reality.

Ultra-Reliable Low-Latency Communication (URLLC): A slice engineered for mission-critical tasks requiring near-instantaneous response and high dependability, such as industrial automation, remote surgery, or autonomous vehicle control.

Massive Machine-Type Communications (mMTC): A slice designed to support a vast number of low-power, low-bandwidth devices, typical of Internet of Things (IoT) deployments.

By logically separating services, network slicing ensures that the stringent requirements of one application do not negatively impact others, thereby enabling operators to offer guaranteed Quality of Service (QoS) and new business models on a common infrastructure.

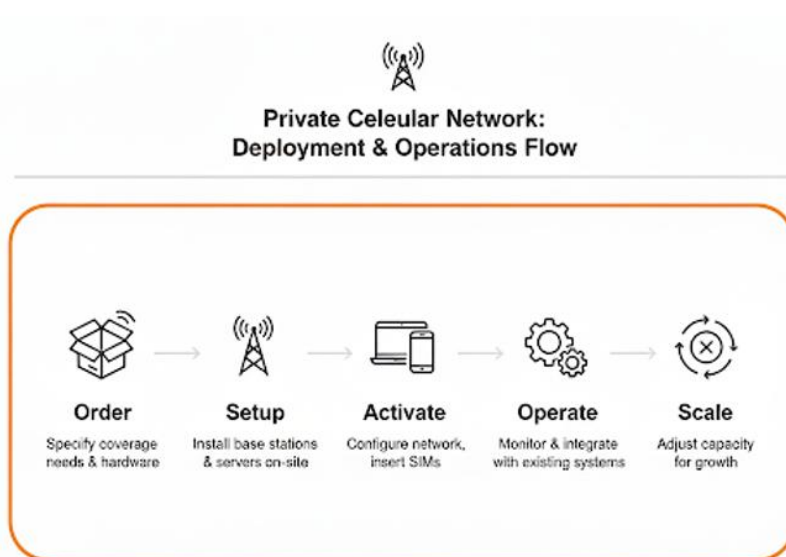


Figure 5.28 Private 5G Setup and Management Process

5.10 CONSTRAINED APPLICATION PROTOCOL (COAP)

The Constrained Application Protocol (CoAP) is an application-layer protocol that was introduced by the Internet Engineering Task Force in the year 2014. It was designed specifically for devices with limited processing power, memory, and energy resources, which are common in the Internet of Things (IoT). CoAP is a special web transfer protocol that operates with constrained nodes and networks. It is intended to be simple, low-overhead, and efficient, making it ideal for machine-to-machine communication.

It is a web-based protocol that resembles [HTTP](#). It is also based on the request-response model. Based on the REST-style architecture, this protocol considers the various objects in the network as resources. These resources are uniquely assigned a URI or [Uniform Resource Identifier](#).

CoAP is defined in RFC 7252 (2014) and extended in later RFCs like:

- RFC 7252: Core specification.
- RFC 7641: Observing resources.

- RFC 7959: Block-wise transfers.
- RFC 8323: CoAP over TCP, TLS, WebSockets.

In short, CoAP is like a small-scale version of HTTP designed for the IoT world.

5.10.1 Architecture of CoAP

The CoAP architecture follows a REST-based model similar to HTTP but optimized for constrained devices and networks. The diagram illustrates how CoAP enables communication between devices in the constrained environment and traditional Internet systems.

Key Components in the Architecture

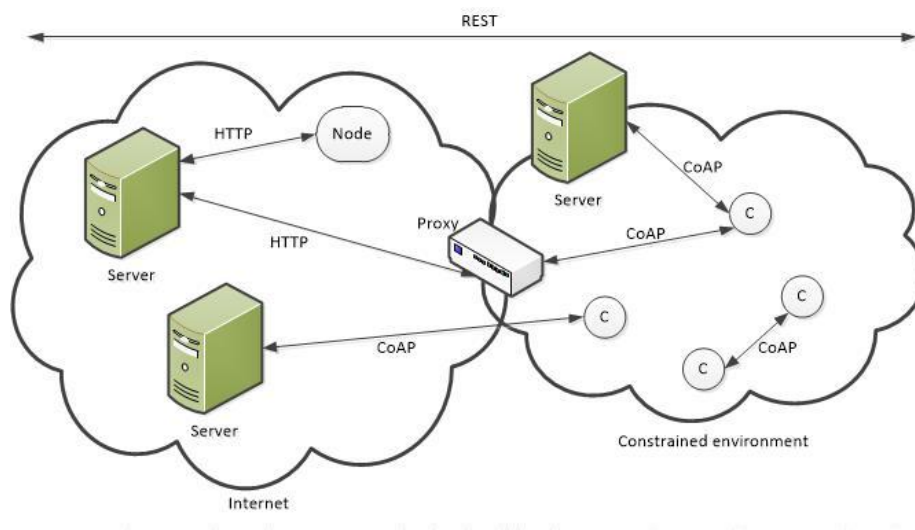


Figure 5.31 Architecture of CoAP

1. Client

- Initiates communication by sending a request (e.g., to fetch sensor data).
- Can use HTTP when communicating over the Internet or CoAP when communicating with constrained devices.

2. Server

- Provides resources (like sensor readings or actuator controls).
- Can respond to HTTP requests from normal Internet clients or CoAP requests from constrained clients.

3. Proxy

- Acts as a translator between HTTP and CoAP.
- Ensures interoperability between traditional web clients and IoT devices.
- Example: A mobile app using HTTP can request temperature data, which the proxy translates into CoAP for a constrained sensor.

4. Constrained Devices

- Small, resource-limited devices such as sensors or actuators.

- Use CoAP for lightweight communication among themselves and with the proxy.

5. Communication Flow

- HTTP (Internet side): Communication between standard clients (like browsers, apps) and servers.
- CoAP (Constrained side): Communication between IoT devices and proxy.
- RESTful model: Both HTTP and CoAP use REST principles (methods like GET, POST, PUT, DELETE), ensuring a uniform interface.

Internet side → HTTP

Constrained side → CoAP

Proxy → Bridges HTTP and CoAP

5.10.2 Methods of CoAP

The four primary request methods defined by CoAP, directly mapping to RESTful operations.

GET – Used to retrieve a resource representation from the server. (Read operation)

POST – Used to create a new resource or submit data to a resource.

PUT – Used to update or replace an existing resource on the server.

DELETE – Used to remove a resource from the server.

Message Types

1. Confirmable (CON) – Must be acknowledged.
2. Non-Confirmable (NON) – No acknowledgment required.
3. Acknowledgment (ACK) – Sent in response to a confirmable message.
4. Reset (RST) – Indicates message was received but could not be processed.

Message Format of CoAP

1. Header (4 bytes)

This is the fixed part of the message and includes:

- Version (2 bits): Usually set to 1
- Type (2 bits): Defines the message type:
 - ✓ Confirmable (CON)
 - ✓ Non-confirmable (NON)
 - ✓ Acknowledgement (ACK)
 - ✓ Reset (RST)
- Token Length (4 bits): Length of the token (0–8 bytes)

- Code (8 bits): Indicates the method (GET, POST, etc.) or response status
- Message ID (16 bits): Helps detect duplicates and match responses

2. Token (0–8 bytes)

Used to match a response with its request. Optional but useful.

3. Options (Variable length)

These are like headers in HTTP. Examples include:

- URI path (e.g., /sensor/temp)
- Content format
- Accept type

4. Payload Marker (1 byte)

A special byte 0xFF that signals the start of the payload. Only present if there is a payload.

5. Payload (Variable length)

The actual data being sent—like sensor readings or control commands.

Example: CoAP GET Request

Imagine a smart bulb system. A GET request to check its status might look like:

- Type: Confirmable
- Code: 0.01 (GET)
- Message ID: 12345
- Token: 0xA1B2
- Option: URI path = /bulb/status
- No payload (since it's a GET)

Example: CoAP Response

Imagine the smart bulb replying to the GET request with its current status:

- Type: Acknowledgement
- Code: 2.05 (Content)
- Message ID: 12345
- Token: 0xA1B2
- Option: Content-Format = text/plain
- Payload: "ON"

Communication Modes

- Piggybacked Response: Server sends ACK with response in the same packet.
- Separate Response: Server sends empty ACK first, then sends actual response later.

Advantages of CoAP

1. Lightweight protocol with very low overhead, suitable for constrained devices.
2. REST-based design (GET, POST, PUT, DELETE) makes it simple and easy to integrate.
3. Works over UDP, enabling faster and efficient communication.
4. Supports multicast communication, which is useful in IoT networks with many devices.
5. Energy-efficient and ideal for battery-powered IoT devices.

Limitations of CoAP

1. Reliability is limited since it runs over UDP (needs confirmable messages for reliability).
2. Security is weaker compared to HTTP; uses DTLS but increases complexity.
3. Limited support in traditional web applications compared to HTTP or MQTT.
4. Performance may drop in highly unstable or lossy networks.
5. Still evolving and not as widely adopted as other protocols.

Applications of CoAP

1. Smart Homes: communication between smart appliances like bulbs, fans, and thermostats.
2. Smart Cities: traffic monitoring, streetlight control, and waste management systems.
3. Healthcare IoT: patient monitoring through wearable sensors.
4. Environmental Monitoring: weather stations, pollution detectors, and water sensors.
5. Industrial IoT: machine-to-machine communication for automation and monitoring.

MQTT (Message Queuing Telemetry Transport)

MQTT (Message Queuing Telemetry Transport) is a lightweight protocol widely used in the Internet of Things (IoT) for exchanging data between devices. It is especially effective where internet connections are unreliable, bandwidth is limited, or devices need to share information in real time.

Unlike direct device-to-device communication, MQTT uses a **publish–subscribe model**, which allows many devices to send or receive data through a central broker without needing to know about each other.

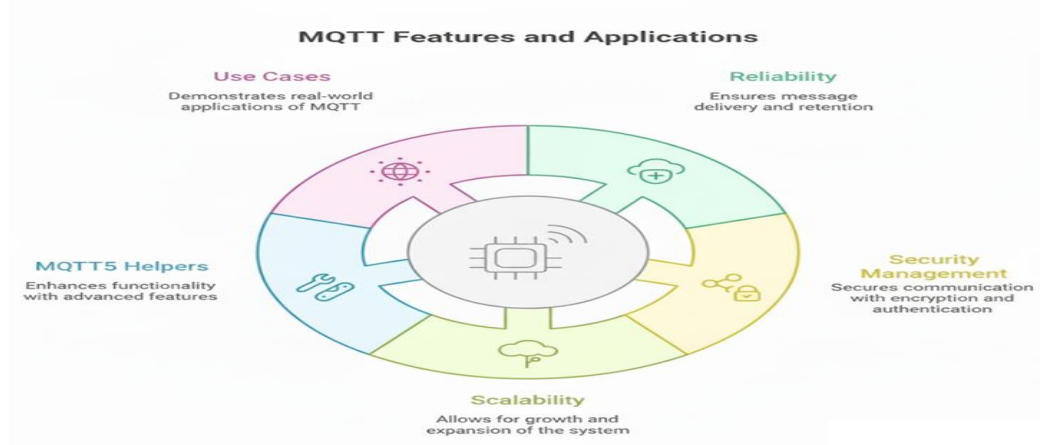


Figure 5.32 MQTT Features and Applications

Main Building Blocks

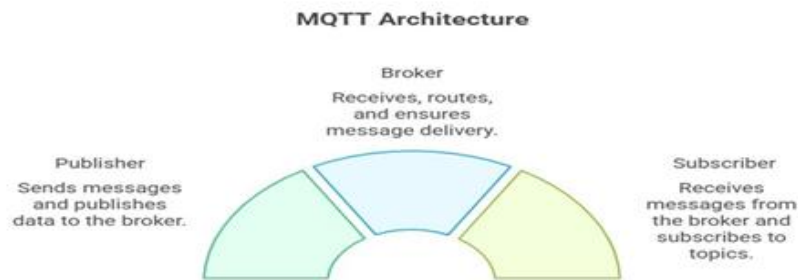


Figure 5.33 MQTT Architecture

- **MQTT Clients:** Any device or application running an MQTT library that can connect to a broker. Each client may act as a publisher, a subscriber, or both. For example, a temperature sensor (publisher), a mobile app (subscriber), or a hybrid device that does both.
- **MQTT Broker:** The central dispatch service for all messages. The broker receives published messages and distributes them to all subscribers registered for that topic. It also manages security, message retention, persistent sessions, and client authorization.
- **MQTT Topics:** Named channels to categorize messages. Topics are hierarchical (e.g. "house/kitchen/temperature"), flexible, and allow filtering so that subscribers only receive relevant data.

5.11.2 How Clients and Broker Communicate

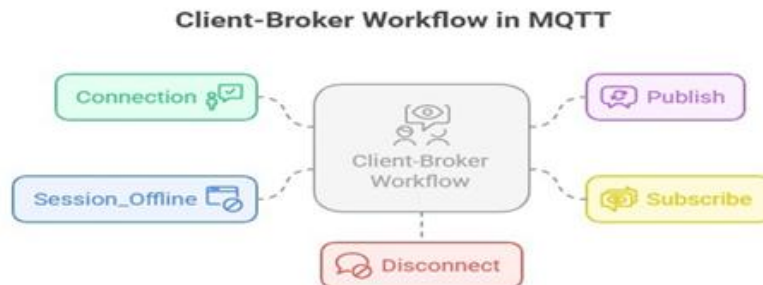


Figure 5.34 Client Broker Workflow

1. **Connecting** – A client opens a TCP/IP connection and sends a request with details such as client ID, version, and login data. The broker replies with an acknowledgment if the connection is accepted.
2. **Session Control** – The broker can keep track of subscriptions and missed data, allowing clients to disconnect and reconnect without losing important information.
3. **Publishing** – Devices send messages tagged with a topic. Each message also includes a delivery quality setting that specifies how reliably it should be sent.
4. **Subscribing** – Clients express interest in specific topics, and the broker forwards only those messages to them.
5. **Disconnection** – Clients can exit cleanly, and the broker either clears or retains session data depending on preferences.

How MQTT works:

MQTT uses a publish/subscribe (pub/sub) communication model designed to optimize bandwidth usage, offering an alternative to the traditional client-server approach where communication happens directly between endpoints. In this model, the client sending data (the publisher) and the client receiving data (the subscriber) do not interact with each other directly. Instead, an intermediate entity called the broker manages the exchange between them.

An MQTT client can function as either a publisher or a subscriber—or even both. When a client wants to share data, it publishes the information to a broker. Conversely, when a client wishes to receive data, it subscribes to specific topics of interest managed by the broker. Multiple clients can subscribe to the same broker, enabling flexible and scalable communication.

If a subscribing client temporarily loses connection, the broker retains the incoming messages in a buffer and delivers them once the connection is restored. Similarly, if a publisher disconnects unexpectedly, the broker can cache instructions and forward them to the subscribers when possible.

To summarize, the workflow can be described as:

- Publishers generate and send messages.
- Subscribers receive messages based on their chosen topics.
- Brokers serve as intermediaries, ensuring the correct delivery of messages from publishers to subscribers.

Any device or application running an MQTT library can act as a client, from lightweight microcontrollers like Arduino to large-scale cloud-based application servers.